CMF 3D-QSAR

The continuous molecular fields approach, as a program

Desktop application, library and command-line tool
for the Continuous Molecular Fields approach to 3D-QSAR
User manual

`github.com/igbaskin/cmf-qsar`

Contents

What this program is 5

- The idea in one page 5
- What you can do with it 5
- How this manual is arranged 6

How a prediction is made 7

- 1. The fields 7
- 2. The overlap integral 7
- 3. Normalisation and scaling 8
- 4. Mixing the fields 8
- 5. Fitting: what is learned from the training set 8
- 6. Predicting a new molecule 9
- The same prediction, as a field 9
- A worked example 10
- What changes for the other tasks 10
- What has to match between fitting and predicting 11

Installation and first run 13

- Where the program lives 13
- Requirements 13
- Starting the application 13
- A first model in five minutes 14

The application window 15

- The Data set panel 15
- The Model panel 16
- The 3D view 17
- The Results panel 18
- The menus 19

Preparing the data 20

- What the program reads 20
- Activities 20
- Alignment 20
- Judging an alignment, and aligning flexibly 21
- Charges and per-atom properties 23
- Quantum-chemical properties with ORCA 24

Before you model: `cmf diagnose` 26

- Two things the command protects you from 26
- Reading the answer 27
- Does the advice work? 27

Building a model 28

Choosing the fields 28
GRID-like probe fields 28
Weighting the fields by solvent exposure 29
Descriptors that are not fields 30
Attenuation, regularisation and the grids 32
How long the kernel takes, and the three settings that change it 32
The learners 35
Fitting the order instead of the values: the ranking SVM 35
Nonlinear kernel layers and multiple kernel learning 37
Which fields does this series need? 37
Choosing α and λ without holding anything out 38
Validation 40
Is it chance? y-randomisation 41
Where does the model apply? 41
Is the whole library inside? The two-sample test 42
How certain is that number? 42
How certain is *one* prediction? Conformal intervals 43
Three estimates of the error, compared 44
What the model thinks the groups are: clustering the kernel 45
Where the validation appears 46
Statistics reported 46
Saving and reusing 47
A report to hand to a colleague 47

Reading the model 49

Co-fields 49
Contributions 50
Shapley values 50
A pharmacophore out of the co-field 51
What a candidate is missing 52
Several endpoints at once: the selectivity co-field 52
Comparing two models by their co-fields 56
Chemical space maps 57

Classification and virtual screening 59

Setting up a classification task 59
One-class models: screening with actives alone 59
Decoys and the figures that matter 59
Measured inactives, and learning from positives alone 60
Screening a library that does not fit in memory 62
Which molecule to make next 63
Designing the next compound 65

Conformations: the fourth dimension 67

Working without an alignment 68

Why the frame is the problem 68

The pose-free count kernel 68
The rotation-invariant autocorrelation kernel 68
Molecules on a surface: the in-plane invariant kernel 71
Crystals: the periodic autocorrelation kernel 72
Topomer fields 73
What the three routes are worth, measured 74
Heterogeneous series, step by step 75

Protein pockets 76

Getting a pocket on screen 76
Complementarity as numbers 76
Docking 77

Command-line reference 78

The options of `build`, by group 78
Worked examples 79

Using it as a library 81

The example data sets 82

When something goes wrong 83

Appendix A — Field parameterisation 84

Appendix B — Files and tags the program understands 85

Appendix C — Glossary 86

Appendix D — Bibliography 90

The method 90
One-class classification and virtual screening 90
Methods and data taken from other work 90

What this program is

CMF 3D-QSAR is a desktop application, a Python library and a command-line tool implementing the **Continuous Molecular Fields** approach to three-dimensional structure–activity modelling. The method was introduced by Zhokhova, Baskin and co-workers (Dokl. Chem. **429** (2009) 273; J. Comput.-Aided Mol. Des. **27** (2013) 427; continuous indicator fields: J. Comput.-Aided Mol. Des. **29** (2015) 233; co-fields: Future Med. Chem. **11** (2019) 2701), and the program follows those papers closely enough that their published statistics can be reproduced from the same data — `docs/benchmark.md` records that comparison set by set.

The idea in one page

A molecule is represented not by a table of descriptors and not by values on a lattice, but by **continuous functions of space**. Each field is a sum of Gaussians centred on the atoms,

$$X_f(\mathbf{r}) = \sum_l w_{lf} \exp\left(-\frac{\alpha_f}{2} |\mathbf{r} - \mathbf{r}_l|^2\right)$$

where w_{lf} is the weight of atom l in field f — its partial charge for the electrostatic field, its Crippen logP contribution for the hydrophobic field, 1 for an indicator field of a given atom type, or any number the file carries per atom — and α_f is the attenuation factor that sets how far the field reaches.

Two molecules are compared by the **overlap integral** of their fields, which for Gaussians has a closed form:

$$K_f(A,B) = \int X_{fA}(\mathbf{r})\, X_{fB}(\mathbf{r})\, \mathrm{d}\mathbf{r} = \left(\frac{\pi}{\alpha}\right)^{3/2} \sum_l \sum_m w_{lf} w_{mf} \exp\left(-\frac{\alpha}{4} |\mathbf{r}_l - \mathbf{r}_m|^2\right)$$

No grid is ever evaluated for this: the kernel is a sum over atom pairs, computed exactly. The kernels of the individual fields are mixed with coefficients h_f into one kernel, and that kernel goes into a kernel method — ridge regression, a support vector machine, a Gaussian process. Three consequences follow, and they are what distinguishes the approach from CoMFA:

- **Nothing is discretised.** There is no lattice spacing to choose, no alignment of grids, no variable selection among tens of thousands of columns, and no truncation at a cut-off distance.
- **The model is a function of space again.** A kernel ridge model predicts $y(X) = \Sigma_j a_j K(X, X_j) + b$, and because the kernel is an integral of a product, the whole sum can be rewritten as the overlap of the molecule's field with one function, the **co-field** $C_f(\mathbf{r}) = \Sigma_j a_j X_{fj}(\mathbf{r})$. The co-field is the program's map: it plays the role of CoMFA's coefficient contour, but it is the model rather than a projection of it.
- **The decomposition is exact.** Because the prediction is bilinear in the fields, it can be split without approximation into contributions of training molecules, of field types, of regions of space, or of individual atoms.

What you can do with it

Task	How
Build a 3D-QSAR regression model on an aligned congeneric series	Model panel, or `cmf_cli.py build`
Work without an alignment	topomer fields, the rotation-invariant autocorrelation kernel, or the pose-free count kernel
Classify actives against inactives, or screen with actives alone	binary (SVC, ν-SVC, Laplace GP) and one-class (1-SVM, centroid, k-NN, GP, kernel PCA) tasks
See what the model says about space	co-fields as isosurfaces, molecular fields, overlap maps, Gaussian-cube export
See what the model says about a molecule	training-molecule contributions, atom contributions, Shapley values and interaction indices
Turn a model into design instructions	pharmacophore extraction from co-field lobes, field-difference maps against a reference ligand
Use the protein	fetch or load a pocket, draw it behind the fields, score field complementarity, dock
Average over conformations	conformational ensembles (4D) and multi-instance learning
Map a data set	kernel GTM chemical-space maps with activity, class and density landscapes

How this manual is arranged

Chapter 2 follows one prediction from the atoms to the number, which is the method in full. Chapters 3 and 4 get you running and name every part of the window. Chapters 5 to 7 are the working cycle: prepare the data, fit a model, read it. Chapters 8 to 11 are the specialised routes — screening, conformations, alignment-free modelling, protein pockets. Chapter 12 is the complete command-line reference, chapter 13 the library, chapter 14 the example data, and chapter 15 collects the things that go wrong. The appendices hold the field parameterisation, the file conventions, a glossary of the terms this manual uses in a particular sense, and the bibliography.

Conventions: *Menu → Item* names a menu command, `monospace` names a file, an option or a piece of code. Equations are quoted in the notation of the papers, with their equation numbers.

How a prediction is made

This chapter follows one number — the activity predicted for one molecule — from the atoms it starts in to the value that appears in the table. Nothing here is a sketch: every formula is the one the program evaluates, and the worked example at the end was computed by running it.

The chain has six links:

1. each molecule becomes a set of **fields**, one per field type, each a sum of Gaussians on its atoms;
2. every pair of molecules gets a **field kernel** per field type, the overlap integral of those two fields, computed in closed form;
3. each field kernel is **normalised** and **scaled**;
4. the field kernels are **mixed** into one kernel matrix;
5. the learner turns that matrix and the activities into **dual coefficients**;
6. a new molecule is pushed through steps 1 to 4 against the training set, and its prediction is the **weighted sum of its kernel values** with those coefficients.

1. The fields

Atom l of a molecule carries a weight w_{lf} in field f — its partial charge in the electrostatic field, its Crippen logP contribution in the hydrophobic field, an Abraham-type increment in the hydrogen-bond fields, the Tripos van der Waals well depth in the steric field, 1 in an indicator field for atoms of that type and 0 elsewhere, or a number read from the file in a `prop:` field. Appendix A lists them.

The field itself is the sum of Gaussians those weights define,

$$X_f(\mathbf{r}) = \sum_l w_{lf} \exp\left(-\frac{b_{lf}}{2}|\mathbf{r}-\mathbf{r}_l|^2\right)$$

with width $b_{lf} = \alpha_f$ for every field except the steric one, where it is scaled by the atom's van der Waals radius, $b_{lf} = \alpha_f / R_l^2$, so that a large atom spreads its bulk over a correspondingly larger region.

There is no grid. The field is never evaluated anywhere in order to make a prediction; it is evaluated only when a map has to be *drawn*.

2. The overlap integral

Two Gaussians have an overlap integral in closed form, so the inner product of two whole fields is a double sum over atom pairs:

$$K_f(A,B) = \int X_{fA}(\mathbf{r}) X_{fB}(\mathbf{r})\,\mathrm{d}\mathbf{r} = \sum_{l\in A}\sum_{m\in B} w_{lf} w_{mf} \left(\frac{2\pi}{b_{lf}+b_{mf}}\right)^{3/2} \exp\left(-\frac{b_{lf}b_{mf}}{2(b_{lf}+b_{mf})}|\mathbf{r}_l-\mathbf{r}_m|^2\right)$$

When every atom shares one width, $b_{lf} = b_{mf} = \alpha$, this collapses to the form the papers quote,

$$K_f(A,B) = \left(\frac{\pi}{\alpha}\right)^{3/2} \sum_l \sum_m w_{lf} w_{mf} \exp\left(-\frac{\alpha}{4}|\mathbf{r}_l-\mathbf{r}_m|^2\right)$$

and the general form is what the steric field needs, since its widths differ atom by atom. A unit test checks both against numerical quadrature of the fields on a fine grid.

Two things follow immediately. The cost of a kernel is the number of atom pairs, not the number of grid points, which is why a 300-molecule set takes seconds. And the attenuation α is the only thing that decides how far apart two atoms may be and still contribute: large α demands close matching, small α lets distant atoms see each other, and in the limit α → 0 the geometry drops out altogether and the kernel becomes a product of atom counts — which is the alignment-free baseline of *Working without an alignment*.

3. Normalisation and scaling

Two corrections stand between the raw integrals and a usable kernel, and they do different jobs.

Cosine normalisation (*Normalise kernels*) divides each field kernel by the norms of the two fields,

$$\hat{K}_f(A,B) = \frac{K_f(A,B)}{\sqrt{K_f(A,A)\,K_f(B,B)}}$$

so that every molecule has unit self-similarity in every field -- the normalised kernel of a molecule with itself is 1 by construction -- and a large molecule cannot look important merely for being large.

Field scaling (*Scale fields*) then multiplies each field kernel by a constant c_f chosen so that its mean diagonal over the training set is 1. It equalises the fields' contributions before they are mixed, which matters when one field's raw magnitudes dwarf another's — an electrostatic kernel built from partial charges and a steric kernel built from well depths are not naturally on the same scale. With cosine normalisation on, every diagonal is already 1 and c_f = 1; the two options are therefore alternatives in practice, and unticking both gives the raw-kernel convention of the papers.

The scaling constants are computed **on the training set and then stored**: a prediction must not rescale anything according to the molecules being predicted, or two test molecules would be scored on different scales.

4. Mixing the fields

The joint kernel is a weighted sum over field types,

$$K(A,B) = \sum_f h_f\, c_f\, \hat{K}_f(A,B)$$

with the mixing coefficients h_f left at 1 by default, edited by hand in the field table, chosen by coordinate descent on the cross-validated error, or set by centred kernel alignment. Since each normalised field kernel has unit diagonal, the joint self-kernel of any molecule is $\Sigma_f\, h_f\, c_f$ — five, for the five default fields with unit coefficients, which is the 5 on the diagonal of the matrix in the worked example below.

5. Fitting: what is learned from the training set

Kernel ridge regression with a bias term solves one linear system (Eq. 12 of the 2013 paper):

$$(\mathbf{K} + \lambda \mathbf{I})\,\mathbf{a} + b\,\mathbf{1} = \mathbf{y}, \qquad \mathbf{1}^{\mathrm{T}}\mathbf{a} = 0$$

The outcome is one **dual coefficient** a_j per training molecule and one bias b. The last row of the system is the constraint $\Sigma_j\ a_j = 0$, which is what makes the bias meaningful: the coefficients describe only *departures* from a constant level, and b carries the level itself. The ridge parameter λ adds to the diagonal, which keeps the system solvable when molecules are nearly identical — as they are in a congeneric series, where the off-diagonal kernel values run above 0.97 — and controls how closely the model follows the training activities.

The other learners differ only in how **a** and b are obtained, never in what they are: ε-SVR and ν-SVR get them from a quadratic program in which most a_j come out exactly zero, C-SVC and ν-SVC the same for class labels, and the Laplace Gaussian-process classifier from the posterior mode of a latent function. In every case the fitted model is the pair (**a**, b) over the training molecules.

6. Predicting a new molecule

For a molecule the model has never seen, the program:

1. builds its fields **with the same field set** — the same α, the same charge model, the same parameterisation, the same atom typing;
2. computes the per-field overlap integrals between its atoms and the atoms of every training molecule, giving one row $K_f(x, j)$ per field;
3. normalises that row with the molecule's own self-kernels and the **stored** training self-kernels, and multiplies by the **stored** scales c_f;
4. mixes those normalised, scaled fields into one row $k(x)_j$ with the mixing coefficients h_f, which is the joint kernel $K(x, j)$ of *Mixing the fields* above;
5. returns

$$\hat{y}(x) = \sum_j a_j\, k(x)_j + b = \mathbf{k}(x)^{\mathrm{T}}\mathbf{a} + b$$

That is the whole prediction: a similarity-weighted combination of the training activities, where "similarity" is the overlap of molecular fields and the weights were fixed when the model was fitted. Molecules with $a_j = 0$ — most of them, after an SVM fit — contribute nothing, which is why the *Contributions* tab can list the handful that actually make a given prediction.

The same prediction, as a field

Because the kernel is an integral of a product and the prediction is linear in it, the sum over training molecules can be moved inside the integral. Writing $s_j = 1/\sqrt{(K_f(j,j))}$ for the normalisation of training molecule j, define the **co-field**

$$C_f(\mathbf{r}) = \sum_j a_j\, s_j X_{fj}(\mathbf{r})$$

and the prediction becomes

$$\hat{y}(x) = \sum_f \frac{h_f c_f}{\sqrt{K_f(x,x)}} \int X_{fx}(\mathbf{r})\, C_f(\mathbf{r})\, \mathrm{d}\mathbf{r} + b$$

The model is therefore a *function of space*, one per field type, and a molecule is scored by how much of its own field lies in the positive regions of that function and how much in the negative

ones. This is the identity that makes the pictures of *Reading the model* the model rather than an illustration of it, and the division by the molecule's own field norm is the only thing the normalisation adds to the picture. It also explains what co-fields cannot survive: a nonlinear kernel layer, which is not an integral of a product, and the autocorrelation kernel source, which has no frame to draw in.

A worked example

Four molecules from `examples/benzamides_synthetic.sdf` as the training set, the five default fields, $\alpha = 0.5$, cosine normalisation, $\lambda = 0.1$, and the fifth molecule as the one to predict. The numbers below are the program's own, printed from the objects it fits.

The joint kernel of the training set, the sum of the five normalised field kernels, so the diagonal is 5:

```
                parent     4F      4'F     4Cl          y
parent          5.0000  2.8362  3.2195  2.5368       5.585
4F              2.8362  5.0000  2.3310  3.9647       5.230
4'F             3.2195  2.3310  5.0000  2.8882       6.471
4Cl             2.5368  3.9647  2.8882  5.0000       5.355
```

Solving the bordered system gives

```
a = [-0.1358, -0.0581, +0.3487, -0.1549]      sum(a) = 0      b = 5.7124
```

The test molecule 4'Cl is then compared with each training molecule, field by field:

```
                        parent      4F      4'F     4Cl
electrostatic           0.0878  0.6028  0.1009  0.3915
steric                  0.9793  0.9824  0.9863  0.9579
hydrophobic             0.8936  0.9412  0.9443  0.8437
H-bond acceptor         0.2600  0.9895  0.1927  0.6539
H-bond donor            0.5815  1.0000  0.4424  0.7379
                        ------  ------  ------  ------
k(x)  (their sum)       2.8022  4.5159  2.6667  3.5849
```

and the prediction is that row times the coefficients, plus the bias:

```
y(x) = 2.8022(-0.1358) + 4.5159(-0.0581) + 2.6667(0.3487) + 3.5849(-0.1549) + 5.7124
     = 5.4444
```

which is exactly what *Pred (test)* shows for that molecule. Two things can be checked directly in this example rather than taken on trust. The steric and hydrophobic rows are nearly constant across the training set — these molecules differ by one halogen — so almost all the discrimination comes from the electrostatic and hydrogen-bond fields, which is visible in the spread of their rows. And the prediction, 5.44 against an observed 7.21, is poor, because four training molecules and no hyper-parameter search is not a model: the arithmetic is the lesson here, not the accuracy.

What changes for the other tasks

Model	What the prediction is
Kernel ridge	$\hat{y} = \mathbf{k}^{\mathsf{T}}\mathbf{a} + b$, as above

Model	What the prediction is
Gaussian process	the same mean, plus the variance $\sigma^2(x) = K(x,x) - \mathbf{k}^{\mathsf{T}}(\mathbf{K}+\lambda\mathbf{I})^{-1}\mathbf{k}$, which the *Align by prediction confidence* tool minimises
ε-SVR, ν-SVR	the same formula; most a_j are zero
C-SVC, ν-SVC	the same formula, read as a decision value: its sign is the class, its magnitude the margin
Laplace GP classifier	the same formula gives the latent mean, which is squashed to a probability and averaged over the latent variance
One-class methods	a score built from the same kernel row, but also from the molecule's own self-kernel: distance to the centroid, to the k-th nearest active, to the principal subspace, or the 1-SVM decision value
Nonlinear kernel layer	the joint kernel is passed through $\exp(-\gamma\, d^2)$ or an arc-cosine function first; the prediction formula is unchanged, the co-field identity is lost
Autocorrelation source	the kernel row comes from the rotation-invariant distance profiles instead of the overlap integrals; everything downstream is identical
Surface source	for molecules over a surface at $z = 0$: the in-plane distance profiles resolved by the heights of both atoms, invariant in the plane and to rotation about the normal only; heights and tilts are kept, no in-plane alignment is needed (command line: `--kernel surface --surface-z ZMIN ZMAX ZSTEP`)
Periodic source	for crystals: the distance profiles summed over the neighbour list of the cell with its images, per atom, background removed, cutoff tapered; invariant to the choice of cell, origin and orientation (command line: `build-crystals`)
Two endpoints	the same formula once per endpoint, with coefficients tied through a task correlation; the difference of the two coefficient vectors gives the selectivity co-field

What has to match between fitting and predicting

Because the kernel row is computed against the stored training atoms with the stored constants, a prediction is only meaningful if the new molecule was prepared the way the training molecules were. The saved `.cmf` model therefore carries the field set, the α values, the charge model, the parameterisation, the normalisation and scaling flags, the mixing coefficients, the training atoms and the coefficients — everything in steps 1 to 5 — and reuses them. It also carries a record of how the training molecules were prepared: the version of the program that wrote it, the date, how many molecules, where their partial charges came from (the file, or computed), whether they carried explicit hydrogens, and the box their atoms occupy. Before any prediction the program compares the new molecules with that record and reports what does not match:

- **The chemistry behind the weights.** Charges read from a file for the training set and Gasteiger charges for the new molecules are two different electrostatic fields under one name. A model fitted on file charges therefore **refuses** a molecule that carries none, rather than falling back to Gasteiger charges silently; `cmf_cli.py predict --allow-mismatch` overrides the refusal, and the application asks. Supply the charges (a MOL2 charge column or a `partial_charge` atom property) or build the model with computed charges.
- **Explicit hydrogens.** The donor field sits on the hydrogens and the steric field counts them, so a set with hydrogens and a set without are two different sets of fields. A mismatch is a **warning**.

- **The frame.** The overlap integral is not invariant to rotation or translation, so a molecule predicted in a different frame from the training set scores low on every field and is predicted towards the mean. A molecule whose centroid lies more than 4 Å outside the box of the training atoms is reported as a **warning**.
- **The pose inside the frame.** A molecule can be in the right place and still be turned the wrong way, and a rotation about its own centre moves no centroid, so the box test above sees nothing. The check that does see it compares the molecule with the *shape* the training set occupies rather than with its box: the share of its atoms that come within 1.5 Å of some training atom, against the share this training set asks of its own molecules — the fifth percentile of the same quantity measured on each training molecule against the others, less a small slack, so that a series of close analogues gets a strict rule, exactly as the applicability domain's threshold does. Measured on the eight Sutherland sets, turning every held-out molecule 90° about its own centre is caught **95 % of the time, for a false-positive rate of 2.6 %** on the molecules left as they were. If the new molecules are not in the frame, align them (*Preparing the data*) or use a frame-free kernel source (*Working without an alignment*).
- **Per-atom property fields** that are zero on a molecule, because it carries no value of that property, are a warning; a file written by another version of the program, or by a version that recorded no provenance, is a note.

The findings appear in the log of the application and on the console of the command-line tool, each marked `[refuse]`, `[warn]` or `[note]`. A model file written by a newer version of the program than the one reading it is not opened.

Installation and first run

Where the program lives

The repository is github.com/igbaskin/cmf-qsar — private, so the link asks for a GitHub account with access:

```
git clone https://github.com/igbaskin/cmf-qsar.git
cd cmf-qsar
```

Everything this manual describes is in that checkout: the `cmfqsar` package with the command-line tool inside it (`cmfqsar/cli.py`), the two launchers (`cmf_app.py`, `cmf_cli.py`), the example data under `examples/`, the benchmark scripts under `benchmarks/` with their recorded results, the tests, and this manual with the script that typesets it.

Requirements

Python 3.10 or newer. The package is declared in `pyproject.toml`, so from the checkout

```
pip install -e .[gui]
```

installs the library with RDKit (chemistry), scikit-image (marching cubes for the isosurfaces), scikit-learn (the SVM solvers), NumPy, SciPy and joblib, and with the `gui` extra PySide6 (the interface) and pyqtgraph with PyOpenGL (the 3D view); it also puts the commands `cmf-app` and `cmf` on the path. The library alone, `pip install -e .`, needs no Qt. The program is pure Python and needs no compilation step; it has been used on Windows, Linux and macOS.

Further extras, only needed for particular work:

- `docs` — reportlab and matplotlib (which supplies the DejaVu fonts) to typeset this manual;
- `dev` — pytest, for the test suite;
- `examples` — `dimorphite_dl`, protonation states when importing SMILES for a given pH;
- a local ORCA or VASP installation if you intend to compute per-atom quantum properties to use as fields (see *Preparing the data* and appendix A); these are programs, not packages.

`pip install -r requirements.txt` installs the working checkout, `.[gui,docs,dev]`. `requirements-lock.txt` pins the exact versions the benchmark tables were produced with: `pip install -r requirements-lock.txt` and then `pip install --no-deps -e .` reproduces that environment. `CITATION.cff` says how to cite the software and the papers.

Starting the application

```
cmf-app
```

or `python cmf_app.py` from the checkout, which takes the same arguments; or directly with a data set, which skips two dialogs:

```
cmf-app examples/benzamides_synthetic.sdf --activity pActivity --test-fraction 0.25
```

The command-line tool is `cmf` once installed and `python cmf_cli.py` from the checkout; the rest of this manual writes the latter, and the two are the same program.

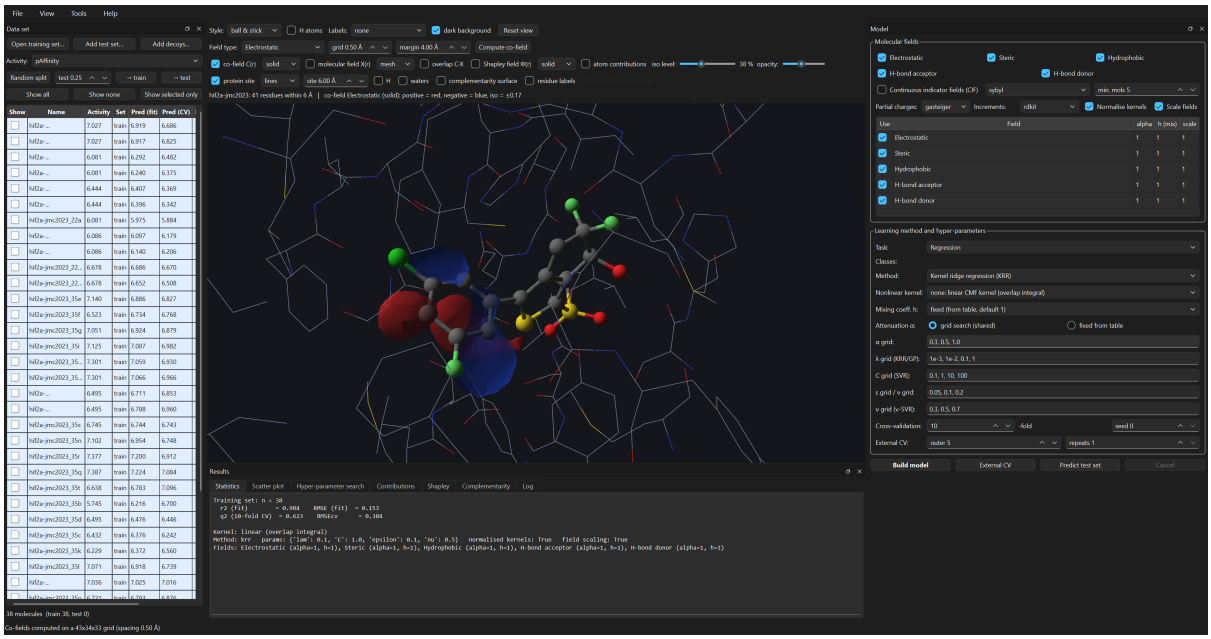
If the 3D view stays black, the OpenGL driver is the usual cause; see *When something goes wrong*.

A first model in five minutes

1. *File → Open training set…* and choose `examples/benzamides_synthetic.sdf`. It holds 109 aligned N-phenylbenzamides with a synthetic activity, so you know the answer the model should find. Pick `pActivity` as the activity property.
2. In the data-set table, press *Random split* and set a quarter of the molecules aside as a test set.
3. In the Model panel leave the five physico-chemical fields ticked, leave the method at kernel ridge regression, and press **Build model**.
4. Read the *Statistics* tab: q^2 should be about 0.87 and R^2_{pred} about 0.84 — the noise ceiling of that set is 0.94, so the model has recovered nearly all of the signal that was put into it.
5. Press **Compute co-field**, choose *Electrostatic*, and move the *iso level* slider. The red and blue lobes that appear are the pseudo-receptor zones the activity was generated from.

Nothing in that sequence is specific to the example: it is the working cycle of the program.

The application window



The HIF-2α series of the Merck free-energy benchmark, 38 ligands: a kernel ridge model with $q^2 = 0.62$, its electrostatic co-field drawn as ± isosurfaces, and the binding site of the reference crystal structure, which the data set brings with it and which needs no superposition.

The window is a 3D view with three dockable panels around it. The **3D view** and its control strip occupy the centre and are always there; the **Data set** panel docks on the left, the **Model** panel on the right inside a scroll area, and the **Results** panel across the bottom under the view. All three are ordinary dock widgets: drag one by its title bar to another edge, drop it on another to stack them as tabs, or pull it off entirely to float it on a second monitor. The layout is remembered between sessions, and *View → Restore default layout* puts it back when an experiment goes wrong.

At the bottom, the **status bar** shows what the program is doing, with a progress bar on the right during a long task. Every long task — a grid search, a co-field, an alignment, a conformer generation — runs in a worker thread, so the window stays responsive and the *Cancel* button in the model panel stops the work rather than the application.

The Data set panel

Three buttons at the top load structures: **Open training set…**, **Add test set…** and **Add decoys…**, the same actions as the first three entries of the *File* menu. Beside them the *Activity* selector names the SD property the activities are read from; changing it re-reads the column without reloading the file.

The table has one row per molecule and seven columns:

Column	Contents
Show	the tick that decides whether this molecule is drawn in the 3D view
Name	the molecule's title from the file
Activity	the experimental value, from the chosen property

Column	Contents
Set	train, test or decoy
Pred (fit)	the model's prediction for the molecule it was fitted on
Pred (CV)	its prediction when it was in the held-out fold
Pred (test)	its prediction as a test molecule
AD	its similarity to the training set, when the applicability domain was computed

An eighth column, **Chemotype**, appears once the chemotypes have been found. Rows are selected singly or in blocks, and the selection drives everything else: the co-field overlap, the atom contributions, the Shapley decomposition and the complementarity numbers all refer to the molecule currently selected.

Sorting. Clicking a column header sorts by it, a second click reverses the order, and a third restores the order of the file. Molecules with no value in that column are always placed at the bottom, whichever direction is in force, so an unpredicted molecule never hides among the predictions. The *Show* ticks, the selection and the 3D view follow the molecule and not the row, so sorting never disturbs what is on screen.

Splitting. *Random split* assigns a fraction to the test set; the → *train* and → *test* buttons move the selected rows by hand. A `subset`, `set` or `split` tag in the file is honoured on loading, which is how the published splits of the example sets arrive without any clicking.

Showing. *Show all*, *Show none* and *Show selected only* control the view in bulk. With one molecule shown, the maps in the viewer refer to it; with all of them shown, the co-field sits in the middle of the series as it should.

A line under the table reports the state of the data set — how many molecules, how many in each subset, which activity property — and a note in brackets records any transformation that has been applied, for instance *[topomer substituents of …]*, so that a changed data set never passes for the original.

The Model panel

Everything that defines a model, in the order one normally works down it. The panel scrolls; on a small screen nothing is hidden, it is merely below.

Molecular fields. Five tick boxes for the physico-chemical fields — Electrostatic, Steric, Hydrophobic, H-bond acceptor, H-bond donor — then *Continuous indicator fields (CIF)* with its typing scheme (`sybyl`, `element`, `element+aromatic`) and the minimum number of training molecules an atom type must occur in to be given a field. Below them, *Per-atom property fields* lists the per-atom properties the loaded file carries, each of which can be turned into a field of its own.

Partial charges chooses between Gasteiger, MMFF94 and the charges stored in the file; *Increments* chooses the parameterisation (`rdkit`, `tripos`, `tripos-legacy`). Two tick boxes set the kernel conventions: *Normalise kernels* for cosine-normalised field kernels, *Scale fields* to rescale every field kernel to unit mean diagonal before mixing — untick it for the raw-kernel convention of the papers.

Conformations switches between one conformation per molecule and an ensemble, with the weighting (uniform or Boltzmann), kT, a cap on the number of conformations, and the multi-instance settings (the softness β, and *one conformation* for the hard selection). A label beside

them reports how many conformations the data set currently holds.

A table at the foot of the group lists the fields that will actually be built, with the columns **Use**, **Field**, **alpha**, **h (mix)** and **scale**: per-field attenuation and mixing coefficients are editable there, which is how a field is given more or less weight than the others without leaving the panel.

Learning method and hyper-parameters. A form, row by row:

Row	What it sets
Task	regression, binary classification, or one-class (virtual screening)
Classes	for classification: a threshold on the activity, with *active below* for data where small is potent, or a class property from the file
Method	the learner; the list changes with the task — KRR, GP, ε-SVR, ν-SVR for regression; C-SVC, ν-SVC, Laplace GP for classification; 1-SVM, centroid, k-NN, one-class GP and kernel PCA for one-class work
Nonlinear kernel	none (the linear CMF kernel), a Gaussian layer, or one or two arc-cosine layers
Mixing coeff. h	fixed from the table, optimised by coordinate descent, or set by centred kernel alignment
Attenuation α	a single value, or the grid below
Kernel from	overlap integrals of the fields, which need one frame; the rotation-invariant autocorrelation, which does not; or the surface autocorrelation for poses over a surface at $z = 0$, which needs only the normal (command line)
α grid, λ grid, C grid, ε/ν grids	the search ranges, as comma-separated lists
Cross-validation	the number of folds and the shuffling seed
External CV	the outer folds and the number of repeats for the nested procedure
Probe fields	GRID-like fields for the water, methyl, carbonyl, amine and cation probes, and the number of Gaussians per atom
Computed descriptors	molecular weight, clogP, TPSA and rotatable bonds, computed by RDKit; all off by default
Descriptors (per molecule)	which numeric molecule properties enter as non-field descriptors, and the block's mixing coefficient
Validation	*shuffles* for y-randomisation, *bootstrap* for confidence intervals, and the *applicability domain* tick — all three off by default, all three run as part of **Build model** rather than from a button of their own

Four buttons close the panel: **Build model** runs the grid search, the final fit, the internal cross-validation and the test prediction; **External CV** runs the nested procedure and reports q^2_{ex}; **Predict test set** applies the current model to the test molecules; **Cancel** stops whatever is running.

The 3D view

The view itself is an OpenGL scene: drag to orbit, the wheel to zoom, middle-drag (or shift-drag) to pan. *Reset view* reframes the whole scene, *Tools → Center view on current molecule* reframes the selected one, and *File → Save 3D view as image…* writes what you see to PNG.

The control strip above the view is four rows, and everything on it redraws immediately:

- **Molecules.** Drawing style (ball & stick, sticks, lines, space fill), *H atoms*, *Labels* (none, element + number, atom type, partial charge), *dark background*, *Reset view*.
- **Maps.** *Field type* chooses which field the maps refer to; **Compute co-field** evaluates the co-fields on a grid around the training set; then the maps themselves — *co-field C(r)*, *molecular field X(r)*, *overlap C·X*, *Shapley field Φ(r)*, *atom contributions*, *difference map*, *pharmacophore* — each with its own solid-or-mesh style where that applies. After a two-endpoint fit an *Endpoint* box joins the row and chooses whose co-field the maps show: either endpoint, or their difference, the selectivity co-field.
- **Levels.** *iso level*, the isosurface threshold as a percentage of the largest absolute value rather than an absolute number, so the surfaces stay comparable when the field changes; and *opacity*.
- **Protein site.** *protein site* draws the residues within the chosen distance of the molecules on screen, as lines or sticks, with *H*, *waters* and residue labels optional, and *complementarity surface* paints the molecule's surface by how well it complements the pocket.

Co-field colours are fixed, so that maps made in different sessions can be compared at a glance:

Field	Positive lobe	Negative lobe
Electrostatic	red	blue
Steric	green	yellow
Hydrophobic	orange	light blue
H-bond acceptor	magenta	dark red
H-bond donor	cyan	purple
Indicator field	red: an atom of that type is favoured here	blue: it is disfavoured here

The Results panel

Twelve tabs, each filled by the work that produces it:

Tab	What it holds
Statistics	every figure of merit, for the fit, the cross-validation, the test set and the external CV; the y-randomisation, bootstrap and applicability-domain results follow at the bottom, under a *Validation* heading, when they were asked for
Scatter plot	observed against predicted, by subset, with the molecule under the cursor named
ROC	the receiver-operating curves of a classification model, per subset
Scores	the ranked decision values or calibrated probabilities: rank, molecule, score, set, class
Enrichment	the enrichment curve against a decoy set, with EF at 0.5, 1, 5 and 10 %, BEDROC and RIE
Map	the kernel-GTM chemical-space map, with the activity, class and density landscapes, and a control for how the molecules themselves are drawn over it
Hyper-parameter search	the whole grid — α, the other parameters, RMSEcv and q^2 — so that an optimum at the edge of the grid is visible rather than hidden

Tab	What it holds
Contributions	two tables for the selected molecule: the training molecules that make its prediction, with `a_j` and `a_j K(X, X_j)`, and its own atoms with their contributions
Shapley	the Shapley decomposition, over atoms, substituents of the common scaffold or field types, with second-order interactions
Complementarity	the field complementarity of the molecules and of the co-field with the pocket, by sign agreement, cosine or correlation
Chemotypes	the Murcko scaffolds found in the set, their sizes, and the leave-one-chemotype-out result beside the alignment-free baseline
Log	everything the program did, with timings and warnings — including the names of the molecules that fell outside the applicability domain — the first place to look when a result surprises you

The menus

File — *Open training set…* (Ctrl+O), *Add test set…*, *Add decoys (background set)…*, *Import activities from CSV…*; *Load protein structure (PDB/mmCIF)…*, *Fetch protein from the PDB…*, *Forget protein structure*; *Load model…* (Ctrl+L), *Save model…* (Ctrl+S); *Export predictions (CSV)…*, *Export aligned data set (SDF)…*, *Export current co-field (Gaussian cube)…*, *Export current Shapley field (Gaussian cube)…*, *Save 3D view as image…*; *Quit* (Ctrl+Q).

View — show or hide each panel, *Results panel spans full width* for when the tables and plots matter more than the view, and *Restore default layout*.

Tools — *Generate conformations (4D)…*; *Dock into the pocket…*; *Align data set to reference…*; *Align by prediction confidence (GP variance)…* and its cross-validated form for the training set; *Replace molecules by their topomer substituents…*; *Field difference from a reference molecule…*; *Extract pharmacophore from the co-field…*; *Model several endpoints (selectivity co-field)…*; *Chemical space map…*; *Align decoys to the training set…*; *Unalign (randomise poses)…*; *Center view on current molecule*; *Quantum-chemical properties with ORCA…* and *Save the data set with its properties (SDF)…*; and the submenu *Heterogeneous sets (several chemotypes)* with its four numbered steps.

Help — *About CMF 3D-QSAR / method summary*, which names the equations of the papers that each part of the program implements.

Every *Tools* entry opens a dialog that states what it is about to do and on which molecules, and each one runs in the background with the progress bar and the *Cancel* button live.

Preparing the data

What the program reads

Format	Notes
`.sdf`, `.sd`, `.mol`	3D coordinates used as they are; hydrogens are added with coordinates if missing
`.mol2`	Sybyl atom types and partial charges are read and can be used directly; a tolerant fallback reader handles blocks RDKit refuses
`.smi`, `.csv`, `.tsv`, `.txt`	2D input: each molecule is embedded with ETKDG and minimised with MMFF94, which means **the poses are arbitrary** until you align them
`.pdb`, `.mmcif`	protein structures only, read by a deliberately permissive reader

A CSV needs a column of SMILES; a column called `name` or the first non-SMILES column names the molecules, and every other column is kept as a molecule property, so activities and metadata travel with the file.

Activities

Three ways, in order of convenience:

1. **An SD property.** The loader guesses the activity property and the dialog lets you change it. Anything numeric will do: `pIC50`, `pKi`, `pActivity`, `CBG_pK`.
2. **A separate table** (*File → Import activities from CSV…*, `--activity-file`): molecule name, activity, and optionally a third column with the split (1 = train, 2 = test, or the words).
3. **A class property** for classification tasks, or a threshold on the activity.

A per-molecule tag named `subset`, `set` or `split` is honoured automatically, which is how the published splits of the example sets arrive.

Alignment

CMF compares fields in one frame of reference. If your molecules came from a superposition, from a crystallographic series or from a docking run, that frame already exists and you should leave it alone. If they came from SMILES, they have no frame at all and you must make one — or use one of the frame-free routes of *Working without an alignment*, which is often the better answer.

The tools, in increasing order of sophistication:

- *Tools → Align data set to reference…* — rigid superposition on the maximum common substructure with a reference molecule, optionally refined by maximising the CMF kernel similarity, which is the alignment criterion proposed in the 2013 paper.
- *Tools → Align by prediction confidence…* — minimises the Gaussian-process posterior variance $\sigma^2(x) = K(x,x) - \mathbf{k}(x)^{\mathrm{T}}(\mathbf{K} + \lambda\mathbf{I})^{-1}\mathbf{k}(x)$ of a molecule under an existing model, which moves it into the model's applicability domain. It uses no activity value, so it is valid for prospective prediction. The program refuses to move training molecules, because the optimum there is degenerate: superimposing everything maximises mutual similarity and destroys the signal.

- *Tools → Align decoys to the training set…* — the same idea for a background set, so that no decoy is rejected merely for having been posed badly.
- *Tools → Unalign (randomise poses)…* — the control experiment. It gives every molecule a random rigid pose, which leaves internal geometry and self-kernels untouched and scrambles only the mutual arrangement. Randomise, realign, and see how much comes back; the program remembers the poses from before and reports the recovery.

Judging an alignment, and aligning flexibly

Everything in this manual says the alignment decides more than the kernel, the learner or the fields. What it could not say until recently is whether one alignment method is better than another, because the obvious measure cannot tell you: realigning changes the geometry q^2 is computed on, so a method is rewarded for making the data look like the model, and q^2 and the external R^2pred have been seen to move in opposite directions under exactly that operation.

The criterion has to come from outside the model, and the FEP benchmark sets supply one: every ligand of `examples/openff` is co-crystallised in the same frame, so the pose a method *should* have found is known. `benchmarks/run_poses.py` fits a model on training molecules in their crystal poses, destroys the poses of the held-out ones, and asks each method to recover them without ever seeing a held-out activity. Two numbers come back for every method — how near the crystal pose it got, measured over heavy atoms, symmetry-aware and **without superposition**, and what the model then predicts — and both are reported between two fixed references: the crystal poses as the ceiling and the scrambled poses as the floor.

That last detail is what makes the protocol work. Superposing before measuring would report 0.00 Å for a molecule placed in the wrong site: on a ligand displaced by 1 Å, `CalcRMS` returns 1.0 and `GetBestRMS` returns 0.0.

What the methods are worth

Measured this way over sixteen FEP sets, with the held-out **conformations regenerated** rather than read from the crystal — which is the case a new molecule actually presents:

method	median RMSD	within 2 Å	R^2pred	conformer limit
crystal poses (the ceiling)	0.00 Å	100 %	+0.334	0.00 Å
scrambled (the floor)	6.12 Å	1 %	−0.696	1.73 Å
substructure (MCS)	1.94 Å	54 %	−0.391	1.73 Å
MCS + rigid confidence	2.27 Å	45 %	−0.396	1.73 Å
MCS + flexible confidence	1.41 Å	74 %	+0.065	1.20 Å
MCS + flexible, 5 conformer starts	**0.86 Å**	**88 %**	**+0.264**	**0.71 Å**

Three things follow. **Freeing the torsions makes the confidence criterion work**: with a conformation to fix, the rigid search of the 2019 paper is worse than not refining at all, which is the failure the protocol was built to expose; the same objective with the rotatable bonds free is the best method available. The **conformer generator is what makes it work** — five generated starting conformations improve the median pose by 0.55 Å, better on 15 of 16 sets, and the combination beats plain substructure alignment on 16 of 16. And the **displacement guard has little left to prevent**: with these starts no set is worse than 1.52 Å, a 1 Å guard costs 0.6 Å, and 2 Å is

indistinguishable from none.

What none of it does is replace knowing the conformation: +0.264 against the crystal poses' +0.334. The conformational residue is 0.71 Å, and that is where the remaining error comes from.

Inside the pocket

Both criteria above know only half of the problem. The confidence criterion knows the series and not the receptor, so nothing stops it putting a molecule where no molecule could go; the docking score of `cmfqsar/docking.py` knows the receptor and not the series, so nothing makes the pose it finds comparable with the poses the model was fitted on. The item that joins them reads the complementarity as a **prior** over poses and the confidence as the **likelihood**, and takes the pose that maximises the product:

$$\min_{p} \sigma^2(p) - w\,[\,\mathrm{complementarity}(p) - \mathrm{clash}(p)\,]$$

over the six rigid-body parameters p, with the same displacement guard. At $w = 0$ this is the confidence alignment exactly; as w grows the receptor takes over.

```
cmf predict model.cmf new.sdf --align-variance --align-pocket complex.pdb --pocket-weight 1
```

Judged by the protocol above on 4 FEP sets with a receptor structure (mcl1, p38, thrombin, tnks2), the held-out poses scrambled and recovered:

method	median RMSD	within 2 Å	R²pred
crystal poses (the ceiling)	0.00 Å	100%	+0.335
scrambled (the floor)	5.03 Å	0%	-0.965
substructure (MCS)	0.16 Å	100%	+0.229
confidence alone	0.30 Å	59%	-1.166
MCS + confidence	0.10 Å	100%	+0.286
pocket alone	3.93 Å	31%	-0.805
pocket + confidence	2.50 Å	46%	-0.831
MCS + pocket + confidence	0.13 Å	100%	+0.265

Per set, the comparison that carries the answer — the receptor added to the confidence criterion, with and without a substructure frame to start from:

set	confidence alone	pocket + confidence	MCS + confidence	MCS + pocket + confidence
mcl1	0.29 Å	0.31 Å	0.17 Å	0.22 Å
p38	0.32 Å	3.73 Å	0.09 Å	0.13 Å
thrombin	5.00 Å	7.58 Å	0.10 Å	0.14 Å
tnks2	0.21 Å	1.26 Å	0.07 Å	0.09 Å

The receptor term is not worth its cost. From a scrambled start the confidence criterion alone finds the pose on three of the four sets, and adding the complementarity makes it worse on two of them and better on none. After a substructure frame every refinement ends within a tenth of an

ångström of the others, and the predictions differ by less than the sets differ among themselves. The joint objective also costs about ten times as much, because the pocket term has no analytic gradient and the search falls back on Powell.

The problem is the pocket term, not the search method. At weight zero the joint aligner is the confidence objective optimised by a different method, and it recovers the pose as well as the gradient search does (0.29 Å on p38 against 0.32 Å). Turning the weight up from there is what makes it fail:

pocket weight	p38	tnks2
0 (the confidence criterion alone)	0.29 Å	0.07 Å
0.25	3.52 Å	0.11 Å
1	3.73 Å	1.26 Å
4	3.63 Å	1.32 Å

On p38 any amount of the receptor destroys the recovery; on tnks2 it degrades it mildly. The reason is visible in what complementarity is: a weak prior over six dimensions. A ligand that fits a site fits it in many placements, so the term adds a broad, multi-modal surface to an objective that was already multi-modal, and its optimum is not where the crystal pose is — not with a scoring function that has no desolvation, no entropy and no torsional search.

The proper use of what is implemented is therefore narrow. `--align-pocket` is for a pose that must be defensible **to the receptor** as well as to the model: it will not improve on the confidence criterion, and it will tell you when the confident pose is one the protein would not allow.

Nothing here is a docking program: there is no desolvation, no entropy and no torsional search, and the internal geometry is never touched. What the receptor adds is a term that knows where the protein atoms are, and the measurement says where that helps and where it does not.

Using it

In the application, *Tools → Align by prediction confidence…* gained a **turn the rotatable bonds as well** tick box, with the number of starting conformations and the strain allowance beside it. From the command line:

```
cmf predict model.cmf new.sdf --align-variance --align-flexible --align-conformers 5
```

The force field acts as a prior on the conformation — it may rise ten kcal/mol above where it started before being penalised, one-sided so that the aligner does not quietly become a conformer optimiser — and the displacement limit now applies to the **rigid motion only**, since a turning bond is not a translation and counting it as one would suppress exactly what this option adds.

Charges and per-atom properties

The electrostatic field needs charges: Gasteiger (the default), MMFF94, or **read from the file**, which is what you want with the Sutherland sets and their MNDO ESP-fit charges, or with anything you computed yourself.

Beyond that, any per-atom number can become a field of its own. `prop:<name>` reads the atomic weights from a property of the molecule — a quantum charge, a Fukui index, a local softness, an atomic polarisability, even a per-atom embedding — and that field then behaves exactly like a

built-in one, with its own attenuation, its own mixing coefficient, its own co-field, atom contributions and Shapley values.

Two conventions are read, in this order:

1. a property set on **every atom**, which RDKit writes to SDF as `atom.dprop.<name>` and reads back automatically;
2. a **molecule property holding one number per atom**, separated by spaces, commas or newlines, which is how most quantum-chemistry packages dump such a table.

The hydrogens are the usual source of error. A file without 3D coordinates is embedded on loading and hydrogens are added, so a list written for the heavy atoms alone no longer matches the atom count. The program says so rather than silently mis-assigning: `--prop-missing error` (the default) stops, `--prop-missing zero` gives the unlisted atoms zero weight. The next section shows the whole route from an ORCA run to fields and descriptors, from inside the program or from the command line.

Quantum-chemical properties with ORCA

Tools → Quantum-chemical properties with ORCA... runs ORCA over the loaded molecules and brings two kinds of number back, to the two places the model keeps them.

Per atom, and so available as `prop:` fields: Mulliken, Löwdin, Hirshfeld and CHELPG charges, the HOMO and LUMO population of every atom, and — when the N+1 and N−1 states are computed as well — the condensed Fukui indices f^+, f^-, f^0, the dual descriptor $f^+ - f^-$, the local softnesses and the local electrophilicity. **Per molecule**, and so available as descriptors: the total energy, the HOMO and LUMO energies and their gap, the dipole moment, and from the ionised states the ionisation potential, the electron affinity, the chemical hardness, the softness, the chemical potential and the electrophilicity index.

That split is the purpose of the feature. A Fukui index is a statement about one atom, so it belongs in a field, where a co-field can then say *where* a nucleophilic centre helps; the gap is a statement about the molecule, so it belongs in the descriptor block, where it cannot pretend to have a position.

The dialog asks for

- **the ORCA program** — found by itself at the standard location, in `$ORCA_EXE` or on the PATH, and otherwise given here;
- **the level of theory** — the ORCA keyword line, which may be edited: `r2SCAN-3c` (fast, with its own basis) is the one to start with, B3LYP-D3/def2-SVP the usual hybrid, and anything else ORCA accepts as a single point will do;
- **which molecules** — all of them; only those that do not carry the properties yet, which is the default, so a set extended with new compounds costs only the new ones; the selected ones; or the training set;
- **the Fukui indices and the global reactivity numbers**, which need the N+1 and N−1 states and so cost three calculations per molecule instead of one;
- **optimising the geometry first**, off by default — see the warning below;

- **how many molecules at a time**, and, where the ORCA build has MPI, how many cores per job. The Windows build is the serial one: it has no MPI helpers, `%pal` fails outright, and the way to use a many-core machine is to run several molecules at once, which is what the first box is for. The dialog disables *cores per job* when it sees a serial build;
- **memory per core**, a **time limit** per job, after which that molecule is given up on rather than holding up the set, and the **working directory**.

Before you press *Run* the dialog says what the run will cost: the number of molecules times the number of ORCA jobs each, and the wall time that comes to at the chosen parallelism. After the first run that estimate is no longer a guess — the time every job took is recorded in the working directory, and the dialog quotes the measured mean. For scale: a molecule the size of methanol at `r2SCAN-3c` takes a few seconds per job, a drug-sized one at B3LYP-D3/def2-SVP a couple of minutes, so a hundred of the latter with their Fukui indices is a few hours with four running at a time.

The dialog also states the charge and multiplicity it read off the structures — the formal charge of the molecule, and the lowest spin state its electron count allows — so an unintended anion or an odd electron is visible before the hours are spent, not afterwards in a column of nonsense.

Three things protect a long run. Every job is **cached** under a key over the molecule, its charge, the level of theory and the geometry, so *Cancel* — which kills the ORCA processes themselves, not merely the bookkeeping — loses only the jobs that were running, and starting again continues from where it stopped. A molecule that **fails** is named in the log and skipped, never fatal to the set. And the working directory keeps every input and output, so any number can be traced back to the calculation that produced it.

When the run finishes, the new quantities appear as tick boxes — the per-atom ones in *Per-atom property fields*, the per-molecule ones in *Descriptors (per molecule)* — and the program offers to write the annotated set as an SDF. Write it: the file carries the properties with it, as atom property lists and molecule properties, so opening it later brings them all back without running ORCA again. *Tools → Save the data set with its properties (SDF)…* does the same at any time.

> **About the geometry.** *Optimise the geometry first is off by default and should normally stay off. ORCA optimises each molecule on its own, which moves every atom: a series that was superimposed is not superimposed afterwards, and a model trained on the old poses no longer refers to the molecules in memory. With the box off, the properties are computed exactly where the molecules stand, which is what a field model needs. If you do optimise, align the set again before building anything.*

The same computation runs from the command line as `examples/orca_atom_properties.py` (`--help` lists its options), which reads an SDF and writes the annotated one. The script and the dialog share the module `cmfqsar/orca.py`, so the numbers are the same either way.

Before you model: `cmf diagnose`

Every expensive mistake recorded in this manual was visible before the model was built, in a number that takes seconds to compute. The corrosion benzenes were modelled with five fields and an automatic frame, and atom-type counts — no geometry at all — beat them. The ACE set is predicted as well by a handful of two-dimensional descriptors as by the whole 3D model. A thrombin model in the exposure study scored +0.059 with its attenuation pinned to the top of the grid, which is what a mis-specified model looks like from outside. None of these is subtle; they are simply never asked, because asking means running four baselines by hand.

```
cmf diagnose data.sdf --charges file
```

The command answers five questions and prints the numbers behind each.

Is there anything to model? The size of the set and the spread of the activity. Twenty molecules across half a log unit cannot support a conclusion, whatever q^2 comes out, and the command says so and stops.

Can a frame exist at all? The number of chemotypes, the size of the common Bemis–Murcko framework, and whether the molecules are actually superimposed. That last test is asked of the shared framework atoms rather than of the centroids, because `EmbedMolecule` leaves every molecule centred on the origin: a set of independently generated conformations has a centroid spread of 0.02 Å and looks perfectly aligned while every molecule points a different way. The test is also allowed to give no answer. A framework that is a small symmetric ring matches a dozen ways and two rings near the origin may coincide by luck, while a framework that is a small part of the molecule may match a *different* copy in each one — measured that way an author-aligned Sutherland set read as 9 Å apart. So a coincidence within half an ångström is a verdict on its own, a large deviation is a verdict only when the framework is a fair share of the molecule, and otherwise the answer is "this cannot tell — look at it".

Does the frame carry signal? The aligned model against the two routes that need no frame: the atom-type count kernel, which has thrown the geometry away entirely, and the rotation-invariant field autocorrelation, which keeps it without a pose. If neither is beaten, the alignment adds nothing and the cheaper route is also the more defensible one.

Does the geometry contribute anything? The same comparison against descriptors alone — what the file carries plus the handful the program computes.

Was the model specified sensibly? Whether the chosen attenuation sits on the edge of its grid, which means the search wanted a value it was not offered.

A fifth route, the topomer rule, is scored where the molecules can be cut at their framework. It covers only those molecules, so the aligned model is refitted on the same ones for the comparison: a subset against the whole set is not a comparison, and it favours whichever route happens to drop the hard compounds.

Two things the command protects you from

The activity as a descriptor. Most files store the activity as a molecule property, so the descriptors available usually include the quantity being predicted. Given it, the descriptor baseline returns $q^2 = 1.000$ and the advice becomes "use descriptors" for every set in the world. Columns

named as the activity are dropped, and so is anything correlating with it above 0.95 — but the leaked information is rarely a plain copy. The corrosion files carry the potency as `activity`, again as `mwila_pooled_raw`, and again as two laboratory-adjusted variants; excluding the first two still left a descriptor baseline at $q^2 = 0.99$, because the remainder reconstruct it between them. No per-column test can see that, so the outcome is judged as well: a descriptor baseline reaching $q^2 \geq 0.95$ on a set of this size is not chemistry but the measurement itself, and the command says so, names the nearest columns with their correlations, and refuses to recommend descriptors.

Searching harder and calling it a better model. The five routes do not carry the same number of hyper-parameters — the field routes search six attenuations and five ridges, the others search ridges only — so scoring each at the best point of its own grid biases the comparison toward whichever route searched the most. Every route therefore chooses its hyper-parameters inside each training fold and is scored outside it. On ACE this costs the aligned model 0.05 and the topomer route 0.07 while leaving counts, descriptors and the invariant kernel where they were, which is the size and the shape of the bias it removes.

Reading the answer

Each comparison is reported with a paired bootstrap interval over the molecules, because on twenty to thirty molecules a difference below about 0.1 in q^2 is not a difference. The last line recommends a route, and it should be read as a prompt rather than a verdict: it says which route to take and which question the set has not answered, never that a model is good.

One wording deserves care. When the command recommends `invariant` it often means *the frame is not worth the trouble* rather than *the invariant kernel predicts better* — on DHFR the aligned model is nominally ahead, +0.659 against +0.641, and the recommendation follows the interval rather than the point estimate. And `aligned, with the frame checked` means the frame carries signal and the molecules really are superimposed, so a frame exists whoever made it; several Murcko scaffolds only mean the program could not have built it, not that it is wrong.

Does the advice work?

A tool that reports five baselines is useful whatever its last line says, but advice that has never been followed is only a rule its author liked. `benchmarks/run_diagnose.py --validate` measures it the only way that matters: hold out 30 % of a set, run the diagnosis on the remaining 70 % alone, build the model it recommends, and score that model on the held-out molecules against every other route — thirteen sets, three splits each.

The advice named the best of the five routes on **21 of 39 splits**, and averaged over all of them the route it chose fell **0.096** in q^2 short of the best available. Always taking the aligned model — the default this command exists to question — names the best route on 12 splits and costs 0.201. Aggregated per set, which is the correct unit because three splits of one set are not three measurements, following the advice costs **0.105 less in q^2 than always aligning** (95 % bootstrap interval −0.259 to +0.019), better on six sets, worse on four, indistinguishable on three.

The shape of that distribution matters more than its mean. The advice is **insurance, not an advantage**: where it loses it loses a little — 0.02 to 0.17 on AChE, BZR, THR and the thermolysin set, all of them well-aligned series where the aligned model was always going to be right — and where it wins it wins a lot, 0.44 to 0.70 on the three corrosion sets, where the automatic frame is meaningless and the aligned model collapses. The interval crosses zero, so on thirteen sets this is not a proven improvement in the average case. What it is, is insurance against the failure described at the start of this chapter, at a price of a few hundredths of q^2.

Building a model

Choosing the fields

The five physico-chemical fields — electrostatic, steric, hydrophobic, H-bond acceptor, H-bond donor — are the default and reproduce the published CMF models. The steric field is special: its width is scaled by the van der Waals radius of each atom (α/R^2_{vdW}, Eq. 10–11 of the 2013 paper) rather than being shared.

Continuous indicator fields (CIF, 2015 paper) add one field per atom type, with weight 1 on atoms of that type and 0 elsewhere. Tick *Continuous indicator fields* and set the minimum number of training molecules an atom type must appear in (3 by default) to be given a field of its own. Three typing schemes are available: `sybyl` (the types of the original scripts, read from MOL2 or assigned), `element`, and `element+aromatic`. CIFs are the most descriptive field set the program has, and on several of the benchmark sets they match or beat the physico-chemical fields on their own.

`prop:<name>` fields are added by naming them in the per-atom property row of the model panel, or in `--fields`.

GRID-like probe fields

Goodford's GRID describes a molecule by the energy a small probe — a water, a methyl, a carbonyl oxygen — would feel at every point around it. That is a field in exactly this program's sense, and it says something the built-in fields do not: not "there is hydrophobic character here" but "a probe placed here would be bound". *Probe fields* in the model panel (`--probes water,carbonyl`) adds one such field per probe. Five are defined:

name	what it stands for	R (Å)	well (kcal/mol)	charge	hydrogen bonding
`water`	water, OH_2	1.70	0.15	0	both, weakly
`methyl`	a methyl group	2.00	0.10	0	neither
`carbonyl`	a carbonyl oxygen	1.60	0.20	−0.40	accepts
`amine`	an amine nitrogen	1.75	0.16	+0.40	donates
`cation`	a generic +1 cation	1.50	0.10	+1.00	neither

Those four numbers are the whole definition of a probe: the radius and well depth enter the dispersion term through the usual combining rules, the charge the electrostatic term, and the hydrogen-bond character decides which atoms it is complementary to — a donor probe feels acceptor atoms and not other donors. The parameters sit in the usual GRID and Tripos ranges rather than being taken from a published GRID table, which matters if you ever want to compare numbers with GRID itself rather than shapes. Adding a probe is one entry in `PROBES` in probes.py; the fitting, the kernel, the drawing and both interfaces read that dictionary, so a new one appears everywhere at once.

The obstacle is arithmetic, not conceptual: a Lennard-Jones term is not a Gaussian, has no closed-form overlap, and diverges at the atom, so the integral of its square does not exist. The solution is to fit each atom's contribution by a few Gaussians, which keeps every kernel in the program a closed form over atom pairs. Three things had to be measured before that worked, and each one shaped what is implemented:

- **The repulsive wall cannot be represented.** Fitting a curve that is +30 kcal/mol at 2.5 Å and −0.13 at 3.5 Å with a handful of Gaussians produces coefficients of $\pm 10^7$ that cancel, and a worst-case error larger than the well it is meant to represent. The wall is a truncation artefact in every grid implementation anyway, and this program already includes it — the steric field *is* a Gaussian van der Waals envelope — so a probe field carries the **attraction** only.
- **A well at a radius is a shell**, and a shell is not a sum of Gaussians centred on the atom. Each term is therefore capped at its own optimum, which makes it monotone in r, and those a sum of Gaussians represents naturally.
- **The r^{-6} tail is not Gaussian either**, and is negligible beyond 6 Å, so the energy is tapered to zero between 5 and 7 Å exactly as GRID's own cut-off does.

What comes out reproduces the attraction to about 15 % in the volume-weighted norm the kernel sees (23 % on a realistic worst case), and the kernel it implies agrees with numerical integration of the fields themselves to $4 \cdot 10^{-7}$ — a test asserts both. Each atom carries K Gaussians (four by default), so a probe field costs K^2 passes over the distance matrix where an ordinary field costs one; three is the fast setting and five the accurate one.

Measured against the fields it would join, same molecules, same α and λ grids, 10-fold CV:

set	n	the five fields	+ water probe	probes only
GPB	66	+0.539	+0.539	+0.406
THER	76	+0.538	+0.543	+0.428
THR	88	+0.672	**+0.699**	+0.665
ACE	114	+0.647	+0.648	+0.607

So: adding a water probe to the five fields adds between nothing and 0.027 to q^2, and probe fields on their own are worse than the fields they would replace. That is the measured result and it was the expected one — CoMSIA replaced GRID-like energies with Gaussian similarity indices for much the same reason, and this program's fields are closer to CoMSIA than to GRID by design. What a probe field is actually *for* is interpretation: its co-field is drawn, exported and decomposed like any other, and it says where a water or a carbonyl oxygen would be bound, which is a statement that is easier to act on than "more acceptor character here".

Two limitations are worth stating plainly. The directional part of GRID's hydrogen-bond term — the angles at the donor and at the acceptor's lone pairs — is **not** represented; the radial part is, broadened to 0.6 Å because a sharper shoulder is not representable by the shared basis. And alignment by field similarity refuses to run on a probe field rather than silently ignoring it, because the gradient of the expansion is not implemented.

Weighting the fields by solvent exposure

A kernel that integrates over all space weights an atom buried inside a molecule as heavily as one on the outside, although a binding partner can only touch the second. That is the claim the surface methods are built on, and `cmfqsar/exposure.py` tests it in the simplest way: every atomic weight

multiplied by its atom's relative solvent accessibility, which keeps the kernel in closed form and keeps the co-field.

It does not help, and on the larger sets it slightly harms the model. Across sixteen FEP sets the median paired gain is −0.002 in q^2 and the weighting wins on eight of sixteen — no better than chance — while the one large win is a model whose attenuation was fixed at the edge of the grid. On the eight Sutherland sets every variant is worse, and the softened one that looked best on the FEP sets has an interval entirely below zero. The effect also shrinks as the sets grow: on DHFR (361) and COX2 (282) the change is −0.004 and +0.003, and the movers are the two smallest sets. An effect that vanishes with more data is noise.

The reason is geometric and settles the general question. Measured by group — a heavy atom and its hydrogens counted as one, because atom by atom a methyl carbon is 93 % occluded by its own hydrogens and ethanol reads as two-thirds buried — the median group exposure is 0.37 to 0.42 and **0 to 3 % of groups are buried** in either family. A ligand of thirty heavy atoms has no interior. Surface-based representations are solving a problem that molecules of this size do not have.

Descriptors that are not fields

Everything above describes a molecule as a function of space. Some things are not functions of space and never will be: a measured logP, a molecular weight, the medium or temperature a series was recorded in, a count of rotatable bonds, an energy from a quantum calculation. A model that cannot see them has to pretend the activity depends on geometry alone, which for an adsorption potency or a permeability is simply false.

The *Descriptors (per molecule)* row of the model panel (`--descriptors A,B`) offers every numeric molecule property the file carries — and hides itself when the file carries none besides the activity, which is why it is invisible on most of the example sets; the computed descriptors of the next section are always there. Ticking some adds one more block to the kernel:

$$K(A,B) = \sum_f h_f c_f \hat{K}_f(A,B) + h_d c_d \, \mathbf{z}(A) \cdot \mathbf{z}(B)$$

where **z** holds the descriptors of a molecule, standardised with the **training set's** mean and standard deviation so that a quantity in kilojoules cannot outweigh one in log units, and h_d is a mixing coefficient like a field's (the *h* box beside the row, `--descriptor-h`; zero switches the block off without removing it).

Three properties of the block are deliberate:

- **It is scaled, not cosine-normalised.** Dividing by the two norms, as the fields are divided, would send a molecule sitting at the training mean in every descriptor — an unremarkable molecule — to a zero row and a zero diagonal, out of the model altogether. Instead the whole block is multiplied by a constant that puts its mean diagonal at 1, so ten descriptors do not outweigh the geometry merely by being ten.
- **The statistics belong to the training set** and are stored in the saved model. Standardising a test molecule by its own set would score two test sets on two different scales, which is the mistake the field scaling avoids as well.
- **The model stays linear in its inputs**, so the decomposition rules survive. A prediction splits into a field part and a descriptor part, and the descriptor part splits further into one number per descriptor, which is what the program reports.

A molecule missing a descriptor is refused by default (`prop_missing`), because a model that silently substitutes the mean is telling you nothing about that molecule while looking as confident as ever; with the `mean` policy it is substituted and the molecule simply contributes nothing through that descriptor. A descriptor that is constant over the training set is reported as inert rather than dividing by a spread of zero.

The obvious danger, stated plainly. A descriptor derived from the activity — the activity itself, a transform of it, a value fitted to it — will dominate the model and make the geometry look worthless, while q^2 goes up. The program cannot detect this, so it reports the mean absolute contribution of every descriptor beside the model statistics:

```
Non-field descriptors (mean |contribution| per training molecule):
  clogp                          0.0779
  mw                             0.0176
  tpsa                           0.0123
```

If one descriptor carries most of the prediction, the model is really a descriptor model with a 3D appearance, and it should be reported as one.

```
python cmf_cli.py build series.sdf --activity pIC50 --descriptors clogp,mw,tpsa
```

Descriptors the program computes itself

A file that carries no numeric tags has nothing to offer the row above, so four descriptors are always available, computed by RDKit from the structure: `calc:mw` (molecular weight), `calc:clogp` (Wildman–Crippen logP), `calc:tpsa` (topological polar surface area) and `calc:rotatable` (rotatable bonds). They have a row of their own in the model panel, *Computed descriptors*, and **all four are off by default**.

Those four are shortcuts, not the whole list. **Any** of RDKit's descriptors — some two hundred of them — can be asked for by its own RDKit name, typed into the box at the end of that row (with completion) or passed on the command line: `calc:NumHAcceptors`, `calc:FractionCSP3`, `calc:BalabanJ`, `calc:LabuteASA`, `calc:qed`. Names are matched case-insensitively, and a name that does not exist is refused before the fit with the nearest ones that do — `calc:NumHDonor` answers *did you mean NumHDonors?*.

The default is deliberate, and so is the warning that belongs with it. These are two-dimensional quantities: they can improve a model that has a size or lipophilicity trend in it, and they can also carry a model that then gains nothing at all from its geometry. Measured on four Sutherland sets, same α and λ grids, 10-fold CV:

set	n	five fields	+ the four computed	the four alone
THR	88	**+0.671**	+0.634	+0.620
GPB	66	**+0.534**	+0.525	+0.494
ACE	114	+0.657	+0.707	**+0.712**
THER	76	**+0.509**	+0.496	+0.480

Read ACE carefully: four numbers that know nothing of the geometry predict better than the whole 3D model. That is not an argument for using them — it is a warning about that data set, and it agrees with what the alignment-free baseline already said about ACE elsewhere in this manual. On

the other three sets they lower q^2 slightly.

So use them to *test* a model rather than to improve it: if adding them changes nothing, the geometry is carrying the model; if they take over, the 3D part was contributing nothing. The contribution table printed beside the statistics is what tells you which happened.

Attenuation, regularisation and the grids

Two hyper-parameters matter. α sets how far each field reaches: large α makes the fields sharp, so molecules must match closely, while small α makes them diffuse. λ (or *C*, ε, ν for the SVM learners) sets the regularisation. Both are chosen by cross-validated grid search, and the whole grid is shown in the *Hyper-parameter search* tab — look at it, because an optimum at the edge of the grid means the grid was too small.

```
python cmf_cli.py build data.sdf --alphas 0.1,0.3,0.5,1,2,5 --lams 1e-3,1e-2,0.1,1,10
```

Refinements, all optional: continuous Brent minimisation over λ as in the original R scripts (`--lam-search brent`), a Nelder–Mead search over α (`--alpha-search nelder-mead`), coordinate descent or centred kernel alignment for the mixing coefficients `h_f` (`--optimize-h`, `--mkl`), and a grid over the bandwidth of a nonlinear kernel layer (`--gammas`).

Two conventions decide how the field kernels are combined, and they change the numbers: *Normalise kernels* uses cosine-normalised field kernels, and *Scale fields* rescales each field kernel to unit mean diagonal before mixing. Untick the second to get the raw-kernel convention of the papers (`--raw-scale`).

How long the kernel takes, and the three settings that change it

A field kernel is a sum over atom pairs, so its cost is the number of atom pairs: the square of the set size times the square of the molecule size. A 200-molecule congeneric series costs under a second per attenuation and nothing has to be done about it. A library, a set of long molecules, or a grid search over six attenuations is a different matter, and three things were measured and changed (`benchmarks/run_speed.py` records all of them).

The training kernel is computed once rather than twice. $K_f(i,j) = K_f(j,i)$, so for a set against itself only the pairs with $j \geq i$ are evaluated and then mirrored. This is the largest of the three, because it removes half of the work rather than part of it, and it is exact; it also makes the matrix exactly symmetric, which it was not before, the two triangles having differed by round-off from summing the atom pairs in two different orders.

The radius-scaled prefactor is a square root rather than a power. The steric field gives every atom its own width, so $(2\pi/(b_l+b_m))^{3/2}$ differs for every atom pair, and a general power is a library call per element. Writing it as $z\sqrt{z}$ is the same number to fourteen digits and costs a third as much. Before this change that one expression was half the time of the whole kernel.

The distances are shared across the attenuation grid. The attenuation enters only through the exponential, so a grid search can compute each molecule's squared distances once and take six exponentials from them. This is the smallest of the three: the distance matrix is 3 % of a block.

Measured on 200 molecules of a congeneric series and 60 decapeptides, all six attenuations of the usual grid (0.1, 0.3, 0.5, 1, 2, 5):

shortcut	COX2 (40.8 atoms each)	AMPH1 (168.2 atoms each)	still agrees to
as before	14.3 s (1.0x)	42.2 s (1.0x)	--
+ square root	10.5 s (1.4x)	38.7 s (1.1x)	3e-16
+ symmetry	4.6 s (3.1x)	16.0 s (2.6x)	1e-15
+ shared distances	3.8 s (3.7x)	15.3 s (2.8x)	1e-15

The last column is the point: every shortcut is exact, and what is left is round-off from summing the atom pairs in a different order. End to end, a `grid_search` over the same six attenuations with five-fold cross-validation:

shortcut	seconds	speed-up	alpha chosen	q²cv
as before	13.8	1.00x	0.1	+0.504175
+ square root	9.9	1.40x	0.1	+0.504175
+ symmetry	5.0	2.76x	0.1	+0.504175
+ shared distances	4.1	3.40x	0.1	+0.504175

3.4 times faster, and the attenuation chosen and the cross-validated q^2 are identical to six decimal places.

Nothing here changes a number the program reports; the benchmark prints the chosen α and the cross-validated q^2 beside every timing for exactly that reason.

The cutoff, which is off unless you ask

A Gaussian overlap falls below a fraction τ of its peak beyond a cutoff radius of $\sqrt{(-4 \ln \tau / b)}$, which at $\tau = 10^{-6}$ is 10.5 Å for a shared-width field at α = 0.5 and 19 Å for the radius-scaled steric field. Atom pairs further apart than that can be dropped, and `--cutoff auto` does so, deciding per field by sampling the real distances and building a neighbour list only when most pairs would go.

Whether this helps is a property of the molecules, not of the program:

set	atoms per molecule	α	pairs kept, shared width	pairs kept, steric	exact sum	`--cutoff auto`	what auto did
COX2	40.8	0.5	98%	100%	0.95 s	0.85 s (1.12x)	kept the exact sum
COX2	40.8	2	56%	96%	0.90 s	0.91 s (0.99x)	kept the exact sum
COX2	40.8	5	24%	67%	0.84 s	0.82 s (1.02x)	kept the exact sum
AMPH1	168.2	0.5	62%	99%	2.61 s	2.64 s (0.99x)	kept the exact sum

set	atoms per molecule	α	pairs kept, shared width	pairs kept, steric	exact sum	`--cutoff auto`	what auto did
AMPH1	168.2	2	14%	52%	2.75 s	2.15 s (1.28x)	used the neighbour list
AMPH1	168.2	5	4%	19%	2.64 s	1.21 s (2.18x)	used the neighbour list

The two columns of surviving pairs explain the rest. The sampling that decides costs about a hundredth of a block, so `auto` is free when it declines, and on these sets it declines wherever the dense sum is faster. Forcing the list on with `--cutoff 10` or `--cutoff auto` where it does not pay can cost a factor of five, which is why it is not the default.

On a congeneric series superimposed in one frame the cutoff keeps almost every pair and `auto` refuses to build a list, so the sum stays exact. On molecules five times longer it is worth a factor at a strong attenuation. A cutoff makes the kernel an approximation — each dropped pair contributed less than a millionth of a peak overlap, and the measured error is about 10^{-7} relative — so it is off by default and every table in this manual was produced without it.

The compiled kernel, if Numba is installed

The kernel above is NumPy, and NumPy is single-threaded. The same arithmetic written as one fused loop over the atom pairs and compiled with Numba can use the other cores: `cmfqsar/fastkernel.py` does that, threaded over the molecules, which is safe because every molecule writes its own row of the kernel and nothing is shared.

Two things are worth knowing before reaching for it.

Per core it is slower, not faster. `np.exp` on an array is vectorised; the same calls written one at a time in a compiled loop are not, unless Intel's SVML is installed. Measured serially on 200 molecules, the NumPy path takes 0.67 s and the compiled loop 0.90 s. All of the advantage below is the threads, and on a machine with two cores there will be almost none.

It is not the default. Install it with `pip install numba` or `pip install -e .[fast]`, and ask for it with `--engine numba` (or `--engine auto`, which uses it where it is installed and falls back silently where it is not). The default everywhere except `screen` is `numpy`, because the two paths differ by about 2e-14 relative — the compiled loop is built with `fastmath` and may add the pairs in another order — and every table in this manual was produced with NumPy. That difference is the same order as the round-off between two orderings of the NumPy sum itself, but a default that depends on whether an optional package is installed is not something a table should rest on.

What it is worth, on 32 cores:

set	atoms per molecule	NumPy	compiled	speed-up	differs by
COX2	40.8	0.50 s	0.09 s	5.6x	4e-15
AMPH1	168.2	1.99 s	0.12 s	16.6x	2e-14

The long molecules gain more because each thread then has more work to amortise the cost of being started. End to end it is less, because the kernel stops being all of the time: a whole grid

search over six attenuations falls from 2.9 s to 0.7 s (4.0 times), choosing the same attenuation and the same cross-validated q^2 to six decimals, and a screening run over 4166 molecules from 46 s to 12 s (3.8 times, 91 to 347 molecules a second) — where reading the file and building the fields are now the larger half of the work.

The compiled loop also takes a cutoff without a neighbour list, since inside the loop a cutoff is one comparison per pair; `screen --cutoff R --engine numba` is therefore worth trying on long molecules, where the NumPy neighbour list of the previous section is also worth building.

The learners

Task	Methods	Key
Regression	kernel ridge regression with bias; Gaussian process (the same fit plus predictive variances); ε-SVR; ν-SVR; the ranking SVM, which fits the order and not the values	`krr`, `gp`, `svr`, `nusvr`, `ranksvm`
Binary classification	C-SVC; ν-SVC; Gaussian-process classifier by the Laplace approximation	`svc`, `nusvc`, `gpc`
One-class / screening	1-SVM; distance to the centre of the actives; distance to the k-th nearest active; one-class Gaussian process; distance from the principal subspace (kernel PCA)	`ocsvm`, `ocmean`, `ocknn`, `ocgp`, `ockpca`

Kernel ridge regression is the method of the papers and the default. The Gaussian-process view adds a posterior variance per prediction at no extra cost, which is what the confidence-based alignment of *Preparing the data* minimises. The Laplace GP classifier adds calibrated probabilities, a posterior variance and model selection by marginal likelihood — it scores no better than C-SVC on the benchmark sets (mean difference 0.001 in Matthews correlation), so it is not the default, but its probabilities are calibrated and its evidence picks hyper-parameters without holding anything out.

Fitting the order instead of the values: the ranking SVM

A regression model is asked for the activity and judged on the squared error, which assumes the numbers are exact. Often they are not. Potencies pooled from several papers carry an offset per source that no descriptor explains, a series measured in two assays carries two scales, and an IC50 read off the end of a dose–response curve carries a bound rather than a value. In all three the *order* survives what the values do not, and a model fitted to the values spends its capacity on the part that is noise.

`--method ranksvm` fits the order. For every pair of molecules whose activities differ it asks that the more active one score higher by a margin, and is penalised for every pair where that fails:

$$\min \tfrac{1}{2}\|w\|^2 + C\sum_p \xi_p \;\text{ with }\; w\cdot(X_i - X_j) \geq 1 - \xi_p$$

over the pairs $p = (i, j)$ with $y_i > y_j$ — an ordinary support-vector classification on the *difference* of two molecules' fields. The kernel of two such differences is a sum of four ordinary kernel entries,

$$K_{\text{pair}}((i,j),(k,l)) = K(i,k) - K(i,l) - K(j,k) + K(j,l)$$

so the whole thing is built from the kernel matrix the program already has, with no new fields and no new parameterisation.

The fitted model is still a dual expansion over molecules. The weight vector is $w = \Sigma_p a_p (X_i^p - X_j^p)$, so collecting the terms gives one coefficient per *molecule* and the score of a new one is $K(x^p, \text{train})$ **a**, exactly as for every other learner here, with no bias. Everything that reads a linear CMF model — co-fields, atom contributions, Shapley values, the pharmacophore — keeps working. What changes is what they mean: the co-field of a ranking model says where more field *promotes* a molecule, not what it adds to the activity.

What is given up is the scale. A ranking score has no units and cannot be compared with an activity; only its order means anything. The program therefore reports Spearman's ρ, Kendall's τ and the share of pairs ordered correctly, and reports no r^2 or RMSE, which would be arithmetic on a quantity that does not exist. The hyper-parameter search selects on ρ for the same reason.

```
python cmf_cli.py build data.sdf --charges file --method ranksvm --Cs 0.1,1,10 \
    --rank-gap 0.5 --rank-groups paper
```

Three options shape the pairs. `--rank-gap` drops the pairs whose activities differ by less than it: a pair separated by less than the experimental error is a matter of chance, and asking a model to call it is how a ranking model learns noise. Half a log unit suits pooled literature potencies and zero suits one careful assay. `--rank-groups PROP` forms pairs only inside a group — the same paper, assay or series — which is what makes the method useful on pooled data, because an offset between two sources then cancels instead of being compared across. `--rank-pairs` caps the number of pairs: the solver works on the kernel *between pairs*, so the cost is quadratic in that number, and the default of 4,000 already asks for 128 MB. The order of a set is heavily over-determined and a few thousand comparisons fix it, so the cap is a bound on cost rather than a loss of information.

When it is worth it, measured

`benchmarks/run_ranking.py` asks the only question that matters — does fitting the order beat fitting the values — on the eight Sutherland sets with five random 70/30 splits each, every model choosing its own hyper-parameters on its own criterion inside the training part.

On clean data it does not. Where the activities are one assay's numbers and mean what they say, Spearman's ρ on held-out molecules is **+0.713 for kernel ridge and +0.689 for the ranking SVM**, ridge ahead on five of the eight sets. That is the expected answer and worth stating plainly: a regression model is already a ranking model, since the order of its predictions is all a ranking model ever produces, and it had the magnitudes to learn from as well. Fitting the order discards how far apart two molecules are, which on reliable data is information.

On pooled data it does, but only with the groups. The second experiment gives the same molecules to three synthetic sources — assigned *by scaffold*, because an offset that travels with the chemistry is what a model will learn as if it were an effect of the chemistry, while an offset assigned at random is simply averaged away — shifts each source by 1.5 standard deviations, and asks for the order of the true activity, which no model ever saw:

	KRR	ranking	ranking by source
same-source pairs ordered correctly	72.5 %	71.6 %	**74.3 %**
Spearman ρ over all held-out pairs	+0.574	+0.565	**+0.663**

A ranking model that is not told about the sources does no better than ridge: it is handed the same corrupted comparisons and learns them. Told which molecules share a source, it beats ridge by **0.089 in ρ**, on six of the eight sets, because the cross-source pairs — the only ones the offsets corrupt — are never formed. Note that it wins on the *global* order too, although it was never shown a cross-source comparison: it does not need one, since it fits a single scoring function over the fields and all that grouping did was stop it fitting an offset.

So the summary is narrow and useful: **the ranking SVM is for data whose numbers are not comparable across their sources, and its value is** `--rank-groups`**.** On a single clean assay, use kernel ridge.

Nonlinear kernel layers and multiple kernel learning

A layer on top of the CMF kernel makes the model more expressive than an overlap integral: a Gaussian layer, $\exp(-\gamma\, d^2)$ with $d^2 = K(i,i) + K(j,j) - 2K(i,j)$, or one or two arc-cosine layers, the kernel of an infinitely wide neural network (Cho & Saul 2009). The cost is interpretability: the model is then no longer linear in the field, so co-fields and atom contributions are unavailable. The similarity decomposition stays valid, and the **Shapley tab** decomposes the prediction instead, which is why it exists.

Multiple kernel learning chooses the mixing coefficients h_f rather than leaving them at 1, either by coordinate descent on the cross-validated error, by centred kernel alignment (`--mkl align|alignf`), which is a closed-form, training-only criterion, or by the marginal likelihood of the mixture (`--mkl evidence`), which is the next section.

Which fields does this series need?

A sum of kernels is the kernel of a sum of independent processes,

$$f(x) = \sum_f f_f(x), \qquad f_f \sim \mathcal{GP}(0,\ h_f c_f K_f),$$

so h_f is not a tuning parameter but the **prior variance of the part of the model that field f carries** — how much of the activity that field is allowed to explain. Put a prior on it and integrate, and the answer to *does this series care about the steric field at all* comes with an interval instead of a number:

```
python cmf_cli.py mixing model.cmf
```

What is sampled is each field's **share** of the kernel, written $h = F\,\mathrm{softmax}(u)$ so that the shares sum to one and can be compared with $1/F$, the share a field would have if all fields mattered equally. The prior is $u_f \sim \mathrm{N}(0,\ 1.5^2)$, independent and the same for every field, so it prefers no field over any other and only limits how far apart their shares can be drawn before the data are taken into account; the regularisation is sampled with them. The sampler is a slice sampler, which has no step size to tune.

Three readings come out, in increasing order of sharpness. The **shares with their intervals** say how much of the kernel each field carries. The list of fields **carrying less than an equal share** names those whose whole interval lies below $1/F$. And the **Bayes factor** against *this field does nothing* — the log evidence lost when the field is removed outright and the others are re-weighted — is the sharpest and the cheapest form of the question.

What it is good for, measured

Eight Sutherland sets, five 70/30 splits each. First, whether weighing the fields predicts better at all — the weights set four ways on the training part, each scored on the molecules held out:

weights	R^2pred
equal (h = 1)	+0.481
centred kernel alignment, per field	+0.487
centred kernel alignment, jointly	+0.485
the evidence optimum	+0.485

The evidence optimum is +0.003 in R^2pred against equal weights, 95 % interval [-0.026, +0.035], which contains zero — the same lesson as the per-field attenuation: **the evidence takes whatever freedom it is given over the shape of the kernel, improves its own number, and leaves the prediction unchanged.** Choosing the weights is not how a CMF model is made better.

What the posterior is *for* is the other question, and that one can be checked. Each field's Bayes factor against *this field does nothing*, beside what actually happens to the held-out molecules when that field is removed outright:

field	mean share	90 % interval	Bayes factor	ΔR^2pred without it
Electrostatic	0.179	[0.024, 0.432]	+1.4	-0.032
Steric	0.337	[0.091, 0.663]	+2.2	-0.012
Hydrophobic	0.184	[0.021, 0.460]	+0.7	-0.006
H-bond acceptor	0.159	[0.032, 0.385]	+1.6	-0.018
H-bond donor	0.142	[0.025, 0.349]	+1.5	-0.003

Over all 200 (set, split, field) cases the Bayes factor and the cost of removing the field correlate at Spearman ρ **+0.121** (positive means the evidence was right). The two orderings barely agree: the Bayes factor says what the training activities can see, which is not the same as what the held-out molecules need, and it should be read as the first and not the second.

The intervals are wide, and that is the answer rather than a failure of the sampler. On a congeneric series the field kernels are strongly correlated: several quite different mixtures explain the activities about equally well, and a point estimate — the alignment's, the evidence optimum, or any other — is claiming a precision the data do not support. That is the argument for integrating rather than optimising here, and it is the reason `--mkl evidence` exists as an option rather than as a default.

Choosing α and λ without holding anything out

Everything above chooses hyper-parameters by cross-validation: ten fits and ten predictions per candidate, and the best held-out score wins. A kernel ridge model is the posterior mean of a Gaussian process, and a Gaussian process scores its own hyper-parameters without holding anything out. The marginal likelihood — the probability of the training activities with the weights integrated away — is one determinant and one quadratic form on the factorisation the fit needs anyway, so a candidate costs **one factorisation instead of ten**:

```
python cmf_cli.py build data.sdf --charges file --select evidence
```

Two adjustments are made to it, and both matter. The bias of the program's KRR is a constant mean in the Gaussian process, so it is **marginalised under a flat prior** rather than fitted and then scored, which would count it twice. And the amplitude of the kernel — unknowable, since the kernels are cosine-normalised and the activities arrive in whatever units the assay used — is **profiled out** at its closed-form optimum, so the criterion depends on α and λ alone and its ranking does not move when the activity is rescaled.

Measured, it chooses as well and costs a quarter as much. Over the eight Sutherland sets, five random 70/30 splits each, with both criteria searching the same grid on the training part and both choices scored on the held-out molecules:

	R^2_{pred}
chosen by cross-validation	+0.471
chosen by the evidence	+0.479
paired difference	**+0.007**, 95 % interval [−0.013, +0.028], better on 6 of 8 sets

The two are indistinguishable in quality and the evidence is four times faster at selecting. The detail worth more than either number is that **they agree on the point of the grid in 0 to 60 % of splits** and reach the same predictive quality anyway: the surface is flat near its optimum, so what a search costs matters more than which point it selects. That is also what makes a finer grid — a separate α per field, say — affordable for the first time, and the next section uses the saved time for exactly that and reports what it gained, which was nothing.

The evidence is not a cross-validated score and does not pretend to be. It answers *how probable are these activities under this model*, not *how well would this model predict a molecule it has not seen*, and the two can disagree when the kernel is misspecified, which for molecular fields is the normal case. On these sets they do not. It is available for regression with `krr` or `gp`; the support-vector methods and the classifiers have no such quantity and are refused rather than silently cross-validated.

One width per field

One α for five fields is a convention, not a result. Electrostatics is long-ranged and a hydrogen-bond donor is nearly a point; there is no reason the same attenuation suits both. The protocol of the papers shared it because the computational cost required it — a grid over one width is five kernels, a grid over one width per field is 5^5 of them — and the evidence removes that objection twice over. The per-field kernels are independent, so every combination is assembled from the matrices the shared grid already built, and each candidate costs one factorisation. A coordinate descent over the widths, starting from the best shared one, then costs a few hundred:

```
python cmf_cli.py build data.sdf --select evidence --alpha-search per-field
```

It reports the width of each field and what the extra freedom gained, in the units it was selected in. Fields whose kernel does not depend on α at all — the probe fields, whose widths come from their own expansion — are left alone rather than searched in vain.

Measured, the extra freedom is real and gains nothing. Eight Sutherland sets, five 70/30 splits each, the shared width chosen by the evidence on the training part and the descent started from it:

log evidence gained by the per-field widths	**+2.6**, on every one of the 40 splits
widths moved away from the shared one	3.5 of 5 on average, and at least one on every split
R^2pred against the shared width	**−0.001**, 95 % interval [−0.020, +0.018], better on 4 of 8 sets
the same count per split	ahead on **20 of 40**
correlation of the evidence gained with the R^2pred gained	**−0.07**

The descent does what it was asked: it moves most of the widths, improves its criterion every time, and settles on widths a chemist would find reasonable — the steric field sharpest (mean α 2.33), the donor field widest (1.16). The molecules held out cannot tell the difference, and the gain in evidence carries no information about whether a split improved.

This is the same flat surface the previous section found in a different way. There, two different criteria picked different shared widths and predicted equally well; here, one criterion is given five widths instead of one, moves them, and predicts equally well. Around its optimum the attenuation does not decide much, and five of them decide no more than one — which is worth knowing, because it says the search is not what decides the accuracy of a CMF model. The alignment is (see *Alignment*, and the third part of `docs/benchmark.md`). Use `--alpha-search per-field` to ask the question of a new series, not as a default.

Validation

- **k-fold cross-validation** (10 by default) gives q^2 and RMSEcv. Folds are shuffled unless you ask for the interleaved folds of the original scripts (`--cv-no-shuffle`).
- **A test set** gives R^2pred and RMSEpred. Both variance conventions are reported, because they differ for sets whose test activities span a narrow range — the steroid benchmark is the classic example, where the ten test affinities cover 1.9 log units and R^2pred computed against the test variance is negative while the predictions are good.
- **External (nested) cross-validation** (`--nested`, *External CV*) wraps the whole model selection in an outer loop and reports q^2ex and RMSEcvex. This is the number to quote: the ordinary q^2 has seen the hyper-parameter choice.
- **y-randomisation**, a **bootstrap** interval and the **applicability domain** are the three sections below.
- **Leave-one-chemotype-out** folds, for a set of several scaffolds, never leave a molecule's own scaffold-mates in the training part. Random folds overstate the quality of such sets badly — by 0.11 to 0.47 q^2 on the five chemotype benchmarks here — and the program reports both, together with an alignment-free fragment baseline that the aligned model has to beat.

Is it chance? y-randomisation

A cross-validated q^2 says how well the model predicts; it does not say whether a model of the same shape could have reached that number from activities that mean nothing. In a congeneric series, where kernel values above 0.97 are normal, that is a real possibility, and it is the first thing a referee asks about a 3D-QSAR model.

The **shuffles** box of the model panel's *Validation* row (`--y-scramble N`) shuffles the activities N times, refits and cross-validates each time, and reports where the real q^2 falls among the scrambled ones — the mean and spread of the shuffled values, the largest of them, how many standard deviations above them the real one lies, and the permutation p-value.

```
python cmf_cli.py build data.sdf --alphas 0.5 --lams 0.1 --y-scramble 100
```

Two properties of the test are worth knowing, and the program states both rather than letting the number mislead:

- **A permutation test cannot report a p below 1/(N+1).** Ten shuffles cannot go below p = 0.091, which is above every conventional threshold *however good the model is*. The program says so — "no shuffle came close, but 10 of them cannot support a p below 0.091" — instead of reporting a chance-correlation warning that the data do not support. Use 19 shuffles for $p < 0.05$ and 99 for $p < 0.01$.
- **The hyper-parameters are held fixed** by default, so the test asks whether the *fit* can be reached by chance. Tick **redo the search** beside the shuffles box (`--y-scramble-select`) for the stricter question — whether the whole procedure, model selection included, can be. It matters: on the steroid set with a 3 x 3 grid, the best scrambled q^2 rises from −0.08 to **+0.32** once the search is allowed to pick the grid point that best fits the noise, while the real value stays at +0.78. The kernels do not depend on the activities, so they are computed once and the extra cost is one cross-validation per grid point per shuffle.

Where does the model apply?

A kernel method always returns a number. For a molecule unlike anything in the training set, that number is the training mean presented as a prediction, and nothing in q^2 or R^2pred reveals it.

The **applicability domain** tick of that row (`--domain`) measures how close each molecule outside the training set is to it, **in the kernel the model itself uses**: the cosine similarity of their fields, averaged over the *k* nearest training molecules (3 by default). The threshold is not an arbitrary number — it is taken from the training set's own spread, the 5th percentile of the same quantity computed among the training molecules, so a series of close analogues gets a tight domain and a diverse set a loose one. Molecules below it are listed by name in the log, their similarity appears in the data-set table's **AD** column — which sorts like any other, so the extrapolations can be brought to the top — and the exported predictions carry both the value and a yes/no verdict, so that an extrapolation stays marked after it leaves the program.

On the steroid benchmark, nine of the ten test compounds fall inside and one — 4-pregnene-3,11,20- trione — falls outside. On BZR, training on the largest chemotype and scoring the others, every molecule of the training chemotype is inside and 29 of 40 molecules from other chemotypes are outside, which is the behaviour one wants from a domain estimate.

The field-space distance is reported alongside the similarity, because the two can disagree: a molecule that is a scaled-up version of a training molecule points the same way (high cosine) but

sits far away (large distance).

Is the whole library inside? The two-sample test

The applicability domain answers the question one molecule at a time. A screening run asks a different one — is this library drawn from the same population as the training set, or is the whole exercise an extrapolation? — and averaging a per-molecule verdict does not answer it: two sets can have identical nearest-neighbour similarities and still occupy different regions.

Every set of molecules has a **mean field**, the mean of its feature maps, which for CMF is itself a function of space. The maximum mean discrepancy (Gretton *et al.*, JMLR **13** (2012) 723) is the distance between two of them:

$$\mathrm{MMD}^2(X,Y) = \overline{K(x,x')} - 2\,\overline{K(x,y)} + \overline{K(y,y')}$$

— three averages over blocks of the kernel matrix and nothing else. The unbiased estimator leaves the self-similarities out of the two within-set terms, which is why it can come out slightly negative: that is how "no difference at all" looks once the bias has been removed, not a defect. The null distribution comes from permuting the labels, which costs three block means per shuffle because the kernel is computed once.

```
python cmf_cli.py compare library.sdf --model model.cmf --permutations 999
python cmf_cli.py compare seriesA.sdf seriesB.sdf --charges file
```

With `--model` the library is compared with that model's training set, in that model's own fields, reusing the cross-kernel the prediction computes anyway. The p-value is never zero: it is one plus the number of shuffles that matched or beat the observed value, over one plus the number of shuffles, because a finite number of them cannot support a claim of impossibility.

The witness says where the difference lies, which is the useful form of a p-value. It is the function whose expectation separates the two sets — a molecule's mean kernel value against the first set minus its mean against the second — evaluated at every molecule: most negative for the compounds that make the library unlike the training set, most positive for those most typical of it. The command prints both ends and `--out` writes the whole column. On the Sutherland benzodiazepine set against the COX2 one, the witness names nordazepam and delorazepam as the most typical benzodiazepines and the COX2 sulphonamides as the least — without being told that there are two targets.

Is the test any good?

A test is useless unless it is calibrated as well as powerful, and `benchmarks/run_structure.py` measures both halves. Twenty random half-splits of each of the eight Sutherland sets, where there is nothing to find: **5.6 % are called significant at the 5 % level**, which is what a correctly calibrated test does and the permutation construction guarantees. All twenty-eight pairs of *different* targets, where there is everything to find: **100 % are called significant, median z = +171**. The test is calibrated where there is nothing and saturated where there is.

How certain is that number?

A difference of 0.02 in q^2 is quoted all over the benchmarks in this repository. Some of those differences are real and some are the particular molecules they were computed over.

The **bootstrap** box of that row (`--bootstrap N`) resamples the (observed, predicted) pairs N times and reports a percentile confidence interval for q^2 and for R^2_{pred}. It is the uncertainty of the *statistic* given these predictions, which is exactly what decides whether two settings differ. On the 21 training steroids, $q^2 = 0.768$ carries a 95 % interval of [0.47, 0.89] — wide enough to show that a difference of a hundredth means nothing.

`--bootstrap-refit N` adds Cramer's bootstrap as well: resample the training set with replacement, refit, and score the molecules that were left out of the bag. That one measures the variability of the fit rather than of the statistic, and it is the more pessimistic of the two. The kernel is never recomputed — a resample is an index into the matrix — so a hundred refits cost about as much as one cross-validation.

How certain is one prediction? Conformal intervals

The bootstrap says how firm a figure of merit is. It says nothing about the molecule in front of you, and neither does anything else the program has offered so far: the Gaussian-process posterior spread is the spread of a model that believes its own kernel, which for molecular fields it should not.

Conformal prediction supplies the missing guarantee and requires almost nothing in return. Take the absolute errors the model makes on molecules it did not fit, let q be their $(1-\alpha)$ quantile — the $\lceil (n+1)(1-\alpha) \rceil$-th smallest, the $+1$ being what makes the result finite-sample rather than asymptotic — and report $\hat{y} \pm q$. Then, as long as the calibration molecules and the new one are exchangeable, the truth lies inside at least $(1-\alpha)$ of the time, **whatever the learner, whatever the kernel, however wrong the model is**. A badly specified model does not lose its coverage; it pays with a wider interval instead.

```
python cmf_cli.py build data.sdf --charges file --conformal 0.9 --save model.cmf
python cmf_cli.py predict model.cmf new.sdf --out predictions.csv
```

`--conformal` calibrates on the errors of the cross-validation the program is running anyway, so it costs nothing and adds no step, prints the half width, and — when there is a test set — the coverage it actually achieved on molecules that were never part of the calibration. The rule is stored in the `.cmf` file, so `predict` prints an interval beside every number and writes `lower` and `upper` columns into the CSV. `--conformal LEVEL` on `predict` re-reads the stored errors at a different level; the errors are kept, not just the width.

What it costs, measured

Calibrating on out-of-fold errors and then predicting with the model refitted on everything is not exactly exchangeable — the errors come from models fitted on nine tenths of the training set and the predictions from a model fitted on all of it — so the guarantee becomes an approximation, and the size of the gap is an empirical question. `benchmarks/run_conformal.py` answers it on the eight Sutherland sets, ten random 70/30 splits each, with the width in units of the activity (whose standard deviation averages 1.36):

nominal	out-of-fold (`cv`)	held-out third (`split`)	GP posterior
80 %	**80.3 %**, width 2.28	80.1 %, width 2.59	77.7 %, width 3.00
90 %	**89.9 %**, width 3.10	90.1 %, width 3.61	83.1 %, width 3.85
95 %	**95.1 %**, width 4.09	94.9 %, width 4.04	86.9 %, width 4.59

The cheap variant holds its level to a few tenths of a per cent at every level, and is **narrower** than the exact split version, which loses accuracy because it fits on less data after holding a third out. The Gaussian-process interval, which is what one would otherwise reach for, misses its level at all three — and misses it unpredictably: across the eight sets its 90 % coverage ranges from 63 % to 99 %, while every conformal rule stays between 89 and 96 %.

Normalised conformal — dividing each error by the distance of its molecule from the training set, so that the interval widens where the model extrapolates — is available as `--conformal-normalise`, and on these sets it gains nothing: it over-covers (93.4 % at a nominal 90 %) and is a third wider. Softening it until it stops over-covering turns it back into the plain rule. The error on these series simply does not grow with distance from the training set strongly enough for the normalisation to be useful, which is itself worth knowing about one's data.

Two limits worth stating. The guarantee is **marginal**, not conditional: 90 % of molecules are covered, not 90 % of *every kind* of molecule, so a subclass the model handles badly can be covered less often than that while the overall figure holds. And a level needs molecules to support it — 19 calibration errors for 95 %, 9 for 90 % — below which the program says the level is not reachable rather than returning its widest error and calling it an interval.

Regression only: conformal classification produces prediction *sets* rather than intervals, and is not implemented.

Three estimates of the error, compared

The program now offers three answers to *should I believe this number*, and they come from three different places: the **Gaussian-process posterior variance**, which is the kernel's own statement about how much of this molecule lies in the span of the training set; the **applicability domain**, which is a geometric statement about how far it is from its nearest training neighbours; and the **conformal interval**, whose normalised form widens where the model is extrapolating. They are in three different units — a variance, a distance in field space, a width in log units of activity — so they cannot be compared by their numbers. What can be compared is the order they put the molecules in, because that is what they are used for: which prediction do I act on, which do I measure first.

```
python cmf_cli.py build data.sdf --test-file test.sdf --domain \
       --conformal 0.8 --conformal-normalise --error-predictors
```

Each estimate is scored against the absolute error actually made on the test molecules, three ways. **Spearman's ρ** asks whether the order is right at all. The **risk–coverage efficiency** asks what is gained by acting on it: trust the most confident fraction, measure the RMSE of that fraction, and compare the area under that curve with two references — the *oracle*, which orders the molecules by the error they really made, and the flat curve of ignoring the estimate. One means the oracle, zero means no better than ignoring it, and a negative number means actively misleading. The **halves** are the crudest and most practical: the RMSE of the half it trusts against the half it doubts.

Measured, the three are indistinguishable. Eight Sutherland sets, five 70/30 splits each, α and λ chosen by the evidence on the training part, the conformal rule calibrated on that part's own out-of-fold errors:

estimate	ρ	efficiency	RMSE trusted	RMSE doubted	ratio
GP variance	+0.142	**+0.198**	0.835	1.024	1.38

estimate	ρ	efficiency	RMSE trusted	RMSE doubted	ratio
distance to the training set	+0.125	+0.186	0.853	1.009	1.31
domain dissimilarity	+0.133	+0.181	0.841	1.022	1.36
normalised conformal width	+0.133	+0.181	0.841	1.022	1.36

The GP variance is ahead by +0.011 in efficiency over the distance, with a 95 % interval of [−0.021, +0.044] — zero is inside it, and the choice between the three does not matter.

Two things in that table deserve more attention than the ranking. The first is that the **normalised conformal width and the domain measure score identically, and must**: the normalisation divides the interval by the molecule's difficulty, which *is* its distance from the training set, so the width is a monotone function of the domain and cannot order the molecules differently. The conformal rule is justified by being calibrated — it covers what it says it covers, which neither of the others claims — and not by ordering. There are three estimates here, not four.

And the remaining two are nearly the same. Over the same splits the estimates agree with each other at a mean Spearman ρ of **+0.80 to +0.86** — the GP variance against the distance to the training set at +0.799, against the domain dissimilarity at +0.853 — while the conformal width against the domain measure is **+1.000**, exactly as the algebra says it must be. That is why the efficiencies in the table are indistinguishable: these are not three independent opinions about a molecule but one geometric fact, how far it sits from the training set, expressed in three sets of units.

The second is the size of all of them. The best captures about a fifth of what the oracle would, and the half it trusts has an RMSE of 0.84 against 1.02 for the half it doubts. That ratio of about 1.4 is real and useful — it is exactly the gain from measuring the confident half first — and it is nothing like a per-molecule error bar. On one of the eight sets (THER) every estimate comes out negative: an estimate of error is itself a model, and it can be wrong about a series. Use them to order molecules, and quote the conformal interval when a number has to be attached to one.

What the model thinks the groups are: clustering the kernel

Every grouped validation in this manual splits the molecules by their Bemis–Murcko scaffold, and the reasoning is sound: random folds put a molecule's own scaffold-mates in the training part and greatly overstate the quality of a mixed series. But it rests on an assumption nobody checks — that a scaffold is what makes two molecules alike *to the model*. The model does not see scaffolds. It sees fields, and its notion of similarity is the kernel.

Spectral clustering partitions that similarity directly. The kernel is already a similarity matrix, so there is nothing to build: form the normalised Laplacian, take the eigenvectors belonging to its smallest eigenvalues, put the rows on the unit sphere and run k-means there (Ng, Jordan & Weiss 2002). Groups that are not linearly separable in field space are separable in those coordinates, which is the point of the construction. How many groups is decided by the **eigengap** unless it is given, and the gap is reported beside the answer so that a weak one can be seen for what it is.

```
python cmf_cli.py cluster data.sdf --charges file
python cmf_cli.py cluster data.sdf --clusters 5 --out clusters.csv
```

The command prints the clusters, the spectrum they came from, the activity of each, and the **adjusted Rand index** between them and the scaffold groups — the number that says whether the usual grouping groups by what the model actually responds to.

The two partitions do not agree, and it matters

Measured on the eight Sutherland sets, with both partitions cut into the same number of groups — holding out one of two clusters removes half the training set and holding out one of nine does not, so a difference in granularity would be read as a difference in difficulty:

	q^2
random 10-fold	+0.508
leave one **scaffold group** out	+0.271
leave one **field cluster** out	+0.057

and the adjusted Rand index between the two partitions averages **+0.140**, which is weak agreement: what the chemist calls a series and what the kernel finds similar are largely different things. On BZR they are different enough that holding out a field cluster takes q^2 to −0.740.

The consequence is uncomfortable and worth stating. This manual recommends the leave-one-chemotype-out figure as the more reliable one, and against random folds it is — 0.271 against 0.508. But it is not the hardest reliable test available: holding out a group of the model's own similarity costs another 0.214, because a scaffold group leaves molecules the kernel still considers close in the training part. Quote the scaffold figure, by all means; just do not read it as the worst case.

Where the validation appears

The three checks of *Building a model* — y-randomisation, the bootstrap intervals and the applicability domain — are set in one place and reported in another, which is worth stating plainly because there is no button and no separate window for them.

Setting them: the *Validation* row at the foot of the model panel's second group, directly under *External CV*. Three controls: **shuffles** (0 = off), **bootstrap** (0 = off), and the **applicability domain** tick. They are settings, not actions — choose them before pressing **Build model**, and they run as part of that build, after the fit and the test prediction.

Reading them: the *Statistics* tab of the results panel, where they are appended below the usual figures under a heading *Validation*. The names of the molecules that fell outside the applicability domain go to the *Log* tab instead, with their similarity values, so that the statistics stay a summary and the list of molecules outside the domain stays with the rest of the log.

Three cases produce no validation block at all, and each says why rather than leaving a blank:

- all three controls at their defaults — nothing was asked for;
- a classification or one-class model — y-randomisation and the bootstrap are implemented for regression only, and the block says so;
- the autocorrelation kernel source — the applicability domain is measured in the field kernel, which that source does not provide.

Statistics reported

Regression: r^2 and RMSE for the fit, q^2 and RMSEcv for the cross-validation, R^2pred and RMSEpred for the test set in both variance conventions, q^2ex and RMSEcvex for the external CV. Classification: accuracy, balanced accuracy, sensitivity, specificity, Matthews correlation, AUC and the ROC curve, each for the fit, the cross-validation and the test set. Screening: AUC, enrichment factors at 0.5, 1, 5 and 10 %, the best achievable enrichment at each of those fractions, BEDROC and RIE.

Saving and reusing

File → Save model... writes a `.cmf` file (joblib) holding the field set, the kernels' definition, the fitted coefficients, the training molecules, every setting needed to predict later, and the record of how the training set was prepared that *What has to match between fitting and predicting* describes. `cmf_cli.py predict model.cmf new.sdf --out predictions.csv` applies it, after checking the new molecules against that record; the GUI loads it with *File → Load model...* and writes the record's summary to the log.

A report to hand to a colleague

A `.cmf` file is for the program. The person who has to judge the model — a colleague, a reviewer, a supervisor — will never start the application, and what they need is one file they can open.

```
python cmf_cli.py report model.cmf --out model.html
```

That writes a single self-contained HTML page: every picture is inside it, so it survives being emailed, attached or put on a share, and it opens in any browser and prints to PDF. It has seven sections, in the order a sceptical reader asks for them.

- **What this model is** — fields, α, learner, kernel, how the charges were made.
- **How well it fits and predicts** — the statistics with the observed-against-predicted plots beside them, because a q^2 is a summary of that picture and not a substitute for it.
- **Whether to believe that** — the validation that was actually run: y-randomisation, the bootstrap intervals, the applicability domain, the conformal level, the comparison of error predictors. **A check that was not run is reported as not run**, never silently omitted, which is the difference between a report and an advertisement.
- **What the model rests on** — the mean absolute contribution of each field, which is what separates a model standing on its geometry from one standing on a single field.
- **The co-fields**, projected onto the three coordinate planes from fixed viewpoints: the strongest signed value along each viewing axis, with the training atoms as grey dots for scale. A projection rather than an isosurface, because a report has no camera and its reader has no mouse.
- **The extremes of the series** — the most and least active training molecules, each decomposed over the fields by exact Shapley values and over its own atoms by the contribution rule.
- **The pharmacophore** read out of the co-fields, and the provenance of the model file.

Everything comes out of the model file, so a report costs no refitting and **cannot disagree with the model it describes**. Missing parts are reported rather than causing a failure: a nonlinear kernel has no co-fields, no atom contributions and no pharmacophore, and the report says so in place of each and keeps the rest; a model fitted without a test set reports no R^2pred.

`--no-cofields` leaves out the pictures, which are the slow part — a report of a 50-molecule model takes about three seconds with them and under one without — and `--spacing` sets how finely they and the pharmacophore are computed. `--open` shows it in the browser straight away.

Reading the model

A CMF model is readable in a way a grid model is not, because the prediction can be taken apart exactly. The 2019 paper states three decomposition rules, and the program implements all three.

Co-fields

The co-field is the model as a function of space:

$$C_f(\mathbf{r}) = \sum_j a_j \, s_j \, X_{fj}(\mathbf{r})$$

a weighted sum of the training molecules' fields with the fitted dual coefficients as weights, each divided by the norm $s_j = K_f(j,j)^{-1/2}$ of its own field when the kernels are normalised, which they are by default. The prediction of any molecule is then the overlap of its own field with the co-field, field type by field type. Press **Compute co-field** and the program evaluates it on a grid around the training set; choose the field with the *Field* selector and set the *iso level* slider, which is a percentage of the largest absolute value rather than an absolute number, so the surfaces stay meaningful when you change field.

What the lobes mean: a positive co-field lobe is a region where *more* of that field raises the predicted activity. For the electrostatic field that means positive charge is favoured there, for the steric field bulk, for the hydrophobic field lipophilicity, and for an indicator field it means an atom of that type is favoured at that position.

Two further maps use the same grid, for the molecule currently shown:

- the **molecular field** $X_f(\mathbf{r})$ of that molecule, in the same colours. The two share a colour scale, so only the drawing style tells them apart: the style box is `auto` by default, which draws this one as a mesh while a co-field is on screen and solid when it is the only surface shown;
- the **overlap** $C_f(\mathbf{r})\, X_f(\mathbf{r})$, whose integral *is* that field's contribution to the prediction (Eq. 7), so the map shows where the prediction comes from.

File → Export current co-field (Gaussian cube) writes a cube file readable by PyMOL, VMD or Jmol, which is the way to put a co-field into a publication figure.

How much of it is noise

A co-field is read as a fact about the chemistry. It is a linear map of the measured activities, so part of its shape is a fact about the measurement error instead, and the program can say which part in closed form.

The dual coefficients are a linear map of the activities — with a bias, the bordered system of the 2013 paper, whose inverse's top-left block is the matrix below — so activities carrying independent noise of variance σ^2 give coefficients carrying covariance $\sigma^2\, G\, G^{\mathrm{T}}$, and the co-field, being linear in them, carries

$$\mathrm{Var}\,[C_f(\mathbf{r})] = \sigma^2 \left\| G^{\mathrm{T}} \mathbf{x}_f(\mathbf{r}) \right\|^2, \qquad x_{fj}(\mathbf{r}) = s_{fj} X_{fj}(\mathbf{r})$$

— one matrix product per block of grid points, no resampling and no refitting. The **hide noise** tick of the 3D view then draws only what lies more than two standard deviations from zero, and

`pharmacophore --min-z 2` reads features off the same masked map, so that a lobe the noise could have produced never becomes a feature. The report (`report`) does both by default.

The noise level σ is the model's own out-of-fold error, RMSEcv, which is the assay's noise and the model's error together; `--sigma` sets it directly.

What this is, precisely. It is not the Bayesian posterior variance of the co-field, and the difference is worth stating. Under the Gaussian process whose kernel this is, the prior on the field weight function is white noise on L^2: its value at a point is not a random variable at all, and the pointwise posterior variance is infinite everywhere. What the prior does allow is the variance of $\langle w, g \rangle$ for a probe field g of non-zero width — which is exactly the GP variance of a prediction for a one-atom molecule placed there, and the program already computes that. The quantity above is the other half of the question: the molecules and the kernel are held fixed, and the noise in the activities is propagated. It answers *how much of this lobe would survive the error in the measurements*, which is the question a co-field picture provokes.

Measured, the mask keeps about a third of the picture, and that third is the part that is stable. Eight Sutherland sets, co-fields on a 0.8 Å grid; the training set is then resampled with replacement thirty times and refitted, and a voxel counts as stable when the sign of its co-field survives that — a harder perturbation than the formula models, since it moves the molecules too:

kept by the mask, of what would have been drawn	**30 %**
of the kept part, holds its sign under resampling	**97.8 %**
of the cut part	**90.6 %**
difference	**+0.072**, 95 % interval [+0.064, +0.083], in the same direction on 8 of 8 sets

One more figure, for completeness: the bootstrap moves the co-field only about 0.6 times as far as the closed form allows for, so cutting at two standard deviations is really cutting at about 3.2 of the bootstrap's — the threshold is conservative, which follows from taking σ to be the out-of-fold error. What it gets right is *where* the uncertainty is, and the two orderings agree across the grid at a rank correlation of +0.82. That is what a mask needs, since a mask is an ordering with a cut-off.

Contributions

The *Contributions* tab answers two questions about the selected molecule:

- **Which training molecules** make its prediction, as $a_j K(X, X_j)$ — the similarity decomposition, rule 2. A prediction resting on one or two training molecules is a warning.
- **Which of its atoms**, as the atom contributions of Eq. 14 of the 2015 paper, drawn in the 3D view as red and blue spheres whose size is the magnitude. This is the map that is most useful for design: it says *this* substituent is helping and *that* one is not.

Shapley values

The *Shapley* tab decomposes a single prediction into additive contributions by the Shapley construction, over **atoms**, over **substituents of the common scaffold**, or over **field types**, with second-order interaction indices available. It serves two purposes. With the linear kernel it reproduces the atom contributions exactly, which is a useful check. With a nonlinear kernel layer,

where co-fields do not exist, it is the only decomposition left — and it still works, because it needs nothing but the ability to evaluate the model on subsets.

A pharmacophore out of the co-field

Tools → Extract pharmacophore from the co-field... turns the co-field into a discrete, typed pharmacophore: the favourable lobes of each field become feature points of the matching kind (positive and negative centres, hydrophobe, donor, acceptor), each with a tolerance radius taken from the size of its lobe, and the unfavourable steric lobes — or the pocket, when a protein is loaded — become exclusion volumes. The result is a table and a PDB-style file of feature spheres that other software can read, and it draws in the 3D view; untick *pharmacophore* in the viewer controls to clear it.

This is the bridge from a CMF model to a screening query: the model is continuous and needs an alignment, the pharmacophore is discrete and does not.

Taking the query out of the program

```
python cmf_cli.py pharmacophore model.cmf --min-z 2 --query query.json
python cmf_cli.py pharmacophore model.cmf --min-z 2 --query query.pml --enable-top 3
```

`.json` writes a **Pharmit** session and `.pml` the **LigandScout-style XML**; both are written to the documented shape of those formats, which is as far as this program can go — it cannot test any particular viewer. Three things about the file are worth knowing before it is opened somewhere else.

A query is a conjunction, so a dozen features at once matches nothing. The strongest `--enable-top` features are written switched on and the rest switched off, for the user to choose among in their own program; exclusion spheres are always on, since they only forbid.

Hydrogen-bond features have no direction. A co-field says that a donor *here* raises the activity; it says nothing about where the hydrogen points, so the features are written as isotropic points rather than as the vectors those formats usually carry.

Three of this program's feature types cannot be expressed at all and are reported rather than quietly dropped. `bulk` asks for atoms of any kind, which no query language expresses. The disfavoured `polar`, `acceptor_bad` and `donor_bad` say that one *particular* kind of group is unwelcome; the only negative a query has is an exclusion sphere, and that would forbid every atom, which is a much stronger claim than the co-field makes.

Is the query any good, measured

Eight Sutherland sets, five 70/30 splits each. The pharmacophore is read out of the training model's co-fields — masked at two standard deviations first, so that a lobe the measurement error could have produced never becomes a feature — and its strongest four features are matched against the molecules held out, by the simple rule a screening program uses: an atom of the right kind inside the sphere. The actives are the most active quarter.

set	n	features	AUC query	AUC random centres	AUC the model
DHFR	361	7.4	0.573	0.498	0.901
COX2	282	8.4	0.662	0.478	0.867
BZR	147	7.8	0.518	0.514	0.797

set	n	features	AUC query	AUC random centres	AUC the model
ACE	114	7.0	0.789	0.552	0.927
AChE	111	7.8	0.631	0.484	0.903
THR	88	8.4	0.595	0.468	0.780
THER	76	7.2	0.711	0.482	0.812
GPB	66	6.8	0.838	0.484	0.902
mean			**0.665**	**0.495**	**0.861**

The query separates the active quarter at an AUC of 0.665, against 0.495 for the same features moved to random places in the same box — a paired difference of +0.170 with a 95 % interval of [+0.102, +0.238], so the places the co-field picked are doing the work, and ahead on 36 of 40 splits. Over every split together the top tenth by the query holds 1.77 times as many actives as chance would put there.

The model it was read from reaches 0.861 on the same molecules, and that gap is the price of the summary: a pharmacophore is four spheres and a rule for counting them, while the model is every training molecule's field weighed by a fitted coefficient. What the query gains is that it can leave this program — it goes into a screening tool and runs over libraries no kernel method here could hold.

What a candidate is missing

Tools → Field difference from a reference molecule... subtracts two molecules' fields on the co-field grid and reports, per field type, the cosine between them, the distance, and how much of the reference's field the candidate **fails to reproduce** against how much it **adds**. The difference map draws as its own isosurface, and the table ranks the whole data set by field similarity to the reference. In a lead-optimisation reading: the "missing" column is what to put back, the "added" column is what may be causing trouble.

Several endpoints at once: the selectivity co-field

A separation problem, or any selectivity, is not one property but two: an extractant has an affinity for americium and an affinity for europium, and what matters is the difference. A model of the ratio works, but it throws information away. Each affinity has its own measurements, often on partly different compounds; a ligand that binds both metals strongly looks, to a model of the ratio, exactly like one that binds neither; and the ratio cannot say *which* of the two affinities a substituent moved.

Tools → Model several endpoints (selectivity co-field)... fits every endpoint you tick on the kernel of the model you have already built, with one bias per endpoint and a task covariance that lets them share information with each other:

$$y_t(X) = \sum_j c_{jt}\, K(X, X_j) + b_t,$$

where the coefficients of the two endpoints are tied through a single correlation ρ. At ρ = 0 the endpoints are modelled separately and share only the kernel; as ρ rises they share information; at ρ = 1 the two sets of coefficients coincide and the model could only predict a *constant* selectivity, so 1 is excluded (and ρ may be negative, for endpoints that move in opposite directions). The dialog searches ρ and the ridge by cross-validation, and the cross-validation holds out whole **molecules**,

never single measurements: with a molecule's first endpoint in the training set and its second in the test set, the correlation would reveal the answer.

Because each endpoint's prediction is linear in the field with its own coefficients, each endpoint has a co-field, and so does their difference — the **selectivity co-field**

$$C_A(\mathbf{r}) - C_B(\mathbf{r}) = \sum_j (c_{jA} - c_{jB})\, X_j(\mathbf{r}),$$

a map of where more of a field raises the first endpoint more than the second. Only a model linear in the field can produce it; it is available here for the same reason co-fields are available at all, and it is refused for a model with a nonlinear kernel layer.

Once the fit is made, an **Endpoint** box appears in the *Maps* row of the control strip with one entry per endpoint and one for the difference of the pair you named — and every map, export and contribution table then refers to the chosen one; a cube exported while the selectivity is chosen carries it in its file name. The endpoints are numeric properties of the data set, so the ones beyond the first are imported with *File → Import activities from CSV...* or read from columns of the file. Molecules that carry only some of them are kept: they inform the endpoints they have, and through the covariance the others. Each endpoint needs at least five molecules.

What the chosen ρ means is itself a result. On the corrosion-inhibitor series of the benchmark (`benchmarks/results/multitask/table.md`) the cross-validation chose ρ = 0 on the two largest series and a non-zero ρ of either sign on the small ones, where it gained at most 0.04 in q^2: an endpoint that already has its own molecules gains little from another, and the ρ the dialog reports in the log is the measurement that says how much.

Beyond one correlation

One ρ says that every pair of endpoints is related to the same degree. For two endpoints that is no assumption at all; for seven it is a strong one, and wrong in the usual case where some endpoints are nearly the same and others are only weakly related. The dialog's **Task covariance** box therefore offers a free-form matrix,

$$\mathbf{T} = \mathbf{L}\mathbf{L}^{\mathrm{T}} + \mathrm{diag}(\mathbf{d}),$$

with **L** of shape (endpoints, rank) and **d** > 0. It is positive definite for every value of its parameters, which is why nothing has to be constrained; at rank 1 it contains the one-ρ model exactly, so the comparison is meaningful; and its **diagonal is free**, which matters whenever the endpoints are in different units — a potency and a free energy in kJ/mol need their own amplitudes, and one shared correlation cannot give them one.

Such a covariance has (rank + 1) × endpoints parameters, which is far too many for cross-validation: ten fits per candidate and several hundred candidates. It is chosen instead by the **marginal likelihood**, the multi-endpoint form of the criterion of *Choosing α and λ without holding anything out*: the mean is marginalised over the design columns under a flat prior — one constant per endpoint, or the task descriptors below — and the kernel amplitude is profiled out, so one factorisation scores a candidate and the value does not depend on the units of the activities. `cmfqsar/multitask.log_evidence` is that criterion and `fit_covariance` optimises it by Nelder–Mead from several starts, because the evidence over a covariance is not concave and a start at ρ = 0.8 and one at ρ = 0.3 do not always converge to the same answer.

An endpoint nobody measured

A free-form row for an endpoint with no measurements is unidentifiable: there is nothing to fit it to. A row read off **task descriptors** is not. Give the endpoints their own descriptor vectors $\mathbf{z}_t$ — the medium, the metal, the temperature; in a host-guest problem the structure of the host itself — and set

$$T(t,s) = (1-\varepsilon)\exp\left(-\frac{|\mathbf{z}_t - \mathbf{z}_s|^2}{2\ell^2}\right) + \varepsilon\,\delta_{ts},$$

whose length scale ell and nugget varepsilon are chosen by the same evidence (`descriptor_covariance`). The nugget is not optional: without it two endpoints with identical descriptors would be perfectly correlated, the covariance singular, and their coefficients forced to coincide. A `linear` form is available too, and is the one to use when two endpoints are expected to be *anti*-correlated, since an RBF kernel cannot be negative.

The mean needs the same treatment, since an endpoint that was never measured has no bias to fit. Passing `task_features` to `Observations` replaces the indicator columns of the bias with the descriptors themselves, so that the mean is linear in them and extrapolates; with no features the design is the indicator matrix and the behaviour is exactly as before. Then

```
from cmfqsar.multitask import Observations, descriptor_covariance, fit
obs = Observations.from_columns(Y, names=hosts, task_features=Z_with_intercept)
dc = descriptor_covariance(K, obs, Z)
f = fit(K, obs, lam=dc.lam, T=dc.T)
y_new = f.predict_new_task(K_cross, dc.row_for(z_new), features=z_new_with_intercept)
```

predicts the unmeasured endpoint, and `f.new_task_coefficients(dc.row_for(z_new))` gives it a co-field like any other. What this cannot do is invent what is peculiar to that endpoint: it interpolates between the endpoints that were measured and inherits only what they have in common.

What it measures

Measured on `examples/cyclodextrin/` (`benchmarks/run_tasks.py`), which has seven real endpoints and descriptors for them: 1669 distinct guest molecules, 2671 association constants against the seven cyclodextrin hosts (beta 1162, alpha 827, gamma 292, hp_beta 135, sbe_beta 135, methyl_beta 77, dimethyl_beta 43), and the hosts are molecules too, so the task descriptors are a structure rather than a label — cavity size, lipophilicity, charge, hydroxyl count, heavy atoms. One kernel for every row, the rotation-invariant field autocorrelation at α = 0.1 (there is no frame available here, and *Working without an alignment* found that kernel the best of the three on this data), and 10-fold cross-validation over **guests**, so that no guest has one host in the training set and another in the test set.

task covariance	mean q^2	beta	alpha	gamma	hp_beta	sbe_beta	methyl_beta	dimethyl_beta	what it is
independent	**+0.412**	+0.448	+0.338	+0.473	+0.545	+0.331	+0.377	+0.374	rho = 0
one rho, CV	**+0.454**	+0.448	+0.338	+0.524	+0.586	+0.389	+0.441	+0.454	rho = +0.90
one rho, evidence	**+0.454**	+0.449	+0.341	+0.523	+0.592	+0.391	+0.435	+0.451	rho = +0.857 over 25 values

task covariance	mean q^2	beta	alpha	gamma	hp_beta	sbe_beta	methyl_beta	dimethyl_beta	what it is
free rank 1	**+0.475**	+0.447	+0.339	+0.525	+0.604	+0.444	+0.504	+0.465	2 x 1 starts, 1603 evaluations of the evidence
free rank 2	**+0.488**	+0.447	+0.340	+0.527	+0.624	+0.452	+0.536	+0.490	2 x 1 starts, 2274 evaluations of the evidence
descriptors	**+0.448**	+0.446	+0.342	+0.505	+0.608	+0.390	+0.430	+0.413	rbf, length scale 4, nugget 0.1

Sharing is worth +0.076 mean q^2 — +0.412 fitting the seven hosts independently on the same kernel against +0.488 for the free rank 2 covariance — and the gain is largest on the endpoints with fewest measurements, which is what sharing information between endpoints is supposed to do (the gain and the number of measurements correlate at -0.89, and the biggest gain goes to methyl_beta, while the smallest set is dimethyl_beta). A free-form covariance beats one shared correlation by +0.034, so with seven endpoints the single number misses part of the gain.

The evidence was the objective for every row but the cross-validated one, and it is worth checking whether it chose what cross-validation would have:

task covariance	log evidence	mean q^2
free rank 2	-2605.3	+0.488
free rank 1	-2608.8	+0.475
one rho, evidence	-2635.3	+0.454
one rho, CV	-2635.8	+0.454
descriptors	-2647.6	+0.448
independent	-2713.1	+0.412

It did: the covariance with the highest evidence is also the one with the highest out-of-fold q^2, which is the agreement that is assumed without a check elsewhere in this manual.

An endpoint nobody measured, measured

Each host was then removed **entirely** — the whole endpoint, not a fold of its guests — and predicted from the other six through the descriptor covariance, with the mean linear in the descriptors so that there is something to extrapolate. Spearman ρ is the column to read: the absolute scale of an endpoint nobody has titrated is not recoverable, and a screen ranks.

host removed	n	nearest host	ρ, descriptor covariance	ρ, nearest host alone	ρ, mean of the measured hosts	q^2, descriptor
beta	1162	sbe_beta	+0.577	+0.320	+0.623 (615)	+0.231
alpha	827	beta	+0.431	+0.433	+0.470 (522)	+0.066
gamma	292	sbe_beta	-0.284	+0.145	+0.533 (225)	-5.499
hp_beta	135	sbe_beta	+0.664	+0.294	+0.806 (105)	+0.483
sbe_beta	135	beta	+0.516	+0.515	+0.498 (74)	+0.200
methyl_beta	77	dimethyl_beta	+0.595	+0.529	+0.894 (73)	+0.416
dimethyl_beta	43	methyl_beta	+0.703	+0.708	+0.863 (38)	+0.044

Mean ρ **+0.457** through the descriptor covariance, against **+0.421** for the single-task model of the nearest host by descriptors and **+0.670** for simply averaging the other hosts' measured values of the same guest. So the covariance does carry real information about an endpoint it has never seen — it beats the nearest host's own model 4 times out of 7 — and it does not beat the cheapest baseline of all. That baseline is not a model, which is its only disadvantage: it can only say something about a guest some other host was measured against, and says nothing about a new molecule. The q^2 column is the necessary warning: it is negative on 1 of the 7 hosts, so the numbers this route produces are a ranking and not a binding constant.

What governs the spread is whether the removed host is interpolated between the others or extrapolated to. Cavity size is the first descriptor, and of the seven hosts five have seven glucose units — β-cyclodextrin and its four derivatives — while **α** has six and **γ** has eight. Remove one of those five and the covariance interpolates; remove α or γ and it has to extrapolate outside the range it was built on. The five interpolated hosts average ρ **+0.611** against **+0.473** for the nearest host's own model, a margin of +0.138; the two extrapolated ones give +0.431 (α) and **−0.284** (γ, with q^2 −5.50, a prediction that is worse than useless).

So the claim this route supports is *interpolation between endpoints that were measured*. γ is the warning: it is not merely the largest cavity but the only one of its size, the task kernel has no neighbour to take its shape from, and the mean — linear in the descriptors — is extrapolated past every host it was fitted on. The length scale the evidence chose is itself the symptom: 4 standardised units, which is the *top of the grid* on every one of the seven fits, so the criterion would prefer an even wider task kernel. With seven endpoints there is not enough to localise it, every endpoint is treated as a neighbour of every other, and that is generous towards γ and wrong.

Comparing two models by their co-fields

The selectivity co-field above is the *difference* of two co-fields. Their *inner product* is the other half of the same algebra, and it answers a different question: how far do two models ask the same thing of the chemistry?

```
python cmf_cli.py transfer targetA.cmf targetB.cmf
python cmf_cli.py transfer round1.cmf round2.cmf round3.cmf --fields --out map.csv
```

It needs no grid. With $C_f^A(\mathbf{r}) = \Sigma_j\, a_j\, s_{fj}\, X_{fj}(\mathbf{r})$ and the same for B, the overlap is a sum of overlap integrals of atom-centred Gaussians — which is the field kernel between the two training sets:

$$\langle C_f^A, C_f^B \rangle = \sum_{j,k} a_j b_k s_{fj} s_{fk} K_f^{\mathrm{raw}}(j,k) = \mathbf{a}^{\mathrm{T}} \mathbf{K}_f(A,B)\, \mathbf{b}$$

one matrix product on a block the program computes for a prediction anyway. The number to read is the **cosine**, because it does not move when either activity is rescaled, when the mixing coefficients change, or when the field scaling does: it compares the *shapes* of two demands rather than their sizes. A unit test checks it against the same integral taken on a 175 000-point grid; they agree to four decimals.

What it is for, in the order the question arises. **Two targets**: a positive cosine says both want the same thing in the same place, which is why selectivity is hard, and a negative one says they want opposite things, which is where selectivity comes from. **Two series on one target**: agreement means the SAR transfers, disagreement means two chemotypes are being fitted by two different stories and a model pooled over both will be an average of them. **One series across rounds**: the cosine with the previous round says how much the new compounds changed the model, and the per-field cosines say in which field.

Is it a map of SAR transfer, measured

Every Sutherland set split into its chemotypes, a model on each group of fifteen molecules or more, the attenuation shared across the set, and every ordered pair scored twice: by the co-field cosine, and by how well one model orders the other group's molecules (Spearman's ρ — not R^2, since two chemotypes often sit at different activity levels and an offset is not a failure of the SAR).

	mean cosine	mean transfer ρ
two halves of one chemotype (the control, 16 pairs)	**+0.401**	**+0.673**
two different chemotypes of one target (64 pairs)	+0.178	+0.334

The cosine separates the two cases by 0.22, which is what it is for: two models of one SAR look alike and two models of two SARs do not. And within the chemotype pairs the cosine and the transfer **correlate at ρ +0.667** over the 64 pairs, which is the claim of this feature, measured — a table of co-field overlaps is a map of SAR transfer.

Everything depends on one assumption. Two co-fields are functions on one space only if their molecules were posed in one frame. The program checks what it can from the model files — the same fields, the same attenuations, training boxes that overlap — and refuses with a reason when they fail. What it cannot check is whether two alignments *mean* the same thing, and nothing in the cosine would reveal that they did not. A cosine is also not a prediction: that two models want the same thing does not mean either is right.

Chemical space maps

Tools → Chemical space map... builds a two-dimensional map of the set from the CMF kernels. Fields are not vectors, so Generative Topographic Mapping cannot be applied to them directly; the

program performs kernel PCA on the CMF kernel — which is functional PCA, and reproduces feature-space distances exactly — and runs the ordinary EM fit of GTM in those coordinates. The out-of-sample projection stays a single kernel row, so new molecules can be placed on an existing map.

Two projections, and they answer different questions. The *Projection* box chooses between the GTM landscape and the **kernel PCA score plot**, which draws the functional principal components themselves — by default the first two, with any other pair available. The GTM spreads the set over a square grid, which is what makes a landscape possible, but it is a nonlinear mapping: a tight cluster is pushed apart to fill the grid and no distance on the map can be read as a distance between molecules. The score plot is the linear projection the GTM is fitted on, so its distances are the kernel's own as far as the two eigenvalues reach, and each axis is labelled with the share of the kernel variance its component explains — on the steroid set, 32.9 % and 15.1 %. Ask the map *where the activity is*; ask the score plot *whether this series is one cluster or three*, and how much of the kernel a flat picture of it can genuinely show. The score plot needs no EM fit, so it also costs a fraction of the time.

From the responsibilities follow the **activity landscape** (responsibility-weighted mean activity per node), the **class landscape** (the fraction of actives) and the **data density**, which is a picture of the applicability domain; nodes with no support are left blank. The landscape is a model in its own right: on the thrombin training set it reproduces the activities with R^2 0.92.

Two cautions. Use cosine-normalised kernels, or the leading components follow molecular size. And a CMF kernel between molecules that share no frame is meaningless, so a map spanning several series would partly be showing alignment quality — unless you use the pose-free count kernel of *Working without an alignment*, which separates three targets' ligands cleanly with no alignment at all.

Classification and virtual screening

Setting up a classification task

Choose *Binary classification* as the task. The classes come either from a threshold on the activity (with *active below* for data where small is potent, such as an IC50 in molar units) or from a class property in the file. The learners are C-SVC, ν-SVC and the Laplace Gaussian-process classifier; the *ROC* and *Scores* tabs replace the scatter plot.

```
python cmf_cli.py build examples/sutherland/AChE.sdf --charges file \
       --task classification --threshold 6.84 --method gpc --Cs 1,10,100
```

One-class models: screening with actives alone

Often there are no reliable inactives, only a set of known actives, and the question is whether a new molecule belongs to the same class. That is one-class classification, applied to CMF by Karpov et al. (2011). Five descriptions of the active class are implemented:

Method	What it measures
`ocsvm`	the 1-SVM: a hyperplane in feature space separating the actives from the origin
`ocmean`	distance to the mean embedding of the actives, the centroid of the fields
`ocknn`	distance to the k-th nearest active
`ocgp`	the one-class Gaussian process, optionally scored by mean over standard deviation
`ockpca`	reconstruction error against the principal subspace of the actives (kernel PCA)

On the sets measured here the 1-SVM is the **most fragile** of the five: the centroid, k-NN and kernel-PCA descriptions are as good or better and have one parameter fewer to go wrong. Try more than one; they disagree in interesting places.

Decoys and the figures that matter

A screening model is only as meaningful as the background it is tested against. *File → Add decoys (background set)…* or `--decoys` loads presumed-inactive molecules used **only** for evaluation, never for fitting. `examples/make_decoy_sets.py` builds property-matched decoy sets, which is the fair kind: decoys matched to the actives in molecular weight, logP, rotatable bonds and formal charge, so that the model cannot win by noticing size.

Two problems the program handles explicitly:

- **A decoy must not be rejected for its pose.** `--align-decoys` places every decoy in the frame of the training set by maximising its field similarity, with several random restarts, and `--decoy-conformers N` generates N conformations of each decoy and keeps the one that poses best (the file's own conformation is always among the candidates).

- **AUC saturates.** With cross-target decoys almost any method reaches 0.95 and above, which tells you nothing. Read the **enrichment factors** at 0.5, 1, 5 and 10 % instead, which the program reports beside the maximum achievable value at each fraction, and **BEDROC/RIE** (Truchon & Bayly), which weight the top of the list.

A measured caution from the benchmarks in this repository: field similarity on these tasks performs about as well as ECFP4 fingerprint similarity, and cross-target decoys are trivially separable by either. If a screening result looks spectacular, check what the decoys were.

Measured inactives, and learning from positives alone

Every background in the previous section is *presumed* inactive, and a presumed inactive is an easier to reject than a measured one. A compound that was tested and did nothing is the kind of molecule a screen actually has to reject, and there is a public collection of them: **LIT-PCBA** (Tran-Nguyen, Jacquemard & Rognan 2020), assembled from high-confidence PubChem dose-response assays — 15 targets, 7761 actives and 382674 inactives, every one with an experimental verdict. The AVE-unbiased subsets are the form the authors recommend, since the asymmetric validation embedding of Wallach & Heifets removes the analogue bias that otherwise lets a screen score well by recognising that actives resemble actives.

What LIT-PCBA does not have is the thing a field method needs: a conformation and a pose. `examples/make_litpcba_examples.py` supplies them. For each target it embeds the actives and a seeded sample of the measured inactives with ETKDG and MMFF, then poses **both classes by one rule** — maximum field similarity to the target's crystal ligand, which the archive ships, so the frame is the one the receptor defines and the placement knows nothing about which molecule is active. The 57 MB archive is downloaded on demand and is not redistributed here; what is kept are the derived structures, `examples/litpcba/<target>.sdf` and `<target>_decoys.sdf`.

Why the posing rule is half the measurement

`--pose-against actives` poses the inactives to maximise their similarity to the posed actives instead, which is what `cmfqsar/decoys.py` does for the background sets of the previous section: no decoy is then rejected for having been posed badly. That sounds like the fair choice and is the generous one, for a reason that is not the obvious one. Posing a decoy where it most resembles the actives does make it harder to reject — but the pose search also *fails* on a molecule whose field cannot be made to match, and the similarity it settles at is then a shape-matching score in its own right. Optimising every candidate's pose against the actives silently makes the alignment part of the scoring function, which is what shape-based screening does on purpose and what a field model measured this way gets for free. The same molecules posed both ways say what it is worth, and the figure below says it is most of the result.

Unlabelled is not inactive

The compounds that came back negative in one assay at one concentration are **unlabelled** rather than inactive: some are inert, some were mis-measured, some would have worked at another concentration, and a few are actives nobody has recognised. Training a classifier on that mixture as a negative class teaches it to reject the very molecules a screen is looking for, and the damage is worst at the top of the ranking, which is the only part anyone reads.

`cmfqsar/pu.py` implements **bagging PU** (Mordelet & Vert 2014), the method that needs neither an estimate of the class prior nor an assumption about how the positives were sampled:

1 draw a small random sample of the unlabelled set and call it negative;

2 fit on the positives against that sample;

3 score the unlabelled molecules that were *not* in the sample — out of bag, so a molecule's score never comes from a model that was told it was negative;

4 repeat, and average each molecule's out-of-bag scores.

An unlabelled molecule that looks like the positives is called negative in only a few bags and scores high in the ones that leave it out; one that is genuinely unlike them scores low wherever it appears. Everything works on a precomputed kernel, so the CMF kernel enters unchanged:

```
from cmfqsar.pu import pu_scores
scores = pu_scores(K, positives=train_actives, unlabelled=library, test=held_out, bags=30)
```

What it measures

Measured on two LIT-PCBA targets (ESR1_ant, TP53), 152 measured actives against 2999 measured inactives, 70/30 on the actives and half the inactives offered to the methods that can use them. AUC on the held-out actives against the held-out inactives:

method	ESR1_ant	TP53	what it sees
distance to the centre of the actives	0.536	0.365	the training actives
distance to the k-th active	0.515	0.382	the training actives
one-class Gaussian process	0.542	0.546	the training actives
distance from the kernel-PCA subspace	0.537	0.444	the training actives
ECFP4 Tanimoto to the actives	0.668	0.701	the training actives
classifier, inactives = negative	0.544	0.613	+ half the inactives, as negative
classifier, inactives = unlabelled (bagging PU)	0.662	0.715	+ half the inactives, as unlabelled

Three things in that table are worth saying plainly.

The one-class field descriptions do not work here. The best of them reaches 0.544 against 0.685 for the maximum ECFP4 Tanimoto to the same actives, and some are below chance (distance to the centre of the actives at 0.365 on TP53: the held-out actives look *less* typical of the training actives than the measured inactives do). Nor is that a badly chosen setting. Every parameter that can reorder the ranking was swept — the field attenuation over 0.3, 0.5, 1 and each description's own parameter, 45 fits per set — and keeping whichever scored best *on the test set*, a bound no legitimate protocol could reach, lifts the best description only to 0.574. (ν is not in that sweep, and cannot be: it places the threshold so that a fraction ν of the training actives falls outside the description, which shifts every decision value by the same constant and leaves every ranking metric unchanged.) What the gap reflects is the benchmark's design: the AVE debiasing removes the analogue bias on purpose, and a method whose entire content is "this molecule resembles the actives" then has nothing left to say.

The kernel is not what fails. The same CMF kernel used discriminatively, with the inactives treated as unlabelled, reaches 0.689 — level with the fingerprint, and +0.110 above the same learner told that the inactives are negatives. For screening against measured inactives, use the

kernel discriminatively; the one-class descriptions are for the case the earlier sections describe, a congeneric series against a structurally different background.

The posing is half of any screening number. Posed instead to maximise their similarity to the actives, the same inactives move the one-class AUCs by +0.141 on average while the fingerprint shifts by +0.000, because it never looks at a coordinate. That is the generous placement the background sets of the previous section are given, and the difference is a measurement of how much a field-based screening figure depends on how its decoys were placed.

Screening a library that does not fit in memory

Everything above loads a data set, builds every molecule's fields and keeps them. That is right for a training set and impossible for a library: a hundred thousand molecules will not fit, and nothing about them needs to be kept once their score is written.

Predicting with a kernel method needs only the rows `K(X, X_train)` — one row per new molecule, as wide as the training set. So the `screen` subcommand reads the library in blocks and never holds more than one:

```
python cmf_cli.py screen model.cmf library.sdf --out scores.csv --beta 1.0 --top 25
```

It reads `--chunk` molecules (500 by default) with a forward supplier, so the file is streamed and never indexed; builds their fields with the model's own field set; computes the block's kernel rows; writes the scores to the CSV as it goes, so a run stopped after an hour keeps everything it had scored; and keeps only the best `--top` of them in memory. A SMILES file is accepted too and its molecules are embedded as they are read.

Measured against the same library loaded whole, scored by a 200-molecule model:

library	how it was read	seconds	molecules per second	peak process memory
4166 molecules (22 MB)	streamed, 500 per block	48	86	295 MB
4166 molecules (22 MB)	streamed, 100 per block	59	71	232 MB
4166 molecules (22 MB)	loaded whole	46	90	571 MB
16664 molecules (87 MB)	streamed, 500 per block	201	83	288 MB
16664 molecules (87 MB)	streamed, 100 per block	237	70	237 MB
16664 molecules (87 MB)	loaded whole	178	94	1706 MB

The memory column is what matters. Over a four-fold library the whole-file path grows 571 to 1706 MB, a factor of 3.0, while the streamed path at 100 molecules per block stays at 232 and 237 MB. Extrapolating the first to 100 000 molecules asks for about 10 GB; the second would still be using 237 MB. The price is 33 % of wall clock, from parsing the file in blocks rather than once.

What the score is. For a regression model it is the prediction, or with `--beta B` the lower confidence bound mean $-$ B sd, which is what a library should normally be ranked by, and the reason the standard deviation is always in the CSV: a library is mostly molecules unlike the training set, a kernel method answers the training mean for those, and a point estimate cannot be told from

a real prediction. Screen the thrombin series against a cyclooxygenase model and every molecule comes back at the training mean with a spread four times the residual error — which is the model saying, correctly, that it has nothing to contribute.

What it does not do. It does not invent poses. A field kernel compares molecules in one frame, so the library has to be in the training set's frame already, or `--pose` has to be given, which places each molecule by maximising its field similarity to the training molecules. That is the placement the background sets of the benchmarks are given, it costs about half a second per molecule — far more than the scoring — and it is optimistic, because a real library has to choose a pose without knowing the actives. `--pose-conformers N` generates N conformations of each molecule and poses every one, which multiplies the cost by N.

Which molecule to make next

The model predicts an activity and, through the Gaussian-process view, a standard deviation beside it. Those two numbers answer a question the 2019 paper poses and leaves as a statement — that maximising the overlap tells one how to modify the molecule — as an operation with a number:

```
python cmf_cli.py suggest model.cmf candidates.sdf --acquire ei --top 10
```

The candidates have to be posed in the training set's frame, as for `predict`. What comes back is the ranked list with the prediction, the standard deviation, and the distance to the nearest molecule already made.

The rule depends on what the asking is for, and the program keeps the two apart rather than pretending one serves both.

`--acquire`	**what it maximises**	**what it is for**
`ei`	expected improvement over the best made so far	finding an active
`ucb`	prediction + β standard deviations	the same, more crudely
`greedy`	the prediction	the same, with no allowance for being wrong
`variance`	the posterior standard deviation	improving the model
`maxmin`	distance in field space to everything made	the same, with no model at all
`random`	nothing	the baseline the others have to beat

Expected improvement counts only the upside, which is why it is the usual choice:

$$\mathrm{EI}(x) = (\mu - y^{*} - \xi)\,\Phi(z) + \sigma\,\varphi(z), \qquad z = \frac{\mu - y^{*} - \xi}{\sigma}$$

Note that this is the **opposite** of how the same two numbers are used to rank a screening library, where `rank_score` subtracts the standard deviation instead of adding it. The difference is not a matter of taste. A screening library is dominated by molecules unlike anything in the training set, a kernel method returns the training mean presented as a prediction for exactly those, and a molecule chosen on that basis is wasted. A molecule *made* to answer a question is different: being wrong about it costs little, and being wrong is informative. Optimism is right for one and wrong for

the other.

What it gains, measured

Eight Sutherland sets, ten runs each, starting from twenty molecules drawn at random that hold no hit and acquiring a quarter of what is left one at a time. α and λ are reselected by the evidence at every step from what has been acquired and nothing else — which is what the evidence was built for. A **hit** is a molecule in the top 5 % of its set.

Finding an active:

rule	best found, of the best in the set	runs that found a hit	median acquisitions to the first hit
random	92.2 %	79 %	6
maxmin	90.9 %	64 %	9
variance	90.6 %	71 %	9
greedy	95.0 %	100 %	4
ucb	**96.6 %**	95 %	4
ei	95.2 %	89 %	**3**

Improving the model, as the RMSE on everything not yet acquired (against the variance of the whole set, so that a rule which acquires the extremes is not rewarded for making the rest look alike):

rule	mean RMSE over the run	RMSE at the end	q^2 at the end	against random, paired
random	1.122	1.064	+0.366	—
maxmin	**1.108**	1.047	+0.384	−0.014 [−0.034, +0.006]
variance	**1.108**	1.042	+0.386	−0.014 [−0.034, +0.006]
greedy	1.179	1.195	+0.158	**+0.057 [+0.038, +0.078]**
ucb	1.144	1.127	+0.279	+0.022 [+0.001, +0.044]
ei	1.121	1.086	+0.331	−0.001 [−0.022, +0.021]

The optimistic rules work and the exploring rules do not. For finding actives, `ucb` reaches 96.6 % of the best activity in the set against 92.2 % for picking at random, hits in a median of four acquisitions against six, and the paired gain in hit rate is +0.163 [+0.062, +0.275]; `ei` hits soonest of all, in three. For improving the model, the best exploration rule gains −0.014 RMSE over random with an interval of [−0.034, +0.006] — zero is inside it, so on these sets **exploring gains nothing measurable**, while `greedy` is clearly worse, by +0.057 [+0.038, +0.078].

The two tables are kept apart because an acquisition rule cannot be judged without saying what the acquiring is for, and three things in the numbers are worth more than the ranking.

The uncertainty rule and the diversity rule are the same rule. `variance` and `maxmin` give the same result, which is the error-prediction finding seen from a different direction: the posterior standard deviation of a CMF model is very nearly the distance to the training set in other units. Acquiring by what the model does not know and acquiring by what the series does not cover are the same experiment with two names.

Exploring gains nothing measurable. The molecule the model is least sure of is the molecule least like the series, and what is learned from an outlier does not transfer to the molecules anyone will ask about next. Uncertainty sampling is a good rule for a space one intends to cover and a poor one for a series one intends to understand.

`greedy` **does better here than it would in general.** It ignores the posterior entirely, so a candidate unlike anything made is predicted at the training mean and cannot be told from a real answer. These sets are series of analogues, where nothing is far enough away for that failure to matter; on a library of diverse candidates the protection is gone, and `ei`, which at least tempers its optimism with the posterior, is the rule to use.

Designing the next compound

`suggest` ranks candidates somebody has already drawn. The design loop makes the suggestion itself: take a parent compound of a series, take one of its attachment points, put every fragment of a library there in turn, and rank what comes out.

```
python cmf_cli.py design series.sdf --parent 45 --position 2 --acquire ei --top 15
```

The command does the whole thing: it finds the scaffold, builds a **topomer model** of the series, chooses α and λ by the evidence, reports the q^2 that the suggestions rest on, enumerates, scores and ranks. There is no model file to pass, because the model has to be a topomer model and the core and the attachment points come from the series anyway.

Why a topomer model and not an aligned one. A substituent nobody has made has no crystal structure and no alignment, and superimposing it on anything would be a guess. The topomer rule builds a fragment into a pose that is a deterministic function of its own topology, so a candidate is posed by exactly the rule that posed the molecules the model was fitted on, and the comparison is like with like at one attachment point. A model fitted on superimposed whole molecules has no way to place a group that was never synthesised, which is why the command builds the topomer model itself instead of taking a model file: there is no aligned model that could answer the question.

Three numbers come back for each candidate, and they answer different questions: the **prediction**, the **posterior standard deviation** (a fragment unlike anything in the series gets the training mean and a wide spread, which is what stops the loop from proposing molecules it knows nothing about), and the **acquisition score** that combines them — the same rules as `suggest`, expected improvement by default. Then the annotations: whether the series already uses that group at that position, whether the whole molecule has been made and what it measured — the only free check there is on a suggestion — and, for the top few, what the candidate's fields have that the parent's do not.

The library is `--fragments common` (a built-in list of about forty ordinary medicinal-chemistry substituents), `series` (what this series already puts there), `both`, or a file of SMILES with one attachment point each.

Would it have suggested the right group, measured

Leave one substituent out: every molecule carrying a given R-group at the varied position is removed, the model is refitted on what is left, and the whole library is enumerated and ranked from one fixed parent. The group's measured activity is the mean over the molecules that carry it.

set	groups held out	position	q² of the models	ρ of the ranking with the measurements	the group it puts first	library mean	library best
THER	46 of 46	0	+0.251	**+0.279**	6.53	5.24	10.17
GPB	41 of 47	2	+0.566	**+0.516**	4.40	2.94	6.80

Over 87 held-out groups the ranking agrees with the measurements at a mean Spearman ρ of **+0.397**, and the group the loop puts first is worth **+1.37** log units more than the library average — it lands in the best 20 % of the library **48 %** of the time. The sharper question is where the groups that really are good end up: of the five best held out on each set, the median rank is 7 of 42 on GPB and 12 of 46 on THER, and the single best lands at 13 and at 32. A loop run twice over would have found most of them and missed one.

Only 2 of the eight sets can be reached at all: ACE (the core comes off cleanly in only 44% of the series), AChE (the core comes off cleanly in only 26% of the series), BZR (the core comes off cleanly in only 42% of the series), COX2 (the core comes off cleanly in only 22% of the series), DHFR (the core comes off cleanly in only 12% of the series), THR (the core comes off cleanly in only 25% of the series).

What it cannot do

The loop varies **one position of one parent**, and it states plainly the three things that this limits.

It needs a series with a scaffold that comes off at acyclic single bonds. Of the eight Sutherland sets only two are that; on the rest the Bemis–Murcko framework is fused into the molecules and what comes off is not a substituent. The command says so instead of ranking the pieces anyway.

It ranks groups, not conformations. The topomer pose is a convention, and a suggestion is a statement about which group belongs at a position, not about how it will sit when it is there.

And it cannot tell you about a position the series never varied. A model learns from contrast; a site that is the same in every training molecule carries no information about what else might go there, and the posterior standard deviation is the only thing that will say so.

Conformations: the fourth dimension

A 3D-QSAR model represents each molecule by one conformation, and the answer depends on which one. 4D-QSAR (Hopfinger et al. 1997) replaces the choice by an ensemble and averages over it. *Model → Conformations* switches between the two; **single is the default and is unchanged**, and the tests assert that by comparing its fields against those built before ensembles existed.

In CMF the averaging needs no new mathematics. A field is a sum of atom-centred Gaussians and the kernel is bilinear in the fields, so the mean field of an ensemble is itself a field — its atoms are the atoms of every conformation with their weights multiplied by that conformation's weight — and the kernel of two ensembles comes out as

$$K(A,B) = \sum_k \sum_l w_k^A w_l^B \, K\left(X_A^{(k)}, X_B^{(l)}\right)$$

the mean-map kernel of the two conformational distributions, positive definite by construction. A test checks that identity against the pairwise kernels of the conformations. The model stays linear in that mean field, so **co-fields, the decomposition rules, Shapley values and the complementarity scores all keep working**; contributions arrive per atom per conformation and fold back onto the atoms by summation.

Tools → Generate conformations... builds the ensembles with ETKDG and MMFF94s, keeping the pose a molecule already has as its first conformation, so an earlier alignment survives. A multi-conformer file can be read instead, with `--ensemble-by name` grouping the records of one compound. Weights are uniform or Boltzmann from the stored energies, with kT and a cap on the number kept.

Where the other conformations go is a choice of its own, and not a small one: what matters is not where a conformation sits relative to its own molecule but where it sits relative to the rest of the series.

`--conformer-align`	**Where each generated conformation is put**
`self`	on the molecule's own pose, over all heavy atoms — the default, so nothing changes silently
`core`	on its own pose, over the common substructure with a reference only, so the scaffold stays where the published alignment put it and only the flexible part moves
`reference`	on a reference molecule of the series, through that same common substructure, which puts every conformation of the series in one frame
`field`	as `reference`, then refined by maximising the normalised CMF kernel — the 2013 paper's criterion, applied per conformation

Multi-instance learning (`cmf_cli.py mil`) goes further: instead of averaging, the model chooses which conformation of each molecule to use, alternating between fitting and selecting. Use it when you believe one conformation is the bioactive one and the others are noise; use the ensemble when you believe the molecule is genuinely flexible.

Working without an alignment

This chapter is the one to read if your molecules came from SMILES, if your series has several scaffolds, or if you simply do not trust the superposition you have. The program offers three frame-free routes, and the scripts in `benchmarks/` measure all of them.

Why the frame is the problem

An alignment is an assumption, and a wrong one is expensive. Measured on the nine targets that occur both in the curated Sutherland collection and in the automatically aligned Mittal collection — same compounds, same activities, same fields and grids — the curated superposition gives a mean q^2 of **+0.597** and the automatic one **+0.295**. Worse, the gain over a baseline with no geometry at all (atom-type counts) is +0.25 on the curated files and **−0.06** on the automatic ones: on five of the nine targets, fields in an automatic frame predict *worse* than counts. An alignment that is not trustworthy is not merely imperfect, it is actively harmful.

The pose-free count kernel

As α → 0 an indicator field tends to the count of atoms of that type, so the kernel tends to the dot product of the atom-type count vectors and nothing of the geometry survives. The program implements that limit in closed form (`chemotypes.count_kernel`), which is exact rather than approached, costs one pass over the atoms, and is 10 to 34 times faster than the Gaussian route. It is the **baseline** any aligned model should be made to beat, and it is what the leave-one-chemotype-out workflow reports beside the aligned model.

One subtlety, because it is easy to get wrong: the normalisation must be applied to the joint kernel, not field by field. A field normalised on its own tends to 1 whenever both molecules contain the type at all, so the counts cancel and the baseline loses the very descriptors it is built from.

The rotation-invariant autocorrelation kernel

Geometry without a frame. For each pair of field types the program computes the smoothed field autocorrelation

$$\Phi_{fg}(d) = \sum_{l}\sum_{m} w_{lf} w_{mg} \exp\left(-\frac{(|\mathbf{r}_l - \mathbf{r}_m| - d)^2}{2\sigma^2}\right)$$

as a function of interatomic distance d, which is invariant under rotation and translation by construction.

Where the two molecules are. That equation describes *one* molecule: both sums run over its own atoms, which is what makes it an autocorrelation. It is a descriptor, not a kernel — one number per channel c, a channel being one pair of field types at one distance — and the kernel is the cosine-normalised dot product of two such vectors:

$$K(A,B) = \frac{\sum_c s_c^2 \, \Phi_c(A)\,\Phi_c(B)}{\|s\,\Phi(A)\|\;\|s\,\Phi(B)\|}$$

with s_c the scale channel c inherits from its block: the reciprocal root mean square of that block over the training molecules, times the block's mixing weight. This is the opposite construction to

the overlap kernel of chapter 2, which is defined on two molecules from the start and has no finite feature vector at all — its feature map is the field itself. That reversal is also what decides how each model can be read afterwards: a dual expansion over a feature map that is a function of space is itself a function of space, which is the co-field, while a dual expansion over channels is a vector of channel weights, which is the co-pattern.

The $d = 0$ channel reproduces the CMF self-overlap exactly when $\sigma^2 = 2/\alpha$, so the descriptor contains the ordinary kernel's diagonal information and adds the distance profile. Select *Autocorrelation* as the kernel source in the model panel (`--kernel autocorrelation`, with `--ac-sigma` and `--ac-dmax`), and the model fits, cross-validates, saves and predicts exactly as any other — only the co-field, which needs a frame, is unavailable.

On the chemotype benchmarks the invariant kernel recovers most of the value of the geometry without a frame (0.172 against 0.189 for the aligned model and 0.072 for counts), and on automatically aligned files it frequently beats the aligned model outright — on six of the nine shared targets, including thrombin at +0.452 against +0.021.

Two points, and what that misses. The descriptor counts pairs of atoms at a distance, and a mirror image has every pair at exactly the same distance, so this kernel cannot tell a molecule from its enantiomer. It is blind for the same reason to the orientation of one field with respect to another — whether the donor sits clockwise or anticlockwise from the bulk, going round the ring. The quantity that is not blind is the signed volume of three displacement vectors, and `--ac-chirality` adds it as a second block of channels, the signed triple products of three field types resolved over `--ac-chirality-shells` radial shells (four by default). The block is appended rather than mixed in, so the two-point part is unchanged to the last bit, and it is scaled separately because a volume carries the cube of a length.

Measured, it does not justify its cost. `benchmarks/run_chirality.py` asks the question in the regime the alignment-free kernel exists for — leave-one-chemotype-out on the five chemotype-split sets, with three regularisations by three distance grids chosen inside the training part of every fold, and the number of shells kept out of the search so the signed channels never get more freedom than the kernel they extend:

	two-point	signed, 1 shell	signed, 2 shells	signed, 4 shells
mean q^2 over the five sets	+0.063	+0.082	+0.095	+0.108
paired difference	—	+0.019	+0.033	+0.045
95 % interval over the sets	—	[−0.051, +0.113]	[−0.031, +0.121]	[−0.028, +0.121]
better on	—	2 of 5	3 of 5	3 of 5

The mean moves in the right direction and the interval contains zero at every setting. More to the point, **one set accounts for all of it**: at four shells COX2 gains +0.187 and the other four sets average +0.009 between them, with AChE and BZR made worse. On five sets that is not a result, and the channels cost 16 % more columns.

The experiment those sets cannot do. None of the five contains an enantiomeric pair, so the kernel's blindness costs it nothing there and the table above is about something else. The same script therefore builds a set where handedness *is* the activity: twenty molecules, each with its exact mirror image — one conformation reflected, not a second embedding, so a pair differs in handedness and in nothing else and their two-point channels agree to one part in 10^{15} — with the

activity `(1 − w)·composition + w·handedness`, the handedness read as the CIP descriptor rather than the SMILES tag, which is not the same label from one scaffold to the next.

At `w = 0` the model works: q² +0.97. At `w = 1`, where the answer is the handedness and nothing else, **no variant learns it** — two-point −0.391, and the signed channels −0.34 to −0.37 however many shells they are given and even when scaled up to carry half the kernel.

And the reason is measurable. A kernel method carries what it learnt from one molecule to another along a direction in feature space. The signed channels of a molecule and its own mirror image are exact negatives — that is what the block is for, and a test asserts it — but between *different* molecules they share nothing: the cosine of the signed block is **−0.016 for two molecules of the same handedness and +0.016 for two of opposite handedness**, a difference of 0.03 against a spread of 0.55. Even within one scaffold family, amino acids differing only in the side chain, the difference is 0.10 against a spread of 0.48. Handedness is not a direction in this space, so there is nothing for a linear model to transfer, and the two negative results above are one result seen twice.

That is the conclusion: the block is correct and does what it says about a molecule and its mirror image; it does not put the handedness of two different molecules on a common axis, which is what a QSAR model needs. The channels stay available and off by default.

The construction that does work reads one centre at a time. `cmfqsar/stereo.py` takes every atom with at least three neighbours, orders those neighbours by their weight in a field — the field weights playing the part the CIP rules play for a chemist, so that the descriptor stays a property of the fields — and takes the signed volume of the ordered frame. It negates under reflection and survives any rotation, like the three-point block, and unlike it two *different* molecules of the same configuration agree: the cosine between same-handedness molecules is **+0.468 against −0.468 for opposite**, a difference of 0.935, where the three-point block manages 0.03.

That difference explains everything, and the synthetic set shows what it gains. Where the activity is handedness and nothing else, the stereo centres reach **q² +0.721** while every variant of the three-point block sits at −0.35; at every mixture from a quarter handedness upward they are ahead of the two-point kernel, and where there is no handedness in the activity at all they cost little (+0.924 against +0.970).

On the five real sets they add nothing — mean +0.039 against the two-point kernel's +0.063 — which is the expected answer rather than a disappointment: those sets contain no enantiomeric pair, so the channels carry no signal and 25 extra columns dilute the kernel. **Turn them on for a series whose stereochemistry varies, and leave them off otherwise.** An order that rests on two nearly equal weights is not an order, so a centre whose top three priorities are within `min_separation` of each other contributes nothing rather than a guessed sign; that guard is what takes the same-handedness separation from 0.18 to 0.86.

Mixing coefficients over the channel blocks. The channels come in blocks, one per pair of field types, each scaled to unit root mean square before all of them enter one cosine-normalised dot product. The overlap kernel weights its fields with h_f; the three invariant sources (autocorrelation, surface, periodic) weight their blocks with `--block-weights` (library: `ac_block_weights` on `CMFModel`), one coefficient per block applied to the scaled features before the normalisation, so that a block's share of the kernel goes as the square of its weight and a weight of zero removes the block exactly. Three ways to set them: `balanced` gives every *kind* of block — the pairs within a group of fields, the pairs across two groups — the same share of the kernel however many blocks each kind has; `KIND=W,...` names the kinds' weights (`indicator=0.3,indicator*physchem=0.5`; unnamed kinds weigh 1); `align` or `alignf` derive one weight per block by centred kernel alignment of the per-block kernels

(`align_block_weights`, the `align_mixing` of the fields carried over, with `--block-rho` the shrinkage of the joint solution), re-derived from the training part of every cross-validation fold, so the reported q^2 never sees the left-out activities. The groups default to the continuous indicator fields (`indicator`) against the rest (`physchem`); `--block-groups 'GROUP=field,field;GROUP=cif:*'` names others. The fitted weights are printed by kind and stored with the model (`block_weight_report`).

Molecules on a surface: the in-plane invariant kernel

A molecule adsorbed on a surface has one direction that is not like the others. Its height above the metal and its tilt are physics — they are what the relaxation computed, and they are most of what decides the adsorption energy. Where it sits in the plane and how it is turned about the normal are not: the slab is periodic and its origin is wherever the cell was cut. The overlap kernel cannot tell the two apart, so it demands a frame for a degree of freedom that has no preferred value, and the invariant kernel of the last section throws the height away along with the rest.

This kernel keeps exactly what is physical. It is the two-point distribution of the field weights over the **in-plane** distance ρ, resolved by the heights of both atoms:

$$\Phi_{fg}(\rho_k, z_i, z_j) = \sum_l \sum_m w_{lf} w_{mg}\, G_\sigma(\rho_{lm} - \rho_k)\, G_{\sigma_z}(z_l - z_i)\, G_{\sigma_z}(z_m - z_j)$$

$$G_s(u) = \exp\!\left(-\frac{u^2}{2s^2}\right), \qquad \rho_{lm} = |\mathbf{r}_l^{xy} - \mathbf{r}_m^{xy}|$$

It is therefore invariant to translation in the plane and to rotation about the normal, and to nothing else: tilt, height and the vertical order of the atoms all survive. Averaged over the direction of ρ it is, by the Wiener–Khinchin theorem, the in-plane power spectrum of the fields resolved by height — the molecule described in reciprocal space across the surface and in real space along the normal. The surface is at $z = 0$ with the normal along $+z$, which is how the adsorption scripts of `examples/corrosion_fe110` write their poses; nothing checks that convention, so a set of poses in some other frame will be described accurately but uselessly.

The channels run over in-plane distances to 12 Å in steps of 0.75 Å (`--ac-dmax`, `--ac-sigma`) and heights from 1 to 6 Å in steps of 1 Å (`--surface-z ZMIN ZMAX ZSTEP`), the spacing of a channel being also its width on both axes. That is 17 × 6 × 6 = 612 channels for each pair of field types, 9,180 features on the five physico-chemical fields, which is why the blocks are scaled and may be weighted exactly as in the preceding section. Pairs of different field types are summed both ways round, as the three-dimensional descriptor counts them, and the $l = m$ terms are kept. Those channels are again a descriptor of one molecule, and the kernel of two of them is the dot product of the preceding section, over this section's channels:

$$K(A,B) = \frac{\sum_c s_c^2\, \Phi_c(A)\, \Phi_c(B)}{\|s\,\Phi(A)\|\ \|s\,\Phi(B)\|}, \qquad c = (f, g, \rho_k, z_i, z_j)$$

The command is `--kernel surface`; the model panel does not offer it.

What it is worth

On the benzimidazole corrosion node — 72 molecules relaxed on Fe(110) with the UMA machine-learned potential, the target their adsorption energy, five physico-chemical fields, KRR with α and λ chosen in a 10-fold inner cross-validation, 5 outer folds, 10 repeats:

model	q^2	RMSE (kJ/mol)
surface kernel, poses as the relaxation wrote them	**0.655**	31.2
surface kernel, the same poses shuffled in the plane	**0.655**	31.2
overlap fields, poses as written	0.505	37.4
overlap fields, poses registered in the plane by their common substructure	0.517	36.9
overlap fields, poses shuffled in the plane	0.480	38.3
rotation-invariant autocorrelation, which discards the height	0.109	50.2

Three things follow. The **invariance is demonstrated rather than asserted**: turning every pose by a random angle about the normal and sliding it up to 6 Å in the plane changes the model by nothing at all, the second row reproducing the first to three decimals. **The height is what produces the score** — the fully invariant kernel differs from this one only in discarding z, and it falls from 0.655 to 0.109. And **the frame the overlap kernel wants is not worth building**: registering the poses laterally on the common substructure moves it from 0.505 to 0.517, a negligible difference, while the kernel that was never given a frame is 0.14 above the better of them.

There is no co-field, for the reason every invariant source has none: the kernel is not an integral of a product of fields, so the prediction cannot be turned back into a function of space. What it offers instead is the co-pattern — the weight of every channel of the fitted linear model, which names the pair of field types, the in-plane distance and the two heights that carry the signal, which `top_channels` ranks against the labels of `surface_channel_labels`. The measurements above come from `surface_kernel_eads.py`, in `examples/corrosion_fe110`, which writes its table next to the script.

Crystals: the periodic autocorrelation kernel

The same descriptor for a periodic structure, summed over the neighbour list of one cell with its periodic images out to a cutoff r_{max}, per atom of the cell, with the uniform background of each field pair removed and the cutoff tapered:

$$\Phi_{fg}(d) = \frac{1}{N}\left[P_{fg}(d) - \frac{S_f S_g}{V} B(d) \right]$$

The first term is the tapered pair histogram of the two fields, over every atom l of the cell and every neighbour m inside the cutoff, periodic images included:

$$P_{fg}(d) = \sum_{l \in \text{cell}} \sum_{m\,:\, r_{lm} < r_{\max}} w_{lf} w_{mg}\, \omega(r_{lm}) \exp\left(-\frac{(r_{lm} - d)^2}{2\sigma^2} \right)$$

The second is what that histogram would be if the same total weight of each field were spread uniformly through the cell, which is the part of P_{fg} that reports only how dense the crystal is:

$$B(d) = \int_0^{r_{\max}} 4\pi r^2\, \omega(r) \exp\!\left(-\frac{(r-d)^2}{2\sigma^2}\right) \mathrm{d}r$$

Here N is the number of atoms in the cell, r_{lm} the distance from atom l to image m, $S_f = \Sigma_l\, w_{lf}$ the total weight of field f over the cell, V its volume, σ the width of the distance smearing (`--sigma`) and ω a half-cosine taper over the last `--taper` ångström. Both sums run over ordered pairs, so a pair of two different field types is counted both ways round, as the molecular descriptor counts it, and its background with it; the $l = m$ terms are kept, which makes the $d = 0$ channel the composition of the cell. As in the two preceding sections Φ describes one structure, and the kernel of two of them is the same cosine-normalised dot product over the channels:

$$K(A,B) = \frac{\sum_c s_c^2\, \Phi_c(A)\, \Phi_c(B)}{\|s\,\Phi(A)\|\ \|s\,\Phi(B)\|}, \qquad c = (f, g, d_k)$$

It is invariant to the choice of cell, origin and orientation, so a supercell is the same material. Averaged over orientations it is the sine transform of the product of the typed structure factors, which makes the pair products of the element indicator fields the partial structure factors a diffraction experiment measures.

The fields of a solid are built by `inorganic_fieldset`: electrostatic (oxidation states guessed from the composition, or the `oxi_states/charges` array of the file), steric (atomic volume), electronegativity, one indicator field per element and one for the rest, and any per-atom array as `prop:<name>`. The command is `build-crystals` (an ASE-readable structure file, `--target KEY` or `--target-file CSV`, `--cutoff`, `--sigma`, `--taper`, `--no-background`, `--elements`, `--charges`, `--external-cv`, `--save`); in the library, `kernel_source="periodic"` on a `CMFModel` with a table from `crystal_table`. There is no co-field, but there is a co-pattern: the weight of every channel in the fitted linear model, which `top_channels` ranks by weight times spread, with the sign that says whether the pair at that distance raises or lowers the property.

On the Matbench `phonons` task (1,265 crystals, the top of the phonon density of states) the linear kernel on four fields -- electrostatic, steric, electronegativity and the inverse square root of the mass -- reaches a mean absolute error of 117 cm^{-1} on the Matbench folds, 106 with a Gaussian layer, against 324 for the dummy predictor and 29 for the best graph network; keeping the uniform background improves the 58-field model from 181 to 119 on this task, so the switch is worth trying on a property that depends on composition and density (the four-field model kept it too reaches 92). The element indicator fields dilute the joint kernel — seventeen hundred equally scaled element-pair blocks against the ten physico-chemical ones — and `--block-weights` is the remedy: with the background kept, the aligned weights take the 59-field model from 119 to 97; they do not make it better than the four physico-chemical fields alone. The full table and the co-patterns are in `benchmarks/results/matbench_phonons/table.md`.

Topomer fields

Cramer's topomer idea (topomer CoMFA, 2003), applied to fields. Cut the molecule at the bonds that attach its substituents to a common core and rebuild each substituent in a pose that is a **deterministic function of its topology**: canonical atom numbering, one embedding with a fixed seed, every rotatable torsion set to 180°, the attachment atom at the origin with its open valence along −x, and the remaining roll fixed by the principal axis of the heavy atoms. The same R-group is then always placed in the same position, whatever molecule it came from — a test asserts the coordinates are identical, not merely close.

Two things make this fit CMF better than it fits CoMFA. There is no grid, so a fragment's field is just its atoms and a fragment–fragment kernel is an analytic number, which means the fragment library can be tabulated once and reused. And each attachment point keeps **its own region of space** (the substituent of position k is placed at $x = 100k$ Å, far enough that no Gaussian reaches the next site), so a molecule's kernel is exactly the sum of its per-position overlaps — which is what topomer CoMFA obtains by giving every position its own block of columns. A test checks that identity to 1e-9.

In the application it is *Tools → Replace molecules by their topomer substituents...*, and it is a **transformation of the data set** rather than a hidden switch: the molecules are replaced by their posed substituents, names, activities and the split are kept, and the data-set line says *[topomer substituents of ...]*. That is the correct form for it, because a topomer model's co-field and contributions live in the substituent frames and not around the original ligands. Reload the file to get the molecules back.

Choosing the core matters more than anything else, which is why the program now chooses it differently. The maximum common substructure of a chemotype mixture is not a scaffold — on GPB it comes out as `CC0`, fused into every ring system, and cuts nothing off at all, losing every molecule. The default is therefore the **Bemis–Murcko framework of the largest chemotype**, the scaffold of one series, with every molecule that framework comes cleanly off forming the series; on GPB that is `C1CC0CC1` and 60 of the 66 molecules. The common substructure is still available for a set you know is congeneric (*maximum common substructure of the series* in the dialog, `--core-mode mcs`), and you can always give the core yourself as SMILES or SMARTS. Check the cuttable fraction the program reports either way: a core fused to the rest of the molecule cannot be removed without opening a ring, and what is left is not a substituent.

What the three routes are worth, measured

Every column below uses the same molecules, fields, α and λ grids and folds.

Collection	Series	Aligned q²	Topomer q²	The cut costs	The pose costs
Sutherland, curated frame	8	**+0.445**	+0.343	+0.035	+0.066
Mittal, automatic frame	31	+0.200	**+0.261**	-0.027	-0.033
Corrosion inhibitors, neutral	17	+0.248	**+0.307**	+0.024	-0.083
Corrosion inhibitors, 1 M HCl	17	+0.303	**+0.358**	+0.037	-0.092

The last two columns separate the two things the topomer construction does at once, by way of a control that keeps the same cut substituents at the same separated sites but leaves each one exactly where the alignment put it. **What the pose rule costs depends entirely on the alignment it replaces.** Against the curated Sutherland frame it costs 0.066 q²; against an automatic superposition, or against the frames of the corrosion series, it *gains* 0.033 to 0.092, because a canonical pose is more consistent than an arbitrary embedded conformation. The **cut** costs 0.02 to 0.04 on average and is the expensive half only where the scaffold carries information — separate sites discard the scaffold's own field and the geometry between one substituent and another, which matters when the arms' mutual placement is the pharmacophore (0.21 on thrombin, 0.14 on thermolysin) and costs nothing or even helps when the scaffold is constant and the decorations small (BZR gains 0.102 q², DHFR 0.054).

Practical guidance. With a trustworthy curated alignment, keep it: the geometry adds about 0.25 to q^2 over counts. With an automatic alignment or none, prefer the frame-free routes — topomer for a congeneric series with a clear scaffold whose R-groups come off at acyclic single bonds, and the invariant kernel for anything pooled, multi-scaffold or ring-fused. Always report the count baseline beside the model, so that the value of the geometry is visible rather than assumed.

Heterogeneous series, step by step

When a set holds several chemotypes with no one common substructure, the *Tools* → *Heterogeneous sets* submenu is a four-step workflow:

1. **Find the chemotypes** — Bemis–Murcko frameworks, optionally generic, with small groups merged so that leave-one-chemotype-out does not degenerate into leave-one-molecule-out.
2. **Align within each chemotype** — a superposition that is meaningful, because the members of one cluster do share a substructure.
3. **Place the chemotypes in one frame** — each cluster positioned by maximising field similarity to the others, which is the best that can be done when no common substructure exists.
4. **Leave one chemotype out, with the alignment-free baseline** — the reliable validation, reported beside the count kernel. Random folds overstate q^2 on such sets by 0.11 to 0.47 on the five chemotype benchmarks here, which is why this step exists.

Protein pockets

Getting a pocket on screen

File → Fetch protein from the PDB... downloads an entry from RCSB and caches it under `~/.cmfqsar/pdb`, with the dialog pre-filled with the identifier the data set names. *File → Load protein structure...* reads a local PDB or mmCIF file instead. The reader is deliberately independent of RDKit, because crystal structures carry alternate locations, missing elements and non-standard residues that a chemistry toolkit would refuse to sanitise, and none of that matters for drawing.

The free-energy benchmark sets in `examples/openff` and `examples/schindler` give their ligands in the frame of a reference crystal structure and name it per record, so **nothing has to be superimposed**: opening such a set brings its pocket with it from `examples/pockets/`, which holds the residues within 8 Å of any ligand, taken whole, without waters and without the co-crystallised ligand. The *protein site* controls above the 3D view then draw the residues within a chosen distance as wire frame or sticks, with hydrogens, waters and residue labels optional.

Complementarity as numbers

A pocket is a set of atoms, so the same parameterisation that builds a ligand's fields builds the protein's, and the overlap of two Gaussian fields has the same closed form on either side. The *Complementarity* tab (CLI: `cmf_cli.py complement`) turns that into four answers:

- **How complementary a molecule is to the pocket** — overlap integrals between ligand and pocket fields, exact and without a grid, each divided by the two norms so that it is a cosine in $[-1, 1]$ and does not grow with molecular size. Complementarity is not similarity: the electrostatic term carries a minus sign, because a positive ligand region should meet a negative protein region; the hydrophobic term carries a plus; the hydrogen-bond terms pair the donors of one side with the acceptors of the other. Steric overlap is reported separately as *burial*, because bulk against the protein is contact and clash at once, with the number of clashing heavy-atom pairs beside it.
- **Where on the molecule it holds** — the same quantities on a smooth van der Waals surface give a value per vertex, drawn as a green-to-red surface, with a correlation per field over the surface. Electrostatics uses potentials rather than Gaussian densities here, because the potential is what the other side feels.
- **Whether the co-field agrees with the pocket** — the model's own map against the protein, by sign agreement, cosine or correlation. This is the strongest external check a CMF model can be given: the co-field was fitted to activities and never saw the protein.
- **Which residues say so** — the same scores, resolved by receptor residue.

Which residues say so

An overlap integral is a double sum over atom pairs, so keeping the per-atom column sums instead of the grand total costs nothing and is exact: every term of the score above splits over the residues of the pocket and adds back to it. One cosine becomes an **interaction profile**, a table of residues against interaction types.

This is worth having because a global score cannot say which contact a pose has lost, and neither can the ligand RMSD — a pose can move very little and still give up the one interaction that

mattered. Across a series the profiles also average into a **consensus**: an interaction seen in most of the complexes is a property of the target, while one seen in a single complex is more likely a property of that ligand, which is the distinction a pharmacophore query needs.

On the ten CDK2 ligands of `examples/openff/cdk2.sdf` against their pocket, every molecule reaches 18 of the 53 residues, and the contacts conserved across the whole series are Glu81 donating to an acceptor, Leu83 accepting from a donor, and Ile10 and Leu134 by burial — the hinge and the hydrophobic floor, recovered without being told that either exists.

In the application, tick **by residue** beside *surface scores* before pressing *Compute*. The table under the Complementarity tab then lists every residue within 4.5 Å of the selected molecule, one column per interaction type, sortable by any of them, with the share of the series that shows each residue's interactions in the last column; the header says how much of the whole score those residues carry, the rest being the tail of the Gaussians over the residues behind them. Tick **contributions** beside *residue labels* in the protein-site controls and the 3D view writes each residue's total on its alpha carbon, green where it helps and red where it does not.

From the command line:

```
python cmf_cli.py complement data.sdf --structure pocket.pdb --residues --profile-out profiles.csv
```

`--residues` prints the strongest contacts of each molecule and the conserved pattern of the series, and scores every molecule against that consensus by cosine and by Tanimoto over the thresholded bits; `--reference NAME` compares against one molecule's profile instead of the consensus, `--residue-threshold` sets what counts as present, and `--profile-out` writes one row per molecule and residue.

Docking

Tools → Dock into the pocket... places the conformations of a molecule in the site by rigid-body optimisation. It is a convenience for getting a pose into the right frame, not a scoring function; the program's purpose is what happens after the poses exist.

Command-line reference

Everything the application does is available without it, which is what makes the program scriptable for a benchmark or a cluster run. The general form is

```
python cmf_cli.py <subcommand> <dataset> [options]
```

and `python cmf_cli.py <subcommand> --help` prints the full option list; what follows is an overview, not every option.

Subcommand	What it does
`build`	build, cross-validate, test and save a model
`build-crystals`	the same on periodic structures (CIF, POSCAR, extended XYZ)
`diagnose`	ask a data set what it can support, before modelling it
`predict`	predict with a saved model, optionally aligning by confidence first
`screen`	score a library against a saved model, reading it in blocks, so its size does not matter
`report`	one self-contained HTML page describing a saved model
`suggest`	which posed candidate to make next (expected improvement and the rest)
`design`	what to put at a position of a parent, ranked by the model
`transfer`	compare models by their co-fields: do they ask the same thing?
`mixing`	the posterior over the field weights: which fields the series needs
`hetero`	the heterogeneous-series workflow: cluster, align, place, leave-one-chemotype-out
`topomer`	model a series by its substituents, posed by rule
`map`	chemical-space map and activity landscape (kernel PCA + GTM)
`explain`	Shapley decomposition of one prediction, for any kernel
`mil`	multi-instance learning over conformations
`pharmacophore`	read a pharmacophore out of a fitted model's co-fields
`difference`	what one molecule's fields have that another's do not
`complement`	field complementarity with a protein pocket
`compare`	are two sets the same population? (MMD, with a witness)
`cluster`	cluster a set by its field kernel (spectral clustering)
`unalign`	give every molecule a random pose, the control experiment

The options of `build`, by group

Data. `--activity PROP`, `--activity-file TABLE`, `--test-file FILE`, `--test-fraction F`, `--ensemble-by KEY`.

Fields. `--fields electrostatic,steric,hydrophobic,hba,hbd[,prop:name]`, `--cif`, `--cif-scheme sybyl|element|element+aromatic`, `--cif-min N`, `--cif-types LIST`, `--charges gasteiger|mmff|file`, `--param rdkit|tripos|tripos-legacy`, `--prop-missing error|zero`.

Kernel. `--kernel fields|autocorrelation|surface`, `--ac-sigma S`, `--ac-dmax D`, `--surface-z ZMIN ZMAX ZSTEP`, `--ac-chirality`, `--ac-chirality-shells S`, `--block-weights W`, `--block-groups G`, `--block-rho R`, `--kernel-layer linear|gauss|arccos1|arccos2`, `--gammas LIST`, `--normalize`, `--raw-scale`, `--optimize-h`, `--mkl [align|alignf]`, `--mkl-rho R`.

Task and learner. `--task regression|classification|oneclass`, `--method krr|gp|svr|nusvr|svc|nusvc|gpc|ocsvm|ocmean|ocknn|ocgp|ockpca`, `--threshold T`, `--active-below`, `--class-property PROP`, `--oc-k K`, `--oc-components Q`, `--oc-lam L`, `--oc-score mean|mean_over_sd`, `--no-bias`.

Hyper-parameters. `--alphas LIST`, `--lams LIST`, `--Cs LIST`, `--epsilons LIST`, `--nus LIST`, `--lam-search grid|brent`, `--alpha-search grid|nelder-mead`, `--nm-start A,L`.

Conformations. `--conformations single|ensemble`, `--conformers N`, `--conformer-window A`, `--prune-rmsd A`, `--conformer-align self|core|reference|field`, `--conformer-align-ref I`, `--conformer-weighting uniform|boltzmann|property`, `--kt KCAL`, `--max-conformers N`.

Validation. `--folds K`, `--cv-no-shuffle`, `--nested`, `--nested-on train|all`, `--outer-folds K`, `--repeats R`, `--seed S`.

Screening. `--decoys FILE`, `--align-decoys`, `--decoy-restarts N`, `--decoy-conformers N`.

Alignment. `--align-train-variance [MAX_RMSD]`, `--align-train-rounds N`, `--align-train-folds K`.

Output. `--save model.cmf`, `--predictions out.csv`, `--json out.json`.

Worked examples

```
# indicator fields, a test split and nested cross-validation
python cmf_cli.py build examples/benzamides_synthetic.sdf --activity pActivity --cif \
       --alphas 0.3,0.5,0.8,1.2 --lams 1e-3,1e-2,0.1,1 --test-fraction 0.25 \
       --nested --save model.cmf

# predict, with the new molecules aligned by prediction confidence first
python cmf_cli.py predict model.cmf new.sdf --align-variance 2.0 \
       --aligned-out aligned.sdf --out predictions.csv

# the published protocol of the 2013 paper on a Sutherland set
python cmf_cli.py build examples/sutherland/THR.sdf --charges file --param tripos-legacy \
       --raw-scale --cv-no-shuffle --lam-search brent --nested --nested-on all

# a frame-free model: the rotation-invariant autocorrelation kernel
python cmf_cli.py build series.sdf --kernel autocorrelation --ac-sigma 0.75 --ac-dmax 12

# a topomer model with the core given explicitly
python cmf_cli.py topomer examples/benzamides_synthetic.sdf --activity pActivity \
       --core "c1ccccc1C(=O)N" --alphas 0.3,0.5,1 --lams 1e-2,0.1,1

# one-class screening against a property-matched background
python cmf_cli.py build actives.sdf --task oneclass --method ockpca \
       --decoys background.sdf --align-decoys --decoy-conformers 5

# a classification model with the Gaussian-process classifier
python cmf_cli.py build examples/sutherland/AChE.sdf --charges file --task classification \
       --threshold 6.84 --method gpc --Cs 1,10,100
```

Using it as a library

The library is the layer the application and the command-line tool are built on, and it is small enough to use directly in a notebook.

```
from cmfqsar import FieldSet, CMFModel, load_dataset
from cmfqsar.fields import AtomTable, build_fields, cif_types_in
from cmfqsar.model import grid_search

ds = load_dataset("aligned.sdf", activity_prop="pIC50")
fs = FieldSet.create(cif_types=cif_types_in(ds.mols, "sybyl", 5))
mf = [build_fields(m, fs, n) for m, n in zip(ds.mols, ds.names)]
train = AtomTable.from_molecules(mf)
model = CMFModel(fieldset=fs, method="krr")
grid_search(train, ds.activity, model, alphas=[0.3, 0.5, 1.0], lams=[1e-3, 1e-2, 0.1, 1])
model.fit(train, ds.activity, cv_folds=10, reuse_kernels=True)
print(model.stats.summary())
```

Decomposing one prediction, which works for the nonlinear kernels as well:

```
from cmfqsar.shapley import atom_shapley, field_shapley, scaffold_decomposition

res = atom_shapley(model, mf[0])                       # one value per atom
res = field_shapley(model, mf[0], interactions=True)   # one value per field type
res = scaffold_decomposition(model, mf[0], ds.mols[0], ds.mols[1:20])   # per substituent
print(res.prediction, res.baseline, res.table(), res.top_interactions(5))
```

The modules, one line each:

```
cmfqsar/fields.py       field parameterisation; FieldSet / MoleculeFields / AtomTable
cmfqsar/kernels.py      analytic field kernels, vectorised
cmfqsar/model.py        KRR / SVR / GP, cross-validation, grid search, nested CV, statistics
cmfqsar/cofields.py     co-fields and molecular fields on grids, isosurfaces, cube export
cmfqsar/dataio.py       reading SDF / MOL2 / SMILES, model persistence, exports
cmfqsar/align.py        3D embedding, MCS alignment, field-similarity refinement, unalign
cmfqsar/varalign.py     alignment by prediction confidence (GP posterior variance)
cmfqsar/invariant.py    rotation-invariant field autocorrelation kernels
cmfqsar/topomer.py      topomer fields: substituents posed by rule
cmfqsar/chemotypes.py   scaffold clusters, placement, grouped folds, the count kernel
cmfqsar/pharmacophore.py  typed features with tolerance spheres, from the co-field
cmfqsar/fielddiff.py    field differences against a reference molecule
cmfqsar/shapley.py      Shapley decomposition over atoms, substituents, field types
cmfqsar/gpc.py          Laplace Gaussian-process classification
cmfqsar/oneclass.py     centroid, k-NN, GP and kernel-PCA one-class descriptions
cmfqsar/conformers.py   conformational ensembles: generation, weighting, grouping
cmfqsar/mil.py          multi-instance learning over conformations
cmfqsar/decoys.py       posing a background set for screening evaluation
cmfqsar/gtm.py          chemical-space maps and activity landscapes
cmfqsar/protein.py      PDB / mmCIF reading, binding-site selection, frame check
cmfqsar/complementarity.py  pocket fields, complementarity scores, co-field agreement
cmfqsar/docking.py      rigid-body placement of conformations in the pocket
cmfqsar/gui/            the PySide6 application
```

The example data sets

Set	Contents
`examples/benzamides_synthetic.sdf`	109 aligned N-phenylbenzamides with a *synthetic* activity: the overlap of their fields with a hidden pseudo-receptor co-field plus noise (σ = 0.25). A model has to recover those zones; $q^2 \approx 0.87$ against a noise ceiling of 0.94. The set to learn the program on, because the right answer is known.
`examples/steroids_cbg.sdf`	the classic CoMFA benchmark: 31 steroids (Cramer, Patterson & Bunce 1988) with corticosteroid-binding-globulin affinities, in the corrected Gasteiger-group geometries, re-superimposed on the steroid skeleton of testosterone. Customary split 1–21 / 22–31, leave-one-out $q^2 \approx 0.79$.
`examples/sutherland/*.sdf`	the eight literature sets of the 2013 paper (Sutherland, O'Brien & Weaver 2004): ACE, AChE, BZR, COX-2, DHFR, GPB, THER, THR, with the aligned MMFF94s conformations as distributed, the published split, and the original Sybyl types and MNDO ESP charges as per-atom properties. Choose *Partial charges: file*.
`examples/mittal/*.sdf`	38 of the 40 benchmark sets of Mittal, McKinnon & Sorich (2009), built here from their 2D supplementary data by ETKDG and an MCS superposition, with each molecule's fit quality recorded in `align_rmsd`. The collection to use when you want to know how a method behaves on an *automatic* alignment.
`examples/openff/*.sdf`	21 congeneric series, 513 ligands, from the Open Force Field protein–ligand benchmark, each posed in its target's crystal structure; activity tag `pAffinity`.
`examples/pockets/*.pdb`	the binding sites of those series, trimmed and already in the ligands' frame, so they open together with the data.
`examples/thrombin_atom_properties.sdf`	a worked example of `prop:<name>` fields: per-atom quantum properties carried in an SDF.

`docs/benchmark.md` compares the program with Table 2 of the 2013 paper on the eight literature sets. `benchmarks/run_sutherland.py` reruns that comparison, `benchmarks/run_topomer.py` measures the alignment against the topomer rule (`--mittal` for the automatically aligned collection), `benchmarks/run_alignment.py` compares the value of a curated alignment with that of an automatic one, and `benchmarks/run_corrosion.py` does the same for corrosion-inhibitor series.

When something goes wrong

The 3D view is black, or the program will not start. The viewer needs working OpenGL through PyOpenGL and pyqtgraph. On a remote desktop or in a virtual machine, software rendering is usually the fix (on Windows, a Mesa `opengl32.dll` beside the program; on Linux, `LIBGL_ALWAYS_SOFTWARE=1`). Everything except the 3D view works regardless, and the command-line tool needs no graphics at all.

q² is high but the test predictions are poor. Three usual causes, in order of frequency: the hyper-parameters were chosen on the same cross-validation you are quoting (run *External CV*); the set holds several chemotypes and random folds are overstating it (use leave-one-chemotype-out); or the alignment carries information that will not transfer (compare against the count baseline).

q² is negative. The model predicts worse than the mean: check that the activity column is the right one and on a logarithmic scale, and that the molecules really share a frame. For R^2_{pred} on a narrow test set it may be an artefact of the variance convention, and the program reports both.

The optimum sits at the edge of the α or λ grid. Extend the grid. The *Hyper-parameter search* tab exists to make this visible.

A per-atom property is rejected. The hydrogen count almost certainly changed when the file was loaded. Write the property for all atoms including hydrogens, or pass `--prop-missing zero` deliberately.

The topomer transformation drops most of the molecules. The core is fused to the rest of them. Look at the cuttable fraction in the log and give a core joined to its R-groups by acyclic single bonds.

A decoy set gives suspiciously good enrichment. Check that the decoys are property-matched and posed in the training frame; cross-target decoys are separable by almost anything.

Co-fields are unavailable. A nonlinear kernel layer or the autocorrelation kernel source is selected, and neither is linear in the field, so the co-field does not exist. Use the *Shapley* tab instead.

It is slow. Kernel cost grows with the square of the total atom count, so the expensive sets are those with many large molecules. Cut the α grid first, since each α needs its own kernel, and remember that the count kernel is 10 to 34 times faster than the Gaussian route for a baseline.

Appendix A — Field parameterisation

Two parameterisations are available (*Increments* in the model panel, `--param`):

Field	`rdkit`, the default	`tripos`, the original CMF scripts
electrostatic	partial charge: Gasteiger, MMFF94, or read from the file	the same
steric	Tripos vdW well depth and width α/R^2_vdW, parameters by element	parameters by Sybyl atom type, Tripos 5.2 table
hydrophobic	Wildman–Crippen atomic logP contribution	in-house one-atom fragment increments by Sybyl type and attached hydrogens
H-bond acceptor / donor	approximate Abraham-type increments, donor weight on the polar hydrogen	in-house increments from Abraham's A and B constants, donor weight on the heavy atom
indicator field	1 for atoms of the given type	the same
`prop:<name>`	the value of a per-atom property carried by the file	the same

`tripos-legacy` reproduces the hydrogen-counting bug of the original R scripts, which is needed to match the published numbers exactly and should not be used for new work.

Appendix B — Files and tags the program understands

Tag or convention	Meaning
any numeric SD property	candidate activity; the loader guesses and the dialog confirms
`subset`, `set`, `split`	the train/test split, as `train`/`test` or 1/2
`atom.dprop.<name>`	a per-atom double property, written and read by RDKit
`atom.prop.sybyl_type`	per-atom Sybyl types, used automatically for indicator fields
a molecule property with one number per atom	the same per-atom property in quantum-chemistry dump form
`.cmf`	a saved model (joblib): field set, kernels, coefficients, training molecules, settings
Gaussian cube	the export format for co-fields, molecular fields, Shapley fields and difference maps
`REMARK 400` in a pocket PDB	the provenance of a trimmed binding site

Appendix C — Glossary

The terms this manual uses in a particular sense, with the chapter that treats each one.

term	what it means here
alignment, or **frame**	the common coordinate system the molecules are placed in before their fields are compared. The overlap kernel needs one and is only as good as it is; the count, autocorrelation, surface and periodic kernels do not. *Preparing the data*; *Working without an alignment*
applicability domain	how close a molecule outside the training set is to it, measured in the kernel the model itself uses, with a threshold taken from the training set's own spread. Reported per molecule as the **AD** column and carried into the exported predictions. *Where does the model apply?*
attenuation, α	the width parameter of a field's Gaussians: α large makes a field that falls away within an atom's own volume, α small one that reaches across the molecule. The single most consequential setting after the alignment. *1. The fields*; *Attenuation, regularisation and the grids*
autocorrelation kernel	geometry without a frame: the smoothed distribution of field-weight products over interatomic distance, invariant to rotation and translation by construction. *The rotation-invariant autocorrelation kernel*
BEDROC, RIE	screening figures that weight the top of a ranked list, where AUC saturates. *Decoys and the figures that matter*
Bemis–Murcko framework	the ring systems of a molecule together with the bonds that join them. `cmf diagnose` measures the common framework to ask whether one frame could exist at all. *Before you model*
block weights	one coefficient per block of channels — a block being one pair of field types — applied before the invariant kernels are normalised, so a weight of zero removes a block exactly. *Mixing coefficients over the channel blocks*
CHELPG, **Hirshfeld**, **Löwdin**, **Mulliken** charges	four ways of dividing a molecule's electron density among its atoms, all of which ORCA can report and any of which can become the electrostatic field's weights. They disagree, sometimes substantially, which is why the charge model is part of what must match between fitting and predicting. *Quantum-chemical properties with ORCA*
chemotype	a group of molecules sharing a scaffold. Splitting a benchmark by chemotype rather than at random is the test of whether a model carries beyond the series it was fitted on. *The example data sets*
co-field, $C_f(\mathbf{r})$	the model as a function of space: the training molecules' fields summed with the fitted coefficients, one per field type. Every prediction is the overlap of a molecule's own field with it. *The same prediction, as a field*; *Co-fields*
co-pattern	what the invariant kernels offer in place of a co-field — the weight of every channel of the fitted linear model, named by its field pair and distance, ranked by `top_channels`. *Crystals*; *Molecules on a surface*
contributions	the decomposition of one prediction over the training molecules, and of a training molecule's share over its atoms. *Contributions*

term	what it means here
count kernel	the pose-free baseline: a kernel on atom-type counts, with no geometry at all. Worth running before any aligned model, because it sometimes wins. *The pose-free count kernel*
decoys	presumed-inactive molecules used **only** to evaluate a screening model, never to fit it; property-matched decoys are the fair kind. *Decoys and the figures that matter*
`cmf diagnose`	the command that runs the five baselines — aligned, counts, autocorrelation, topomer, descriptors — on a data set before any model is built, and says which route to take. *Before you model*
dual coefficients, **a**, and **bias** b	what fitting produces: one coefficient per training molecule and one constant, from which every later prediction is a weighted sum of kernel values. *5. Fitting: what is learned from the training set*
ECFP4	a circular fingerprint of the atoms' environments out to two bonds: the standard 2D similarity measure, and the baseline the field similarity of this program is compared against on screening tasks, where it does about as well. *Decoys and the figures that matter*
ETKDG	RDKit's distance-geometry conformer generator, which builds the 3D structure whenever the input is 2D and supplies the starting conformations for the conformer ensembles, the flexible alignment and the topomer rebuilds. Its poses are arbitrary until they are aligned. *What the program reads*; *Conformations: the fourth dimension*
external cross-validation, q^2_{ex}	the whole model-building procedure, hyper-parameter search included, repeated inside each outer fold; the only figure that is not optimistic about the search. *Validation*
field, $X_f(\mathbf{r})$	a continuous function of space: one Gaussian per atom, weighted by that atom's charge, bulk, lipophilicity, H-bond strength, indicator value or file-borne property. The object CMF models. *1. The fields*
field scaling, c_f	the factor that puts a normalised field kernel's mean diagonal at 1 over the training set, so the fields enter the mixture on equal terms. Fixed on the training set and stored. *3. Normalisation and scaling*
field set	the list of field types a model uses, with their attenuations, charge model and parameterisation. It must match exactly between fitting and predicting. *What has to match between fitting and predicting*
flexible alignment	alignment that turns the rotatable bonds as well as moving the molecule, with the force field as a prior on the conformation and several generated starting conformations. The best pose method measured here: 0.86 Å median against the crystal poses. *Judging an alignment, and aligning flexibly*
Fukui index, f^+, f^-, f^0	a per-atom measure of where a molecule gives or takes electron density, computed from the N+1 and N−1 states by ORCA and used as a `prop:` field. The **dual descriptor** is $f^+ - f^-$. *Quantum-chemical properties with ORCA*
h, the **mixing coefficient** h_f	the weight of one field's normalised kernel in the joint kernel. Set by hand, optimised, or derived by kernel alignment. *4. Mixing the fields*
increments	the table of per-atom parameters a field is built from — the `rdkit` default or the `tripos` tables of the original CMF scripts. Not a setting of the method but of the chemistry it reads. *Appendix A*
indicator field	a field whose weight is 1 for atoms of a given type and 0 elsewhere, so that its co-field says where atoms of that type help. *Choosing the fields*

term	what it means here
joblib	the serialisation library a saved `.cmf` model is written with. A `.cmf` file is therefore a Python object graph, not an interchange format: it is read by this program and not meant to be read by another. *Saving and reusing*
joint kernel, $K(A,B)$	the sum of the normalised, scaled field kernels with their mixing coefficients: the single similarity the learner sees. *4. Mixing the fields*
Kabsch superposition	the least-squares rotation and translation that best superimposes two sets of matched points. It is the second half of an MCS alignment -- the substructure search finds the correspondence, this places the molecule -- and the pose benchmark measures how far the result lands from the crystal pose. *Alignment*; *Judging an alignment, and aligning flexibly*
kernel layer	a nonlinear function applied to the joint kernel — Gaussian or arc-cosine — after which there is no co-field, because the prediction is no longer linear in the fields. *Nonlinear kernel layers and multiple kernel learning*
MCS alignment	superimposing molecules on their maximum common substructure. The conventional frame, and the baseline every pose method here is measured against. *Alignment*
MKL	multiple kernel learning: deriving the mixing coefficients from the kernels and the activities by centred kernel alignment rather than setting them. *Nonlinear kernel layers and multiple kernel learning*
nested cross-validation	choosing the hyper-parameters inside each training fold and scoring outside it, so that the reported figure has not seen the molecules the choice was made for. *Validation*
normalisation	dividing a field kernel by the geometric mean of the two self-kernels, so every molecule has unit self-similarity and a large molecule cannot look important merely for being large. *3. Normalisation and scaling*
one-class model	a model fitted on actives alone, which asks whether a molecule resembles them rather than how active it is. *One-class models: screening with actives alone*
overlap integral, $K_f(A,B)$	the integral over all space of the product of two molecules' fields, which for Gaussians is a closed-form double sum over atom pairs. The kernel CMF is built on. *2. The overlap integral*
periodic kernel	the autocorrelation descriptor for a crystal: the pair histogram of one cell with its periodic images, per atom, with the uniform background removed and the cutoff tapered. *Crystals: the periodic autocorrelation kernel*
pharmacophore	a set of typed spheres extracted from the lobes of a co-field, which is the model's own statement of what a molecule should carry and where. *A pharmacophore out of the co-field*
probe field	a GRID-like interaction energy with a chosen probe, expanded in Gaussians so that it keeps the closed-form kernel and the co-field. *GRID-like probe fields*
`prop:` **field**	a field whose per-atom weights are read from the file or computed by ORCA rather than from a parameter table. *Charges and per-atom properties*

term	what it means here
q^2, R^2_{pred}, RMSE	the cross-validated, test-set and fit figures of merit. q^2 is not a test: it is what the search optimises, and a model can be chosen for it. *Validation*; *Statistics reported*
selectivity co-field	the difference of two endpoints' co-fields in a multi-task model, which draws where a molecule should differ to be selective rather than merely potent. *Several endpoints at once*
Shapley value	the share of a prediction attributed to each field or atom by averaging over all subsets, with the pairwise interaction indices beside it. *Shapley values*
solvent exposure weighting	multiplying each atomic weight by its relative solvent accessibility. Measured here and found not to help: ligands of this size have no interior. *Weighting the fields by solvent exposure*
surface kernel	the in-plane invariant kernel for molecules on a surface: distances in the plane, resolved by the heights of both atoms, invariant to lateral position and rotation about the normal and to nothing else. *Molecules on a surface*
topomer field	a field built from substituents rebuilt in a pose that is a deterministic function of their topology, so the same R-group always lands in the same place and no alignment is needed. *Topomer fields*
y-randomisation	refitting the whole procedure on shuffled activities many times; if the shuffled models score as well as the real one, the real one is chance. *Is it chance? y-randomisation*

Appendix D — Bibliography

References are grouped by what they are to this program: the papers that define the method it implements, those that apply it to one-class classification and virtual screening, and the work it takes methods or benchmark data from. Journal names follow the ACS abbreviations; a DOI is given wherever the work has one.

The method

[1] Zhokhova, N. I.; Baskin, I. I.; Bakhronov, D. K.; Palyulin, V. A.; Zefirov, N. S. Method of continuous molecular fields in the search for quantitative structure–activity relationships. *Dokl. Chem.* **2009**, *429*, 273–276. doi:10.1134/S0012500809110056

[2] Baskin, I. I.; Zhokhova, N. I. The continuous molecular fields approach to building 3D-QSAR models. *J. Comput.-Aided Mol. Des.* **2013**, *27*, 427–442. doi:10.1007/s10822-013-9656-4

[3] Baskin, I. I.; Zhokhova, N. I. Continuous molecular fields approach applied to structure–activity modeling. In *Application of Computational Techniques in Pharmacy and Medicine*; Gorb, L., Kuz'min, V., Muratov, E., Eds.; Challenges and Advances in Computational Chemistry and Physics, Vol. 17; Springer: Dordrecht, 2014; pp 433–459. doi:10.1007/978-94-017-9257-8_13

[4] Sitnikov, G. V.; Zhokhova, N. I.; Ustynyuk, Y. A.; Varnek, A.; Baskin, I. I. Continuous indicator fields: a novel universal type of molecular fields. *J. Comput.-Aided Mol. Des.* **2015**, *29*, 233–247. doi:10.1007/s10822-014-9818-z

[5] Baskin, I. I.; Zhokhova, N. I. Descriptors based on continuous indicator fields for 3D-QSAR studies. In *Proceedings of MOL2NET 2016, International Conference on Multidisciplinary Sciences*, 2nd ed.; MDPI: Basel, 2017. doi:10.3390/mol2net-02-03854

[6] Baskin, I. I.; Zhokhova, N. I. Continuous molecular fields and the concept of molecular co-fields in structure–activity studies. *Future Med. Chem.* **2019**, *11*, 2701–2713. doi:10.4155/fmc-2018-0360

One-class classification and virtual screening

[7] Karpov, P. V.; Baskin, I. I.; Zhokhova, N. I.; Zefirov, N. S. Method of continuous molecular fields in the one-class classification task. *Dokl. Chem.* **2011**, *440*, 263–265. doi:10.1134/S0012500811100016

[8] Karpov, P. V.; Baskin, I. I.; Zhokhova, N. I.; Nawrozkij, M. B.; Zefirov, A. N.; Yablokov, A. S.; Novakov, I. A.; Zefirov, N. S. One-class approach: models for virtual screening of non-nucleoside HIV-1 reverse transcriptase inhibitors based on the concept of continuous molecular fields. *Russ. Chem. Bull.* **2011**, *60*, 2418–2424. doi:10.1007/s11172-011-0372-8

Methods and data taken from other work

[9] Cramer, R. D.; Patterson, D. E.; Bunce, J. D. Comparative molecular field analysis (CoMFA). 1. Effect of shape on binding of steroids to carrier proteins. *J. Am. Chem. Soc.* **1988**, *110*, 5959–5967. doi:10.1021/ja00226a005

[10] Cramer, R. D. Topomer CoMFA: a design methodology for rapid lead optimization. *J. Med. Chem.* **2003**, *46*, 374–388. doi:10.1021/jm020194o

[11] Bemis, G. W.; Murcko, M. A. The properties of known drugs. 1. Molecular frameworks. *J. Med. Chem.* **1996**, *39*, 2887–2893. doi:10.1021/jm9602928

[12] Hopfinger, A. J.; Wang, S.; Tokarski, J. S.; Jin, B.; Albuquerque, M.; Madhav, P. J.; Duraiswami, C. Construction of 3D-QSAR models using the 4D-QSAR analysis formalism. *J. Am. Chem. Soc.* **1997**, *119*, 10509–10524. doi:10.1021/ja9718937

[13] Bishop, C. M.; Svensén, M.; Williams, C. K. I. GTM: the generative topographic mapping. *Neural Comput.* **1998**, *10*, 215–234. doi:10.1162/089976698300017953

[14] Cho, Y.; Saul, L. K. Kernel methods for deep learning. In *Advances in Neural Information Processing Systems 22 (NIPS 2009)*; Curran Associates: Red Hook, NY, 2009; pp 342–350.

[15] Rasmussen, C. E.; Williams, C. K. I. *Gaussian Processes for Machine Learning*; MIT Press: Cambridge, MA, 2006. doi:10.7551/mitpress/3206.001.0001

[16] Truchon, J.-F.; Bayly, C. I. Evaluating virtual screening methods: good and bad metrics for the "early recognition" problem. *J. Chem. Inf. Model.* **2007**, *47*, 488–508. doi:10.1021/ci600426e

[17] Sutherland, J. J.; O'Brien, L. A.; Weaver, D. F. A comparison of methods for modeling quantitative structure–activity relationships. *J. Med. Chem.* **2004**, *47*, 5541–5554. doi:10.1021/jm0497141

[18] Mittal, R. R.; McKinnon, R. A.; Sorich, M. J. Comparison data sets for benchmarking QSAR methodologies in lead optimization. *J. Chem. Inf. Model.* **2009**, *49*, 1810–1820. doi:10.1021/ci900117m

[19] Tran-Nguyen, V.-K.; Jacquemard, C.; Rognan, D. LIT-PCBA: an unbiased data set for machine learning and virtual screening. *J. Chem. Inf. Model.* **2020**, *60*, 4263–4273. doi:10.1021/acs.jcim.0c00155

[20] Wallach, I.; Heifets, A. Most ligand-based classification benchmarks reward memorization rather than generalization. *J. Chem. Inf. Model.* **2018**, *58*, 916–932. doi:10.1021/acs.jcim.7b00403

[21] Mordelet, F.; Vert, J.-P. A bagging SVM to learn from positive and unlabeled examples. *Pattern Recognit. Lett.* **2014**, *37*, 201–209. doi:10.1016/j.patrec.2013.06.010